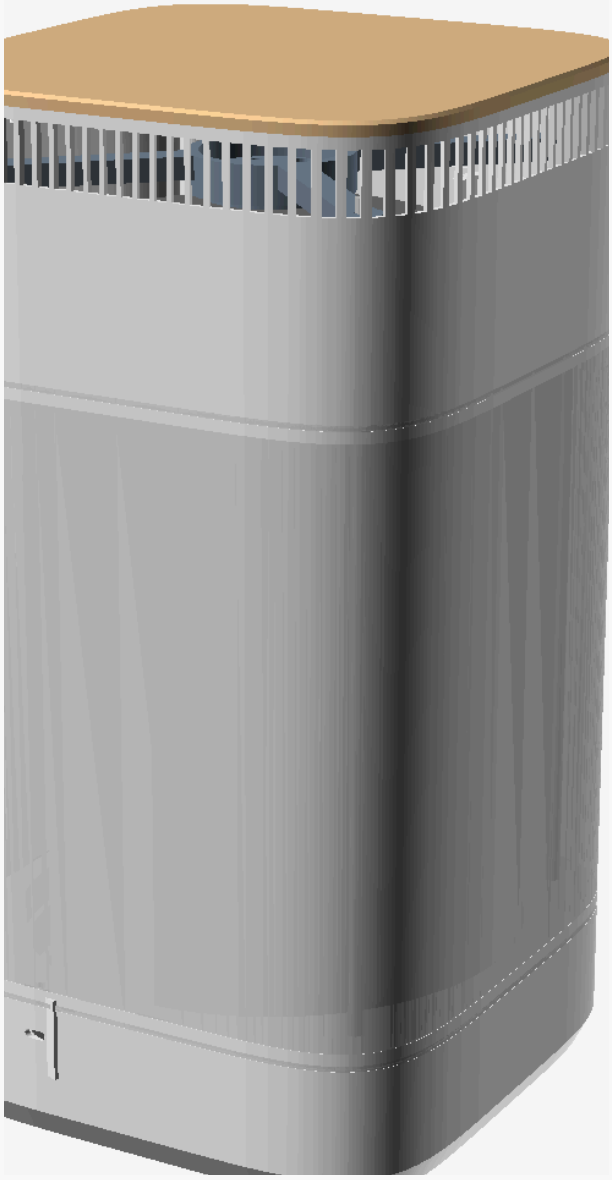
DESIGN DOCUMENT · V2.0

OpenAir

A 3D-printed, ESP32-S3 smart HEPA air purifier: a sub-A$200 Side Table and Value Tower, plus a 445 m³/h Reference Tower.

Side Table	Blueair-Signature-style squircle, fabric sleeve, ~245 m³/h, **~A$195**
Value Tower	Ø240 tower, same internals, ~245 m³/h, **~A$194**
Reference	ebm-papst R1G190 + SEN66 + TFT, ~445 m³/h, ~A$817
Filter	Xiaomi-compatible H13 + carbon, 200 × 293 mm (~A$18–30)
Power	USB-C PD 20 V (value) · RCM 24 V brick (reference); no mains inside
Control	ESPHome 2026.9 → Home Assistant, local web UI, touch + light ring
Print	PETG, 256 mm bed, 1.55–1.94 kg depending on the build

> Status: designed and verified in CAD and firmware compile only. No physical prototype yet, so CADR and noise are engineering estimates.



Contents

1 **Overview**
- Three builds
- Features (all builds)
- Repository layout
- Build order
- What has and hasn't been verified
- Safety

2 **Feature parity**
- Benchmarks sold in Australia
- The union of features, and where OpenAir lands

3 **Value build: under A$200**
- What changed, and what it costs you
- Expected performance
- Choosing the 160 mm blower
- Tower differences (value profile)
- Wiring summary (value)

4 **Side Table**
- Parts (`cad/stl/sidetable/`, export with `./render.sh sidetable`)
- Fabric sleeve
- Assembly
- Notes

5 **Airflow, CADR and blower choice**
- Target
- Why not PC fans
- System curve (estimate)
- Blower choice
- Blower requirements
- Expected performance
- Measuring your build

6 **Electronics**
- Design rules
- Power budget (24 V rail)
- Circuit blocks
- Pin map (ESP32-S3-DevKitC-1, N16R8 or N8R8)
- Perfboard layout (50 × 70 mm)
- Bring-up and calibration

7 **Firmware**
- Modes
- Touch pads (front arc of the top island, left to right)
- Display (1.28" round, LVGL)
- Air-quality ring (24 × WS2812B)
- Safety logic
- Filter life
- Home Assistant entities
- Calibration (substitutions at the top of `openair.yaml`)
- Building and flashing

8 **Printing**
- Material
- Per-part settings
- Inserts and hardware
- If your filter differs

9 **Assembly**
- 1. Inserts
- 2. Lower shell
- 3. Shell stack
- 4. Filter seal
- 5. Blower
- 6. Control unit (top cap)
- 7. First power-up (before the filter goes in)
- 8. Filter and base
- Changing the filter

10 **Parts gallery**

11 **Bill of materials**
- Totals
- Reference tower (ebm-papst R1G190, SEN66, TFT)
- Value tower (160 mm blower, PMS5003, USB-C)
- Side Table (value internals, fabric sleeve)
- Running cost

Appendix A: CAD parameters

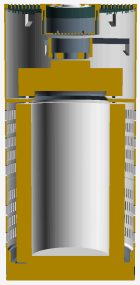
Appendix B: Firmware listing

Appendix C: OpenSCAD source

1 Overview

A 3D-printed ESP32-S3 air purifier with feature parity to the Levoit Core 400S, Xiaomi Smart Air Purifier 4 and Philips 2200/3200 class. It runs on **Home Assistant / ESPHome**, uses **cheap off-the-shelf Xiaomi-compatible filters**, and has **no mains wiring inside**.

Three builds

	Side Table	Value Tower	Reference Tower
			
Look	Blueair-Signature-style squircle side table, fabric sleeve, louvres under the top	Ø240 tower, perforated shell, top grille	same tower
Cost (AUD)	**~A$195**	**~A$194**	~A$817 (~A$533 with an AliExpress 190 mm blower)
Blower	160 mm 24 V EC fan (AliExpress)	same	ebm-papst R1G190 (datasheet-verified)
CADR (estimate)	~245 m³/h (USB-C 20 V) · ~275 (24 V brick)	same	**~445 m³/h**
Best room	~25 m² at 4 ACH · 35–40 m² at ~2.6 ACH	same	35–40 m² at 4.6 ACH
Sensors	PMS5003: PM1/2.5/10	same	SEN66: PM1–10, **CO_2**, VOC, NOx, T, RH
Display	10-LED light line + 5 touch pads	24-LED ring + 5 touch pads	round colour TFT + ring + 5 touch pads
Power	USB-C PD charger you own (≥ 45 W)	same	RCM 24 V 3 A brick
Size	250 × 250 × 443 mm	Ø240 × 467 mm	Ø240 × 507 mm
Filament	~1.55 kg PETG	~1.83 kg	~1.94 kg
Docs	§Side Table	§Value build: under A$200	§Airflow, CADR and blower choice, §Electronics

The value builds keep every software feature. You give up raw airflow (Levoit Core 300S class instead of 400S) and the gas/CO_2/climate sensing. You can **upgrade the blower later** by reprinting two small parts. Every price is broken down in §Bill of materials.

Feature comparison against 11 commercial models: §Feature parity.

Features (all builds)

- **Auto mode.** Smoothed PM2.5 sets the speed (plus VOC/NOx on the SEN66). It steps up fast and down slowly, has a configurable ceiling ("quiet auto"), and can optionally stop when the air is clean.
- **Modes: Sleep** (fixed low speed, lights and sounds off), **Turbo** (100 %, back to Auto after 30 min), and a **Manual** 1–100 % slider plus 5 touch steps.
- **Air-quality light** in six colours, auto-dimmed by ambient light. After each touch it turns into a **speed bar graph**. The reference build adds a 1.28" round TFT with PM2.5, CO_2, VOC, temperature and filter pages.
- **5 capacitive touch pads:** power, mode, speed, timer, lock. **Child lock**: hold the lock pad for 3 s.

- **Off timer** (1/2/4/8 h on the unit, 0–24 h from the app) and an on-device **night schedule**.
- **Filter life** counts both the air volume processed and the calendar age, whichever runs out first. It alerts at 10 %; reset it with a 5 s hold or from the app.
- **Safety interlock.** A hall sensor and magnet in the twist-off base stop the blower when the base is off.
- **Blower tach** with stall/fault detection, plus *estimated airflow*, *CADR* and *air changes per hour* sensors.
- **Integrations:** Home Assistant API (on to Alexa, Google, Apple Home and Matter via HA), local web UI, OTA updates. No cloud account.

Being straight about the money: a Levoit Core 300S costs about A$199 on sale, the same as the value builds. What DIY gets you:

- local control with no cloud account;
- a fully repairable and modifiable unit;
- a design you can grow (bigger blower, better sensor);
- generic filters at about A$18–30 instead of A$49–79.

Repository layout

```
cad/
  openair.scad          tower parts + build profiles (build = "reference" | "value")
  sidetable.scad        side-table parts (includes openair.scad, value profile)
  parts/*.scad          one entry file per printable part (+ assembly/preview files)
  stl/tower-reference/  ready-to-print STLs, print orientation
  stl/tower-value/
  stl/sidetable/        + top_template.svg (1:1 outline for a timber top)
  render/*.png          previews
  render.sh             ./render.sh [tower-reference|tower-value|sidetable|all] on the build box
  preview.sh            render a PNG of any part
  tools_components.py   checks each STL is a single solid
  tools_volume.py       filament estimate per STL
electronics/
  wiring.svg            reference build
  wiring-value.svg      value builds
firmware/
  openair.yaml            reference tower (SEN66, TFT, BH1750)
  openair-value.yaml      value tower (PMS5003, LED UI, LDR)
  openair-sidetable.yaml  side table (value + 10-LED line)
  packages/               core logic, UI (TFT / LEDs) and light-sensor packages
  sensors/                SEN66 or PMS5003 packages
  secrets.example.yaml
  check.sh                ./check.sh [config|compile] <entry> on the build box
bom/                    build_bom.py → bom.md + bom-<build>.csv
pdf/build_pdf.py        compiles every doc into OpenAir-Design.pdf (headless Chrome)
docs/
  feature-parity.md     commercial benchmarks and how each feature maps
  value-build.md        how the value builds hit < A$200, performance, wiring
  sidetable.md          side-table design, fabric sleeve, assembly
  airflow.md            CADR target, system curve, blower choice, how to measure yours
  electronics.md        reference circuit, pin map, perfboard layout, bring-up
  firmware.md           modes, auto curve, touch map, display, HA entities, calibration
  printing.md           materials, orientation, settings per part (tower)
  assembly.md           step-by-step tower build and filter changes
```

Build order

1. **Pick a build** from the table above.
2. **Buy the blower first.**
   - With the ebm-papst R1G190 (reference) everything is already set.
   - For any other blower, get the seller's drawing and enter it in the `[Blower]` block of `cad/openair.scad`, then re-export the inlet ring and bridge. See §Value build: under A$200, Choosing the 160 mm blower.
3. **Measure a filter** (height, outer diameter). If it isn't 200 × 293, set `filter_*` and re-export.
4. **Print** the matching STL set:

    - Tower: §Printing.
    - Side table: §Side Table.
5. **Build the perfboard and bring it up on the bench:**
    - Reference: §Electronics.
    - Value builds: §Value build: under A$200.
6. **Flash** the matching `firmware/openair*.yaml` (copy `secrets.example.yaml` → `secrets.yaml`), then calibrate the touch pads and blower RPM.
7. **Assemble:**
    - Tower: §Assembly.
    - Side table: §Side Table.
8. **Optional:** measure your real CADR with a decay test and calibrate the firmware (§Airflow, CADR and blower choice, Measuring).

What has and hasn't been verified

Verified:

- Every printable part of all three builds renders as a single valid manifold solid in OpenSCAD 2026.10.02, checked with `tools_components.py`.
- Parts were checked against each other in assembly and section renders. Collisions found that way were fixed.
- All three firmware builds pass `esphome config` and **compile** for the ESP32-S3 on ESPHome 2026.9.1.
- Component choices, pin rules and listed AUD prices come from manufacturer and retailer pages. BOM lines marked "estimate" are typical prices, not a specific listing.

Not verified (no physical prototype yet):

- **CADR and noise are engineering estimates.** No maker publishes a pressure curve for the Xiaomi cartridge, so the system curve is modelled. For the value builds, the 160 mm fan curve is also scaled from an ebm-papst model. Measure your unit.
- **The value blower's dimensions and wiring are placeholders** until you have the seller's drawing.
- **The reference blower geometry** comes from the ebm-papst R1G190-AB27-02 datasheet, whose curve was read off a chart (±10 %). The 66 mm back-face contact diameter isn't dimensioned on the sheet. AU stock and price weren't confirmed.
- **The filter's top-hole diameter** isn't documented; listings say 117–145 mm. The two-ring foam seal covers that range.
- **The sensor inlet/outlet layouts** for the SEN66 and PMS5003 windows come from text descriptions, not drawings.
- **Touch thresholds** depend on your print and pads. Calibrate them.

Safety

- **Reference build:** mains stays in the external RCM-approved 24 V brick.
- **Value builds:** they run from a USB-C PD charger (20 V).
- **Inside the purifier:** 24 V maximum, fused and TVS-protected.
- **Blower stop:**
    - Reference: the blower is hard-switched off at boot and on any interlock trip.
    - Value builds: the PWM line is held low during boot, and firmware drives it to 0 % on any interlock trip.
    - Don't defeat the interlock: the impeller is exposed with the base off.
- **Material:** use PETG or ASA, not PLA, near the motor.
- **Omitted on purpose:** ionisers, plasma and UV-C (ozone and by-products).

2 Feature parity

Researched 2026-10-04 from Australian retailer and manufacturer pages. AUD prices move weekly, so treat them as a snapshot. CADR figures are each maker's own and use different standards (GB/T 18801, AHAM, or unstated), so compare them loosely.

Benchmarks sold in Australia

Model	CADR (m³/h)	Noise (dB)	Power	Sensors	Display / UI	Smart home	Price (AUD)	Filter (AUD)
Xiaomi Smart Air Purifier 4 Pro	500	33.7–65	50 W	PM2.5, PM10, T, RH	OLED + colour strip, touch	Mi Home, voice	n/a	76–94
Xiaomi Smart Air Purifier 4	400	32.1 min	—	PM2.5, T, RH	OLED, touch	Mi Home	n/a	47–68
Levoit Core 300S	258	24–56	23–39 W	PM2.5 (laser)	touch, colour ring	VeSync, Alexa, Google	199	49
Levoit Core 400S	433	28–54	38 W	PM2.5 (laser)	touch, ring + PM2.5 number	VeSync, Alexa, Google	299	79
Levoit Core 600S	711	26–55	49 W	PM2.5, light	touch, ring + number, auto-dark	VeSync, Alexa, Google	459	119
Coway Airmega 250	423–443 (AHAM)	22–54	64 W	dust	AQ light	none	599	—
Blueair Blue Pure 211i Max	~697	23–53	46 W	PM1/2.5/10	touch	app, Alexa, Google	—	—
Philips PureProtect Quiet 2200 (AC2221)	420 (GB/T)	13–45	28 W	PM2.5	colour ring + number	Air+, Alexa, Google	295–449	—
Philips PureProtect 3200 (AC3220)	520	15–47.5	36 W	PM2.5, T, RH	colour ring + number	Air+, Alexa, Google	299–329	141–159
Dyson Purifier Cool TP07	not published (~160 measured by HouseFresh)	39–54 measured	~29 W measured	PM, VOC, NO_2, T, RH	LCD	MyDyson, Alexa, Google	869–949	99
IKEA STARKVIND (floor)	55–270	24–51	33 W	PM2.5	knob + LEDs	Zigbee via DIRIGERA → Apple Home, Alexa, Google	199	19 + 25 gas

None of the listings checked mentions native Matter, and only IKEA lists Apple Home (through its hub). None lists a CO_2 sensor; formaldehyde sensing appears only in Dyson's TP09.

The union of features, and where OpenAir lands

Must-have (every mid-range unit does this)

Feature	Typical implementation	OpenAir
CADR ~350–450 m³/h for a 35–40 m² room	DC backward-curved centrifugal fan, 30–50 W	24 V EC backward-curved blower (ebm-papst R1G190 reference): ~445 m³/h estimated, see §Airflow, CADR and blower choice
HEPA + activated carbon filter	proprietary cartridge	Xiaomi-compatible 200 × 293 mm cartridge (cheapest, most available in AU)

Feature	Typical implementation	OpenAir
Laser PM2.5 sensor	Plantower or Cubic OEM	Sensirion SEN66 (PM1/2.5/4/10)
Auto mode	PM2.5 → fan step table	PM2.5 + VOC-index step table with hysteresis and slew limiting
Sleep / night mode	low cap, lights off	caps speed, lights off, buzzer muted; optional schedule
Manual speeds + turbo	3–5 steps	5 touch steps plus a 1–100 % slider in the app
Air-quality display	colour ring + PM2.5 number	1.28" round colour TFT + 24-LED ring
Touch controls	capacitive buttons	5 capacitive pads (ESP32-S3 native touch)
Child lock	long-press	long-press the lock pad; also a switch in HA
Display dim / off, auto-dark	light sensor (600S, Philips)	BH1750 ambient light → dims ring and screen, off in the dark if wanted
Timer	1–12 h	1, 2, 4, 8 h on the unit; 0–24 h from the app
Schedules	in app	on-device night schedule plus any HA automation
Filter-life tracking and alert	hours used, sometimes RFID	volume-weighted hours **and** calendar age, whichever runs out first; alert at 10 %; reset by long-press or app
App + voice assistants	vendor cloud app	ESPHome native API → Home Assistant → Alexa, Google, Apple Home, Matter via HA bridges. No cloud needed.
Door / filter interlock	microswitch	hall sensor + magnet in the twist-off base: the blower stops if the base is off
Audible feedback	buzzer	passive buzzer, muted in sleep mode

Nice-to-have (premium models), included

- PM1 / PM4 / PM10, VOC index, NOx index, temperature, humidity: all from the SEN66.
- **CO_2** (SEN66 has an on-board photoacoustic sensor). No consumer purifier above shows it.
- Local web UI on the device (ESPHome `web_server`), so it works without Home Assistant.
- OTA updates, captive-portal Wi-Fi setup.
- Blower tachometer with fault detection (stall or disconnected blower → red ring + beep + HA alert).
- Estimated live airflow and "room air changes per hour" sensor.

Deliberately left out

- **Ionisers / plasma / UV-C**: ozone and by-product risk for little measured benefit. Several of the benchmarks also skip them.
- **RFID filter recognition**: genuine Xiaomi filters carry an NTAG213 tag on the bottom cap, but generics often don't. The tag sits in the twist-off base, where a reader would need wiring across the bayonet. Filter life is tracked in firmware instead.
- **Formaldehyde sensing**: optional. A Sensirion SFA30 fits on the same I²C bus; see §Electronics.

3 Value build: under A$200

The value build is the same purifier with the expensive parts swapped out. It comes as two enclosures:

- the **Value Tower**: the original tower, re-profiled;
- the **Side Table**: the Blueair-Signature-style enclosure, see §Side Table.

Both come to about **A$195** (see §Bill of materials), assuming you reuse a USB-C PD charger you already own.

What changed, and what it costs you

Area	Reference	Value	Saving	Trade-off
Blower	ebm-papst R1G190, 190 mm, ~A$400	160 × 45 mm 24 V EC backward-curved fan, ~A$45 (AliExpress)	~A$355	~245 vs ~445 m³/h CADR. Specs are the seller's, not a datasheet.
Air sensor	Sensirion SEN66, A$99	Plantower PMS5003, ~A$22	~A$77	PM1/2.5/10 only: no CO_2, VOC, NOx, temperature or humidity
Power	24 V 3 A RCM brick + IEC lead + chassis jack, A$58	USB-C PD trigger at 20 V, ~A$3, plus a ≥ 45 W USB-C charger you own	~A$55	Ties up a charger. Blower runs at 20 V (≈ 83 % of rated speed).
5 V supply	Pololu D36V28F5, A$36	LM2596 module, A$2.95	~A$33	Bigger and less efficient. Set it to 5.0 V before connecting anything.
Display	1.28" round TFT + backlight	none: the LED ring / light line shows air quality and a speed bar graph	~A$6 + simpler wiring	Readings in Home Assistant or the web UI only
Light sensor	BH1750	LDR + 10 k on an ADC pin	~A$3	Approximate lux, but enough for auto-dimming
Blower switch	P-MOSFET high-side switch	none. A 4.7 k gate pull-up holds the PWM line low while the ESP32 boots.	~A$8	~0.5 W standby in the blower. "Off" relies on PWM 0 % stopping the fan, so check yours.
LED level shift	74AHCT125	1N4001 diode drop on the LED supply	~A$1.40	none in practice
SEN66 LDO	LD1117V33	not needed	~A$2	n/a
Fasteners	brass heat-set inserts	M3 self-tapping screws into 2.8 mm pilots (`use_inserts = false`)	~A$11	Fine for occasional disassembly. Don't over-tighten.
Connectors	JST-XH kit, stripboard	Dupont/JST pins, plain perfboard	~A$12	n/a
Touch pads	copper tape	aluminium foil tape / kitchen foil	~A$5	n/a
Filament	30 mm solid base	20 mm base at 6 % infill, shorter head (blower_max_h 65)	~A$3	n/a

Expected performance

There's no datasheet for the 160 mm fan, so it is modelled by scaling the ebm-papst R1G175 curve with the fan laws ($Q \propto n \cdot D^3$, $\Delta p \propto n^2 \cdot D^2$) and intersecting it with the same filter system curve as the reference (§Airflow, CADR and blower choice).

Supply	Max fan speed (assumed)	Airflow	CADR	ACH, 25 m² × 2.4 m	ACH, 40 m² × 2.4 m	Max noise (est.)
20 V USB-C PD	~2600 rpm	~250 m³/h	**~245 m³/h**	4.1	2.6	~50 dB(A)
24 V brick	~3000 rpm	~285 m³/h	~275 m³/h	4.6	2.9	~53 dB(A)

That is Levoit Core 300S territory (258 m³/h, AU figure). It's comfortable for a bedroom or a 25 m² room, and fine for everyday dust and pollen in your 35–40 m² living room. During a bushfire-smoke event in the big room, though, it clears the air about 40 % slower than the reference.

Upgrade later without reprinting the body:

1. Buy a 190 mm blower: an AliExpress EC fan (~A$117) or the ebm-papst R1G190.
2. Set the `[Blower]` block, then re-export `inlet_ring` and `blower_bridge` (Side Table: `st_inlet_ring`, `st_bridge`).
3. Power it from a 24 V 3 A brick. The Value Tower head is shorter (`blower_max_h` 65), so a blower taller than 65 mm needs the reference `head_body`.

Choosing the 160 mm blower

AliExpress lists 24 V EC backward-curved fans around 160 × 45 mm from about A$41 (e.g. item 1005013134063403). **Before you buy, get from the seller:**

- a dimensioned drawing: impeller Ø, eye Ø, inlet edge to back face, motor flange bolt circle and thread;
- the wiring: it must have a **PWM** (or 0–10 V) speed input and an **FG/tach** output; 2-wire fans won't do;
- the PWM spec (frequency, and polarity: does 0 % stop it?), FG pulses per revolution, and the supply range (does it run at 20 V?);
- a P-Q curve if they have one. You want ≥ 280 m³/h at ≥ 90 Pa at 24 V.

Enter the drawing numbers in the `[Blower]` block of `cad/openair.scad` (the value branch of each `is_value ? … : …`) and the electrical ones in the substitutions of `firmware/openair-value.yaml`. **The values shipped there are placeholders.**

Tower differences (value profile)

`./render.sh tower-value` exports these to `cad/stl/tower-value/`:

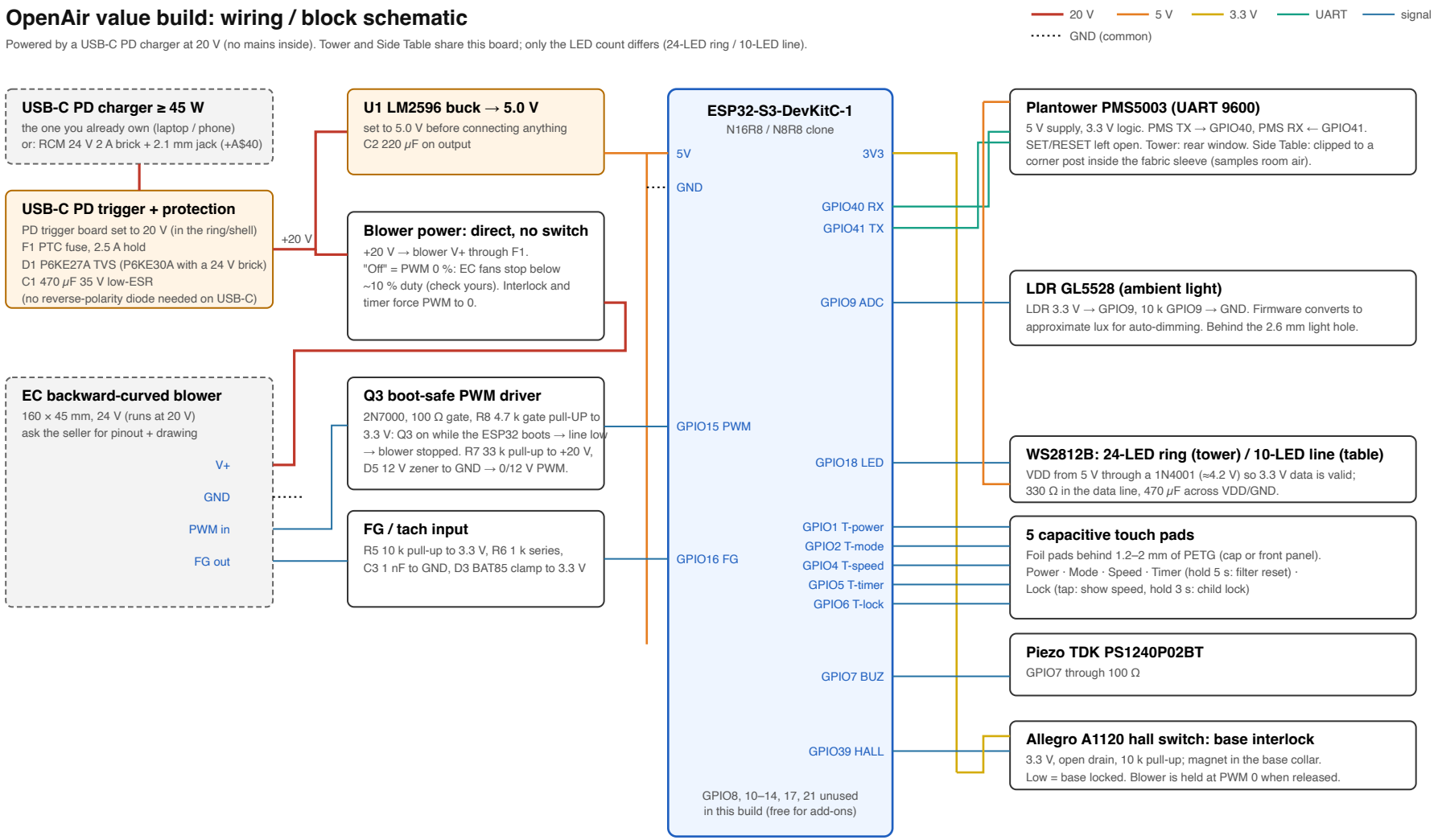
- **Power input:** a 9.8 × 4 mm USB-C slot replaces the DC-jack hole on the rear spine. Glue the PD trigger board behind it with its receptacle flush.
- **Sensor window** sized for the PMS5003 (38 × 50 mm face, 21 mm deep), held by the same clip.
- **No inserts:** every insert hole becomes a 2.8 mm pilot for M3 self-tapping screws.
- **Head** 30 mm shorter (139 mm), inlet ring sized for a 110 mm eye.
- **Base** 20 mm tall instead of 30. Print it at 6 % infill.
- **Control disc** carries an embossed "OpenAir" in place of the display window. The ring still shows air quality, with a speed bar graph after each touch.

Firmware: flash `firmware/openair-value.yaml` (tower) or `firmware/openair-sidetable.yaml` (side table).

Wiring summary (value)

OpenAir value build: wiring / block schematic

Powered by a USB-C PD charger at 20 V (no mains inside). Tower and Side Table share this board; only the LED count differs (24-LED ring / 10-LED line).



Perfboard 50×70 mm, Dupont/JST connectors: power 2, blower 4, PMS 4, hall 3, LEDs 3, LDR 2, touch 5. 20 V in 22 AWG, logic in 26 AWG.
Pin map is defined in ../firmware/packages/*.yaml. Parts and prices: ../bom/bom-value.md.

Wiring / block schematic (electronics/wiring-value.svg)

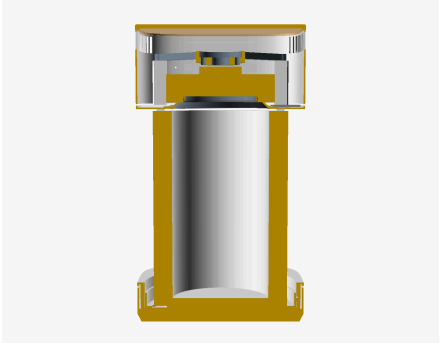
From	To	Notes
USB-C PD trigger 20 V +	PTC 2.5 A → +20 V rail	P6KE27A TVS and 470 µF across the rail
+20 V rail	blower V+, LM2596 in	
LM2596 out (5.0 V)	DevKitC 5V, PMS5003 VCC, 1N4001 → LED VDD	
GPIO15	100 Ω → 2N7000 gate; 4.7 k gate → 3.3 V	the drain pulls the blower PWM line; 33 k from the line to +20 V, 12 V zener from the line to GND
GPIO16	blower FG	10 k to 3.3 V, 1 k series, 1 nF, BAT85 clamp
GPIO40 / GPIO41	PMS5003 TX / RX	
GPIO9	LDR to 3.3 V, 10 k to GND	
GPIO18	330 Ω → LED DIN	470 µF across the LED supply
GPIO1, 2, 4, 5, 6	touch pads	
GPIO7	100 Ω → piezo	
GPIO39	A1120 output	10 k pull-up to 3.3 V

4 Side Table

A second enclosure for the value internals, inspired by the Blueair Blue Signature:

- a soft **fabric sleeve** on all four sides doubles as the 360° intake and the washable pre-filter;
- air leaves through a **louvre band under the tabletop** on the back and both sides;
- the **flat top** takes a plant, a lamp or a cup of tea;
- the front carries a **light line** and five **touch pads**.

It is a squircle (superellipse, n = 5), the largest one-piece footprint a 256 mm bed allows.

With sleeve	Frame without sleeve	Cutaway
		

	Side Table	Blueair Blue Signature Large, for scale
Size	250 × 250 × 443 mm	437 × 393 × 366 mm
CADR	~245 m³/h on USB-C 20 V (~275 on a 24 V brick), estimate	434–455 cfm (≈ 740–770 m³/h)
Intake / outlet	360° fabric sleeve / louvres on 3 sides under the top	360° / side
Noise	~25 dB(A) sleep, ~50 max (estimate)	23–55 dB
Cost	**~A$195** (§Bill of materials)	~A$700 (€429–450 on blueair.co/en-au)

Same internals as the Value Tower:

- the Xiaomi-compatible filter;
- the twist-lock base with the hall interlock;
- the two-ring foam seal on the deck;
- the inlet ring and blower bridge;
- the PMS5003, the ESP32-S3 and the firmware (`firmware/openair-sidetable.yaml`, which sets 10 LEDs).

Parts (`cad/stl/sidetable/`, export with `./render.sh sidetable`)

Part	Qty	Print	Notes
`base`	1	flat, 6 % infill	Squircle plinth with a chamfered toe, round bayonet collar inside. Twist ~35° to unlock: the corners going out of line show the base is unlocked.
`ring`	1	upright, 15 %	Bayonet J-slots, hall pocket, USB-C slot + board pocket (back, left of centre), elastic groove, post sockets. Hollow from below.
`post`	**4**	lying flat, **diagonally** (285 mm), 4 walls	12 mm square corner posts. One M3 × 16 self-tapper at each end.
`head`	1	upright (deck down), 15 %	Louvre band, front light line (0.8 mm skin) + 5 pad recesses + icons + LDR hole, card guides for the board, 4 columns for the posts and bridge, ribbed 2.4 mm deck with the seal grooves.
`inlet_ring`	1	flange down	Same part as the Value Tower
`bridge`	1	flipped (spokes down)	Hub for the blower, screws to the column tops

Part	Qty	Print	Notes
`top`	1	top face down, 15 %	Or cut it from 12–18 mm timber with `top_template.svg` (1:1)
`pms_clip`	1	flat	Cradle zip-tied to the back-right post, sensor face toward the fabric

Filament: about 1.55 kg. Print the **head in white or light-grey PETG** so the LED line glows through the 0.8 mm skin. The ring, base and posts can be any colour; a single light "fog" colour looks closest to the Blueair. A wood-effect PETG or a real timber top finishes it off.

Fabric sleeve

The sleeve is the intake and the pre-filter, so the fabric must be **breathable**: hold it up to a lamp and you should see light through it. Good choices:

- polyester stretch knit (jersey);
- acoustic speaker-grille cloth;
- a sports-mesh fabric.

Avoid felt, fleece and anything coated.

	Size
Frame perimeter at the surface / in the grooves	899 mm / 884 mm
Groove to groove (visible height)	279 mm
Cut the fabric	**900 × 330 mm** (stretch along the 900 mm)
Seam	10 mm allowance → finished tube ~880 mm round, slightly under the frame so it sits taut
Hems	fold 20 mm at top and bottom into a channel and thread 3 mm round elastic. Tie each elastic loop at ~800 mm so it snaps into the 2.5 mm grooves.

To fit or wash it:

1. Slide the tube over the head and pop the bottom elastic into the ring's groove.
2. Pull it up and pop the top elastic into the groove just above the head's deck.

Washing takes 30 seconds to remove; then wash cold and dry flat. Air passes the fabric at about 0.3 m/s, so the fabric adds a negligible pressure drop and catches hair and lint before they reach the HEPA.

Assembly

1. **Ring**
   - Glue the A1120 hall sensor into the bayonet band pocket. Same as the tower: check polarity before gluing the magnet into the base.
   - Push the USB-C PD trigger board up into its pocket from below, receptacle in the slot, and glue it.
   - Run the 20 V and hall wires up through the hole beside the pocket.
2. **Posts.** Push the four posts into the ring sockets. Fix each with an M3 × 16 self-tapper driven up from under the ring (base off).
3. **PMS5003.** Clip it into its cradle and zip-tie the cradle to the back-right post at mid-height, sensor face out toward the corner. Run its cable up the post.
4. **Seal.** Stick the two foam rings into the grooves under the head's deck.
5. **Head.**
   - Lower it onto the posts. The posts enter the sockets under the four columns.
   - Fix each post with an M3 × 16 driven down the column's long bore (a long, thin screwdriver).
   - Feed the cables through the deck wire pass at the back-left and seal it with hot glue.
6. **Blower.**

    - Screw the inlet ring into the counterbore, with a thin silicone bead.
    - Bolt the blower to the bridge hub.
    - Lower the bridge so the eye slides over the nozzle lip, and screw it to the four column tops: M3 × 12 into the pilots 7 mm inboard of each post.
7. **Front.**
    - Stick the 10-LED strip behind the light line, LEDs facing out.
    - Stick foil pads into the five recesses and the LDR behind its hole.
    - Slide the perfboard down the card guides, components facing inward.
8. **Sleeve and top.** Fit the sleeve (above) and lay the tabletop on the rim; its lip locates it.
9. **Filter.**
    - Stand the filter in the base, open end up.
    - Lower the body over it and twist to lock: about 35° clockwise seen from above, until the squircle corners line up.

Filter change: twist the base anticlockwise and lift the body off; the sleeve stays on.

Notes

- **Air path:** room air enters through the fabric, passes radially through the HEPA cylinder, rises through the deck nozzle into the blower, then turns up and out of the louvres under the top. The louvres sit on the back and sides only, so the jet never blows at you from the front.
- **Keep about 10 cm behind and beside** the unit clear. Don't stand anything flush against the louvres.
- **Tabletop load:** the posts and columns take far more than a pot plant (each 12 mm PETG post buckles at roughly 500 N). Keep heavy items, and water, away from the louvres.
- **The Blueair has a touchscreen; this has touch pads and a light line.** Readings and history live in Home Assistant or the local web UI.

5 Airflow, CADR and blower choice

Target

A 35–40 m² living room with a 2.4 m ceiling is about 96 m³ of air.

- **4–5 air changes per hour** (the usual recommendation for allergies and smoke) needs a **CADR of 380–480 m³/h**.
- That puts OpenAir in the class of the Xiaomi 3H (380), Xiaomi 4 (400), Levoit Core 400S (433, AU figure) and Philips AC2221 (420).
- CADR ≈ airflow × single-pass efficiency. An H13-class filter catches ~97 %+ of PM2.5 per pass, so CADR ≈ 0.97 × airflow.
- The blower therefore has to move **~400–450 m³/h through the filter**.

Why not PC fans

Build	Measured result	Source
One cylindrical H13 cartridge + 120 mm PC fans	**~16 m³/h CADR** in a smoke-decay test	Kels316 / Hackaday 2020
9× Arctic P14 on a 0.6 m² flat HEPA panel	~430 m³/h CADR, 45 dB(A), 15 W	RevSpace GhettoBlast
AirFanta 3Pro: 4 PC fans, 4 flat H11 panels	353 cfm (600 m³/h) PM1 CADR at 56 dB(A)	HouseFresh

PC fans work when they push through a *large* filter area at low face velocity, as Corsi-Rosenthal-style boxes do. A Xiaomi cartridge packs ~2 m² of media into a 200 mm cylinder whose only exit is a ~140 mm hole. That needs a few hundred pascals at the operating point, and an axial PC fan tops out at 20–100 Pa. Xiaomi's own units reach 310–380 m³/h through this filter using a backward-curved centrifugal fan drawing 30–40 W, and so does OpenAir.

System curve (estimate)

No maker publishes a pressure-drop curve for these cartridges, so this estimate is built from the media and checked against Xiaomi's figures.

Filter (the dominant resistance):

- Xiaomi quotes 22,000 cm² of media for the 4-series filter; the 2/3 series is similar.
- At 420 m³/h that is a face velocity of about 5.3 cm/s, the standard rating velocity for HEPA media.
- Electret PP media (what Xiaomi uses) runs roughly 60–100 Pa clean at that velocity. The granular-carbon layer adds 10–30 Pa.
- Media resistance is close to linear in airflow, so **ΔP_filter ≈ 0.25–0.30 Pa per m³/h**, about 105–125 Pa at 420 m³/h.

Everything else (roughly quadratic in airflow), at 420 m³/h:

Element	Loss
Shell slots (~0.065 m² open, ~1.8 m/s)	~2 Pa
Deck mouth + 45° converging nozzle (Ø136 → Ø124)	~8 Pa
Plenum turn + top grille (~0.023 m² open, ~5 m/s)	~20 Pa
Subtotal	**~30 Pa**

Operating point: about 420 m³/h at 135–155 Pa with a clean filter. As the filter loads, expect 1.5–2× the pressure at the same flow.

Air power = 0.117 m³/s × 145 Pa ≈ **17 W**. A small EC plug fan in a cramped plenum is perhaps 30–40 % efficient overall, which means **45–55 W electrical** at full speed. That is consistent with Xiaomi's 38 W (3H) and 50 W (4 Pro).

Blower choice

Two ebm-papst 24 V EC backward-curved "plug fans" have full public datasheets and suit this filter. Their Ø125.5 inlet throat sits inside the filter's ~140 mm open top.

	R1G190-AB27-02 (reference)	R1G175-AB63-02
Impeller	Ø190, 67 mm to the inlet edge, eye Ø131	Ø175, ~69 mm overall, eye Ø131
Motor	24 V (16–28 V), 51 W, 2.3 A, 2950 rpm, 1.0 kg	24 V, 34 W, 1.6 A, 3100 rpm, 0.74 kg
Mounting	4 × M4 in the motor back face on a Ø58 bolt circle, max 6 mm deep	same
Curve, 24 V (m³/h @ Pa)	≈612 @ 0, ≈495 @ 127, ≈375 @ 230, ≈205 @ 330 (read off the chart, ±10 %)	555 @ 0, 435 @ 100, 300 @ 200, 145 @ 275
Control	yellow: PWM 1–10 kHz or 0–10 V; white: tach, open collector	same; tach 2 pulses/rev
Noise (catalogue)	68 dB(A)	65 dB(A)
Price	US$244–282 (Sager, out of stock); RS stock no. 825-7608	US$247–286; RS 825-7591
Datasheet	r1g190ab2702eng.pdf	R1G175-AB63-02

Operating points against the system curve above (0.27 Pa per m³/h plus 30 Pa × (Q/420)²):

Blower	Operating point, clean filter	CADR	Notes
R1G190	**≈ 460 m³/h at ≈ 160 Pa, ~55 W**	**≈ 445 m³/h**	Reaches 400 m³/h at ~87 % speed: quieter, and leaves headroom as the filter loads
R1G175	≈ 405 m³/h at ≈ 135 Pa, ~39 W	≈ 395 m³/h	On target, no headroom; fine for ~30 m²
1× Noctua iPPC-3000 140 mm	≈ 130–165 m³/h	≈ 130–160	For comparison: why PC fans fall short

The model assumes a linear filter resistance of 0.27 Pa per m³/h. With a lower value (the fan agent's 0.25), the R1G190 point rises to ~495 m³/h.

Cheaper blowers. AliExpress lists 190 mm 24 V EC backward-curved fans with 0–10 V/PWM control for about A$115–120 (e.g. item 1005013146632072). Their listings don't publish dimensions, curves or pinouts. **Ask the seller for a drawing** before buying (impeller Ø, eye Ø, flange bolt circle and thread, P-Q curve, wire colours, PWM frequency, tach pulses/rev). Check it can do ≥ 450 m³/h at ≥ 150 Pa, then enter its numbers in the `[Blower]` block and re-export `inlet_ring` and `blower_bridge`.

Blower requirements

Spec	Requirement
Type	EC (brushless, integrated driver) backward-curved centrifugal, "plug fan" (no scroll housing)
Impeller	175–200 mm (the head's inner diameter is 234 mm)
Duty point	≥ 420 m³/h at ≥ 150 Pa, i.e. ~700+ m³/h free air and ≥ 400 Pa shut-off
Supply	24 V DC, ≤ 60 W
Control	PWM speed input (1–10 kHz, 0–100 %), FG/tach output
Mounting	stator flange on the motor side (hung from the bridge), inlet facing down

The head body is universal: it fits any plug fan up to Ø200 mm and 95 mm from inlet edge to back face. Only two small parts follow your blower. Enter its numbers in the `[Blower]` block of `cad/openair.scad` (impeller Ø, inlet edge to back face, eye Ø, rotor-can Ø, bolt circle, hole count) and re-export:

- `inlet_ring`: the nozzle lip goes 2.5 mm inside the eye with 1.5 mm radial clearance.
- `blower_bridge`: the hub drops to meet the motor's back face, bearing only on the stationary contact diameter.

Expected performance

With the R1G190:

Speed	Airflow (est.)	CADR (est.)	ACH in 96 m³	Blower power (est.)	Noise at 1 m (est.)
Sleep, 15 %	~70 m³/h	~65	0.7	~1 W	~25 dB(A)
Auto, typical 35 %	~160	~155	1.6	~3 W	~35 dB(A)
55 %	~255	~245	2.6	~10 W	~45 dB(A)
~87 %	~400	~390	4.1	~36 W	~54 dB(A)
Turbo, 100 %	~460	~445	4.6	~55 W	~57 dB(A)

Airflow scales roughly linearly with speed on this mostly linear system curve, and power roughly with its cube. The noise column starts from ebm-papst's 68 dB(A) catalogue figure, which is probably a sound *power* level: about 11 dB lower as sound pressure at 1 m, and about 15 dB less per halving of speed. Turbo is therefore a little louder than the Levoit 400S's 54 dB maximum, while Auto, where the unit spends most of its time, is quieter than most of the benchmarks. **These are engineering estimates, not measurements.** The next section shows how to measure your own build.

Measuring your build

CADR by decay, which is how AHAM-style figures are produced:

1. Close a room of known volume V and raise PM2.5 to 150+ µg/m³ (burn a match or a stick of incense away from the unit).
2. With the unit **off**, log PM2.5 from a *separate* sensor for 10 minutes. This gives the natural decay rate k_n (per hour), the slope of ln(PM) against time.
3. Run the unit at the setting under test and log again to get k_p.
4. **CADR = V × (k_p − k_n)**.

Home Assistant's history graph of the unit's own SEN66 works too, but a separate sensor across the room is more honest.

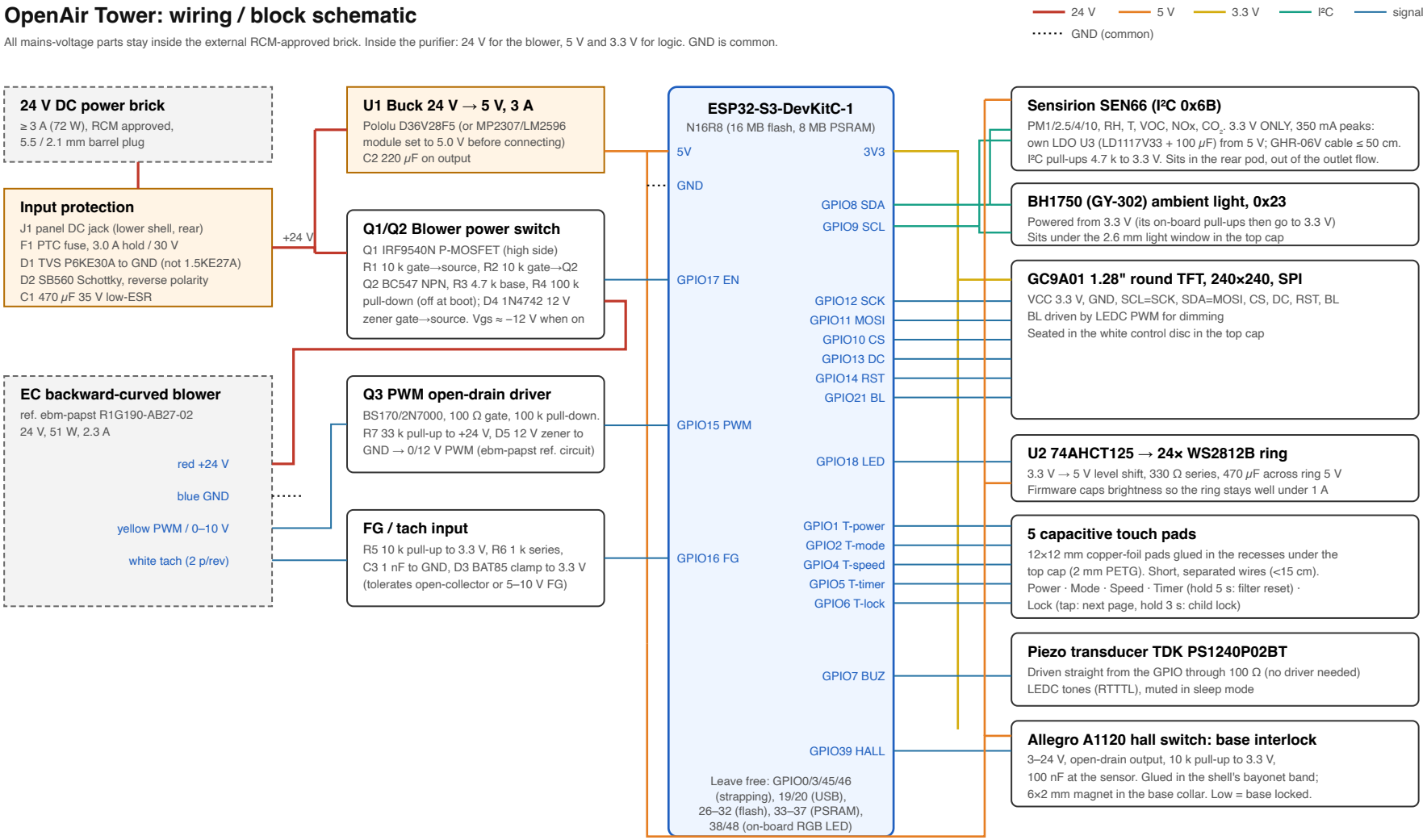
Calibrating the firmware's estimate:

- With a clean filter, run at 100 % and read *Blower speed*. Enter it as `blower_max_rpm`.
- Enter `CADR_at_100% / filter_efficiency` as `airflow_at_max`.
- The *Estimated airflow*, *Estimated CADR* and *Air changes per hour* sensors and the volume-based filter-life counter are then calibrated to your unit.

6 Electronics

OpenAir Tower: wiring / block schematic

All mains-voltage parts stay inside the external RCM-approved brick. Inside the purifier: 24 V for the blower, 5 V and 3.3 V for logic. GND is common.



Perfboard (50×70 mm) connectors, JST-XH 2.54 mm: power 2, blower 4, SEN66 4, hall 3, LED ring 3, BH1750 4, display 8, touch 5. 24 V in 20 AWG, logic in 26 AWG.
Full part list with prices: ../bom/bom.csv. Pin map is defined in ../firmware/openair.yaml.

Wiring / block schematic (electronics/wiring.svg)

Design rules

- **No mains inside the printed enclosure.** An external RCM-approved 24 V brick feeds a panel jack on the rear of the lower shell. Everything inside is 24 V or lower.
- **24 V for the blower, 5 V for LEDs and logic, 3.3 V for sensors.** All grounds are common.
- **The blower has a hard power switch**, so "off" means off: it boots off and stops if the ESP32 resets.
- **One perfboard, plug-in modules.** The ESP32-S3 DevKitC, buck module and LDO sit in sockets or headers, and every off-board item uses a JST-XH connector. The tray and top cap lift off as one unit.

Power budget (24 V rail)

Load	Typical	Peak
EC blower (ebm-papst R1G190: 51 W nominal, 2.3 A)	25–40 W at 80–90 %	~56 W at 100 %
ESP32-S3 + Wi-Fi	0.4 W	1.7 W (5 V × 340 mA TX bursts)
24 × WS2812B ring (firmware caps brightness)	0.3–1 W	7 W if forced to full white
SEN66	0.3 W	350 mA × 3.3 V for 2 ms
Display + backlight, BH1750, hall, buzzer	0.3 W	0.5 W
Buck + LDO losses	0.3 W	1 W
Total	**~30–45 W**	**~65 W (2.7 A)**

Use a **24 V, 3 A (72 W) or larger** brick. A 2.5 A brick is fine only with the smaller R1G175 (34 W). Standby (blower switched off, display dimmed) is about 0.7 W.

Circuit blocks

Input

- **J1**: Altronics P0622 2.1 mm metal chassis jack (5 A at 24 V, 8 mm hole). Match it to your brick's plug: a 2.5 mm plug in a 2.1 mm jack is loose.
- **F1**: PTC fuse, 3.0 A hold / 30 V (e.g. Littelfuse RXEF300), or a 4 A fast blade fuse.
- **D2**: SB560 Schottky in series for reverse-polarity protection, about 1.4 W of loss at full load. Leave room around it on the board.
- **D1**: P6KE30A TVS across the rail. Don't use a 1.5KE27A: its 23.1 V stand-off is below 24 V, so it would conduct.
- **C1**: 470 µF 35 V low-ESR across the rail.

5 V: U1 buck

Pololu **D36V28F5** (5.3–50 V in, about 3 A out) is the recommended part. A budget LM2596 module also works, but **set it to 5.0 V before you connect anything**. Avoid MP1584 and DFR0571 modules: their 28 V maximum input leaves too little margin on a 24 V rail. Add 220 µF on the output.

3.3 V: two rails

- The DevKitC's own LDO powers the ESP32, display logic, BH1750 and pull-ups.
- **U3**, an LD1117V33 with 10 µF in and 100 µF out, powers **only the SEN66** from 5 V. The SEN66 needs 3.15–3.6 V with less than 30 mV ripple and draws 300–350 mA peaks for 2 ms, which would sag the shared LDO.

Blower

Hard power switch (Q1/Q2):

- Q1: IRF9540N P-MOSFET on the high side.
- Gate: R1 10 k from gate to source, R2 10 k from gate to the Q2 collector. This divider gives Vgs ≈ −12 V.
- D4: 1N4742 12 V zener from gate to source, as a clamp.

- Q2: BC547 NPN with a 4.7 k base resistor from GPIO17 and a 100 k pull-down, so the blower stays off while the ESP32 boots.

PWM (Q3):

- BS170 or 2N7000 as an open-drain driver: 100 Ω gate resistor from GPIO15, 100 k pull-down.
- **ebm-papst-style 0–10 V / PWM inputs (yellow wire) have no internal pull-up** (input impedance > 100 kΩ). Their reference circuit is an open collector with **R7 33 k pull-up to the 24 V rail** and **D5, a 1N4742 12 V zener, from the line to GND**. That gives a 0/12 V PWM signal.
  - **Don't pull up to 5 V.** These inputs treat amplitude as part of the command, so a 5 V swing caps the blower at about half speed.
  - If your blower pulls its input up internally, leave R7 and D5 out.
- The driver inverts the signal; the firmware compensates (`blower_pwm_inverted: "true"`).
- ebm-papst: 1–10 kHz, 100 % = full speed, about 10 % = minimum, below 10 % = stop (`blower_min_power: "0.12"`).

Blower wire colours (ebm-papst R1G175 / R1G190): red +24 V, blue GND, yellow PWM / 0–10 V, white tach (open collector, 2 pulses/rev, ≤ 10 mA). Other blowers vary; check the label.

FG (tach):

- R5 10 k pull-up to 3.3 V (0.33 mA, well under the 10 mA limit), R6 1 k in series, C3 1 nF to GND, D3 BAT85 clamp to 3.3 V, into GPIO16.
- This works whether the blower's FG output is open-collector or a 5–10 V push-pull.

LEDs

- **U2**: 74AHCT125 level shifter from 3.3 V to 5 V, then a 330 Ω series resistor into the ring's DIN.
- 470 µF across the ring's 5 V supply.
- The firmware scales brightness by ambient light and caps it at 60 % by default. The ring draws about 1.4 A at full white, so don't remove the cap.

Touch pads

- Five 12 × 12 mm pads cut from copper foil tape and stuck into the recesses under the top cap. The cap is 2 mm PETG above them.
- Wire each pad straight to its GPIO (1, 2, 4, 5, 6) with thin wire under 15 cm, keeping the wires apart.
- GPIO3 is skipped because it is a strapping pin.

Interlock

- **Allegro A1120** hall switch, glued into the pocket in the lower shell's bayonet band at the back. It is rated 3–24 V; the common A3144 is out of spec at 3.3 V.
- Supply 3.3 V, open-drain output with a 10 k pull-up to 3.3 V, and 100 nF at the sensor.
- A 6 × 2 mm N52 magnet goes in the base collar pocket. **Test polarity before gluing**: the A1120 switches on a south pole facing its branded side.

Sound

A TDK PS1240P02BT piezo, driven straight from GPIO7 through 100 Ω. Tones come from LEDC (RTTTL), and they are muted in sleep mode or by the *Sounds* switch.

Pin map (ESP32-S3-DevKitC-1, N16R8 or N8R8)

GPIO	Function	Notes
1, 2, 4, 5, 6	Touch pads: power, mode, speed, timer, lock	native capacitive touch
7	Piezo	LEDC

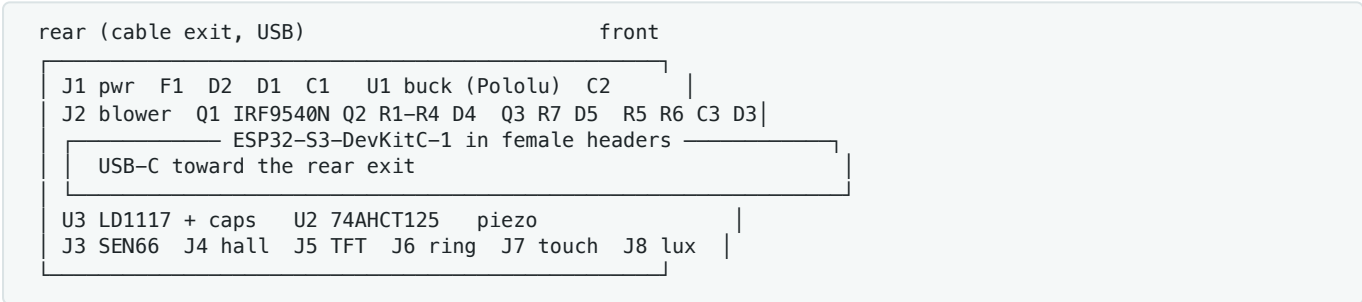
GPIO	Function	Notes
8 / 9	I²C SDA / SCL	SEN66 0x6B, BH1750 0x23
10 / 11 / 12 / 13 / 14	TFT CS / MOSI / SCK / DC / RST	SPI2
15	Blower PWM	via Q3, inverted
16	Blower FG	pulse counter
17	Blower power enable	via Q2/Q1
18	LED ring data	via U2
21	TFT backlight	LEDC 20 kHz
39	Hall interlock	low = base locked

Left free on purpose:

- 0, 3, 45, 46: strapping pins
- 19, 20: USB
- 26–32: flash
- **33–37: used by octal PSRAM on R8 modules**
- 38 and 48: the on-board RGB LED on v1.0 and v1.1 boards

Spare pins for add-ons: 40, 41, 42, 47.

Perfboard layout (50 × 70 mm)



Keep the 24 V traces (J1 → F1 → D2 → Q1 → J2) short and wide: tin them or run 20 AWG wire on the underside. Put Q1 at the board edge with a small clip-on heatsink. It dissipates about 0.6 W at 2.2 A, which is comfortable, but the tray has little airflow.

Bring-up and calibration

1. **Power the board with nothing plugged in.** Check 24 V at the input, 5.0 V after U1, and 3.3 V on both the DevKitC and U3.
2. **Flash over USB** with `esphome run openair.yaml`. Confirm the I²C scan finds 0x23 and 0x6B.
3. **Calibrate the touch pads.**
   - Set `esp32_touch: setup_mode: true` and reflash.
   - Watch the log for each pad's value, untouched and touched through the closed cap.
   - Set `touch_threshold` to about half the difference, then turn `setup_mode` off.
4. **Blower, before it goes into the head.**
   - Connect it and set *Purifier* to 10 %, then 100 %.
   - Confirm it starts and stops cleanly, and that *Blower speed* reads sensibly.
   - If the speed is wrong or inverted, check `blower_pwm_freq`, `blower_pwm_inverted` and `blower_pulses_per_rev` against the datasheet.
   - Note the RPM at 100 % with the filter fitted and enter it as `blower_max_rpm`.
5. **Interlock.** Twist the base off: the ring flashes red and the blower stops. Lock it again and *Base locked* turns on.

6. **Airflow** (optional, for an honest CADR figure). See §Airflow, CADR and blower choice.

7 Firmware

ESPHome configuration for the ESP32-S3-DevKitC-1 (N16R8 or N8R8). Requires **ESPHome 2026.9 or newer**: the `sen6x` component landed in 2026.3, and 2026.8 renamed `voc` / `nox` to `voc_index` / `nox_index`. It compiles to about 1.2 MB of flash and uses 36 % of RAM.

```
firmware/
  openair.yaml           main config: fan, control loops, UI, Home Assistant entities
  sensors/sen66.yaml     default air sensor package (I²C)
  sensors/pms5003.yaml   budget package (UART, PM only)
  secrets.example.yaml   copy to secrets.yaml
  check.sh               validate/compile on the build box
```

To use the budget sensor, change `sensors/sen66.yaml` to `sensors/pms5003.yaml` in the `packages:` block. The PMS5003 package still defines the gas and climate ids, but never publishes them, so the shared logic compiles unchanged and the display shows "--" for those pages.

Modes

Mode	Behaviour
Auto (fan preset)	Speed follows the smoothed PM2.5 using the table below. VOC index > 250 or NOx index > 100 forces at least 55 %.
Sleep (preset)	Fixed speed (*Sleep speed*, default 15 %). Ring and display off, sounds muted.
Turbo (preset)	100 %. Reverts to Auto after 30 minutes.
Manual (no preset)	Any speed from 1–100 % from the app, or the 5 touch steps (20/40/60/80/100 %). Setting a speed from Home Assistant clears the preset, following HA's fan convention.

Auto curve

PM2.5 is smoothed with an exponential moving average (α = 0.2, sampled every 5 s).

Smoothed PM2.5 (µg/m³)	Target speed
≤ 3	15 % (or **stop**, if *Auto stops when air is clean* is on)
≤ 6	25 %
≤ 12	35 %
≤ 25	55 %
≤ 50	75 %
> 50	100 %

Rate limits: speed rises by up to 10 % every 5 s, and falls by at most 5 % per minute, so it doesn't hunt. *Auto max speed* caps the result for a quiet room.

Touch pads (front arc of the top island, left to right)

Pad	Tap	Hold
Power	on / off (resumes the last mode; first use starts Auto)	—
Mode (A)	cycle Auto → Sleep → Turbo → Manual	—
Speed (fan)	step 20 → 40 → 60 → 80 → 100 % (switches to Manual)	—
Timer (clock)	cycle off timer 1 → 2 → 4 → 8 h → off	**5 s: reset filter life**
Lock (padlock)	next display page	**3 s: child lock on/off**

With child lock on, every pad except a 3 s hold on Lock gives a double "denied" beep and shows LOCKED.

Display (1.28" round, LVGL)

- **Centre:** a big number for the current page: PM2.5 → CO_2 → VOC index → temperature → filter %. Tap Lock to step through them.
- **Top:** mode and speed, e.g. `AUTO 35%`, or `BASE OPEN` / `FAN FAULT`.
- **Bottom:** status flags `LOCK`, `42m` (timer), `FILTER` (≤ 10 %), `NO WIFI`.
- **Outer arc:** blower speed.

The backlight dims with ambient light. It goes off in Sleep, and in the dark if *Lights off in the dark* is on.

Air-quality ring (24 × WS2812B)

PM2.5 (µg/m³)	Colour	Text sensor
≤ 5	blue	excellent
≤ 12	green	good
≤ 25	yellow	fair
≤ 50	orange	poor
≤ 100	red	very poor
> 100	purple	hazardous
fault or base open	**pulsing red**	—

The bands are WHO-2021-inspired indoor thresholds, stricter than typical outdoor air-quality categories. Edit the `refresh_ui` script to change them. Brightness = ambient lux scaling × *Light brightness*.

Safety logic

- **Blower power switch (GPIO17)** is turned off at boot priority 600, before anything else runs. It is only turned on when the fan turns on with the base locked.
- **Interlock.** If the base is twisted off (hall sensor released), the fan turns off and the power switch opens. Turning on with the base open is refused with an error beep.
- **Stall detection.** If the fan is commanded to ≥ 5 % but the tach reads < 60 rpm for 40 s, *Blower fault* turns on: red pulse, beep, and a `FAN FAULT` label.
- **Filter due.** At ≤ 10 % remaining, *Filter replacement due* turns on and `FILTER` shows on the display.

Filter life

Two counters, stored in flash every 5 minutes:

- **Volume:** `filter_m3` accumulates *Estimated airflow* × time while running, against `filter_rated_m3` (default 750,000 m³).
- **Age:** time since reset, against `filter_rated_days` (default 365). It starts once SNTP time is valid.

Filter life = min(volume remaining, age remaining) × 100 %. Reset with a 5 s hold on Timer, the *Reset filter life* button, or the `reset_filter` API action.

Home Assistant entities

Type	Entities
Fan	**Purifier**: on/off, 1–100 %, presets Auto / Sleep / Turbo
Sensors	PM1.0, PM2.5, PM4.0, PM10, CO2, VOC index, NOx index, Temperature, Humidity, Ambient light, Blower speed, Estimated airflow, Estimated CADR, Air changes per hour, Filter life, Filter air processed, Off timer remaining, Wi-Fi signal, Uptime
Text sensor	Air quality (excellent … hazardous)

Type	Entities
Binary sensors	Base locked, Blower fault, Filter replacement due, Status
Switches	Child lock, Sounds, Lights off in the dark, Night schedule, Auto stops when air is clean
Numbers	Off timer (h), Auto max speed, Sleep speed, Light brightness, Dark threshold, Room volume, Night starts / ends (hour)
Buttons	Reset filter life, Identify, Restart
Lights	Air quality ring, Display backlight (both normally driven by the firmware)
API actions	`reset_filter`, `set_timer(hours)`

To reach Alexa, Google Home, Apple Home or Matter, expose these through Home Assistant's own integrations (Nabu Casa or manual Alexa/Google, the HomeKit Bridge, the Matter bridge). The **local web UI** at `http://openair.local` works without Home Assistant.

Calibration (substitutions at the top of `openair.yaml`)

Substitution	Default (ebm-papst R1G190)	Set it to
`blower_pwm_freq`	2000Hz	your blower's PWM range (ebm-papst: 1–10 kHz)
`blower_pwm_inverted`	"true"	true with the board's open-drain driver
`blower_min_power`	"0.12"	just above the duty where the blower stops
`blower_pulses_per_rev`	"2"	tach pulses per revolution
`blower_max_rpm`	"2950"	measured rpm at 100 % with a clean filter
`airflow_at_max`	"460"	measured CADR at 100 % ÷ `filter_efficiency`
`filter_rated_m3` / `filter_rated_days`	750000 / 365	your filter's rating
`touch_threshold`	"1500"	about half the touched-minus-untouched delta from `setup_mode`

Building and flashing

1. Copy `secrets.example.yaml` to `secrets.yaml` and fill in Wi-Fi, the fallback AP password and a new API key (`openssl rand -base64 32`).
2. **First flash over USB:** `esphome run openair.yaml`, or use the ESPHome add-on's "Install → Plug into this computer". After that, OTA updates use the API encryption key, and the web UI's plaintext OTA is disabled.
3. If Wi-Fi can't connect, the device starts an `OpenAir Setup` access point with a captive portal.

8 Printing

All parts fit a 256 × 256 × 256 mm bed (Bambu P1S / X1 / A1 class). The largest diameter is 240 mm, so print **without a brim** and check that your slicer's exclusion zones don't clip the outline.

Material

Part	Material	Why
Base, shell rings, head body, inlet ring, blower bridge, top cap, tray, sensor clip	**PETG** (or ASA)	Heat near the motor (~40–50 °C), long-term creep under filter-seal load, and a summer car trip home. PLA will slowly deform around the motor and the bayonet.
Control disc	**White or natural PETG**	The LED ring glows through a 0.8 mm skin.
Optional feet	TPU 95A	Or use 12.7 mm self-adhesive bumpers.

About **1.94 kg of PETG** for the reference tower and **1.83 kg** for the value tower. These are measured from the STLs with `cad/tools_volume.py`, assuming 4 walls and the infill below. Print time is roughly 50–60 h on a P1S at standard speeds; the base, the two shell rings and the head take the longest. STLs: `cad/stl/tower-reference/` or `cad/stl/tower-value/`. For the side table see §Side Table.

Per-part settings

Part	Orientation (STLs are exported this way)	Walls	Infill	Supports	Notes
`base`	flat side down	4	6–10 % gyroid	none	A heavy base keeps the centre of gravity low; the value profile makes it 20 mm tall.
`shell_lower`	upright, bayonet band down	4	25 %	none	Slot roofs are 45° and the J-slot roofs are chamfered. Sensor window included.
`shell_upper`	upright, socket end down	4	25 %	none	
`head_body`	upright, deck down	4	25 %	none	Universal for any plug fan up to Ø200 mm and 95 mm from inlet edge to back face.
`inlet_ring`	flange down	4	40 %	none	**Blower-specific**: re-export after setting `blower_eye_d`. 45° converging mouth.
`blower_bridge`	spoke tops down (exported flipped)	4	40 %	none	**Blower-specific**: hub drop and flange holes come from the `[Blower]` block.
`top_cap`	top face down (exported flipped)	3	20 %	none	Smooth top, and the icons come out crisp as first-layer recesses.
`control_disc`	flange down	3	100 %	none	White or natural PETG, 0.2 mm layers. The 0.8 mm skin over the LEDs bridges ~13 mm.
`electronics_tray`	flat	3	20 %	none	
`sensor_clip`	flat	3	100 %	none	

Layer height 0.2 mm, 0.4 mm nozzle. Print the shell rings at moderate speed: tall thin walls with 300+ slots each.

Inserts and hardware

- **M3 × 5.7 × 4.6 mm brass heat-set inserts.** The holes are 4.0 mm; set them at ~230 °C with a flat tip.
  - Head body: 4 at the bottom, 4 at the top, 4 bridge seats.
  - Shell upper: 4 at the bottom.
- The inlet ring uses 3 × M3 × 10 countersunk self-tapping screws into the deck's pilot holes.

- Then join the parts with M3 × 12 socket-head screws, driven up from below through the deep counterbores. Assembly order is in §Assembly.
- The top cap uses 4 × M3 × 12 countersunk screws into the head's top inserts.

If your filter differs

Measure your filter's height and outer diameter and set `filter_h`, `filter_od` and `filter_spacer` in `cad/openair.scad`. Re-export everything with `cad/render.sh`; the shell height follows automatically. Filters sold as "Xiaomi 2/2S/3/3H/Pro compatible" are listed at 200 × 293 mm. The newer 4-series filters are different sizes (4: 210 × 293, 4 Lite: 200 × 310).

9 Assembly

Allow about 3 hours, excluding printing and soldering the perfboard (see §Electronics).

```
          ┌──────────── top cap + control disc + electronics tray (lift-off unit)
          │ ┌────────── blower bridge (blower hangs from it)
          │ │ ┌──────── head body: deck, inlet nozzle, plenum
          │ │ │
┌─────────┴─┴─┴─┐
│  shell_upper  │    perforated intake rings, joined with M3 screws
│  shell_lower  │    + SEN66 window, DC jack, hall sensor
└───────────────┘
┌───────────────┐
│     base      │    twist-lock bayonet, carries the filter, magnet for interlock
└───────────────┘
```

1. Inserts

Heat-set **M3 × 5.7 brass inserts** at ~230 °C:

- **head_body:** 4 in the bottom bosses (from below), 4 in the top bosses, 4 in the bridge seats (from above).
- **shell_upper:** 4 in the bottom bosses.

2. Lower shell

1. **Hall sensor.** Solder three 26 AWG wires (about 60 cm, to reach the tray) to the **A1120**, sleeve the joints, and push it into the pocket in the bayonet band at the back. Branded face toward the inside; check polarity in step 7 before gluing. Run the wires up the groove.
2. **DC jack.** Fit the P0622 in its 8 mm hole at the back. Solder 20 AWG red/black, about 60 cm.
3. **SEN66.**
   - Plug the GHR-06V cable in. Wire SDA, SCL, 3V3 and GND to a JST-XH 4-pin plug at the far end.
   - Slide the sensor into the window frame from the inside, face outward.
   - Fit a thin foam strip across the middle of its face so its outlet can't feed straight back into its inlets, matching the solid band in the vent pattern.
   - Hold it with the printed `sensor_clip` and two M2.5 × 8 self-tappers.
   - Put a 100 µF capacitor across 3V3/GND at the sensor end of the cable.
4. **Wiring loom.** Bundle the jack, hall and sensor wires and thread them through the cable guides beside the window. Leave the ends loose above the ring.

3. Shell stack

1. Join `shell_upper` to the **head body**. Drive 4 × M3 × 12 up from inside the upper ring, through its top bosses, into the head's bottom inserts. The ring is open at both ends, so a screwdriver reaches easily.
2. Hold the lower ring under the stack and feed its loom up through the upper ring and the **wire pass** in the deck (rear, Ø7 mm). Seal around the wires with hot glue or silicone: the plenum above is pressurised, so a leak here bleeds airflow back to the intake side.
3. Join `shell_lower` to the upper ring with 4 × M3 × 12, again driven up from below inside the lower ring.

4. Filter seal

Stick two rings of **9 × 6 mm EPDM foam tape** into the guide grooves on the deck underside (Ø156 and Ø180 mm). Butt the ends together without overlapping. These rings are the **only** seal between unfiltered and filtered air: any gap lets room air bypass the filter. Press them down firmly.

5. Blower

1. Drop the **inlet_ring** into the counterbore around the deck hole and fix it with 3 × M3 × 10 countersunk self-tappers. Run a thin bead of silicone under the flange: a gap lets pressurised air from the plenum recirculate into the blower

inlet, which costs airflow.

2. Bolt the blower's motor back face to the **blower_bridge** hub floor with **4 × M4 × 16**, driven down from inside the hub's bore. That is 11 mm of floor and contact boss plus 5 mm of thread: ebm-papst allows at most 6 mm. Feed the blower lead through one of the four notches between the bolts. Thin rubber washers under the screw heads help decouple vibration.
3. Lower the bridge and blower into the head so the blower's inlet ring slides over the nozzle lip. Spin the impeller by hand: it must not touch the lip, with about 1.5 mm radial clearance and 2.5 mm overlap. If it rubs, recheck `blower_eye_d` and `nozzle_clear`.
4. Screw the bridge to its 4 seats with M3 × 12.
5. Route the blower cable up to where the tray will sit.

6. Control unit (top cap)

1. **Touch pads.** Stick a 12 × 12 mm copper-foil square into each recess under the cap. Solder a thin wire to each and tape the joint flat.
2. **Light sensor.** Glue the BH1750 board under the 2.6 mm light window at the back of the island, sensor centred.
3. **Control disc.** Fit the LED ring into the white `control_disc`'s annular pocket and the display into its centre pocket, glass up. Fix both with a few dots of hot glue. Push the disc up through the cap from below; its flange rests under the island skin.
4. **Tray.** Mount the perfboard on the tray's four standoffs (M2.5 self-tappers).
5. Plug in the display, ring, pads and light sensor. Screw the tray to the 4 pod bosses (M3 × 12 through the tray, into 2.8 mm pilot holes).

7. First power-up (before the filter goes in)

1. Plug in the head-side connectors: power, blower, SEN66, hall.
2. Flash and calibrate following §Electronics, Bring-up.
3. **Interlock.** Hold the base in place, locked, without the filter. *Base locked* should be on. If it isn't, flip the magnet and only then glue it.
4. Seat the top cap with 4 × M3 × 12 countersunk screws.

8. Filter and base

1. Remove the plastic bag from the filter. Check its height: if it is under 293 mm, set `filter_spacer` and reprint the base, or add a foam ring on the platform.
2. Stand the filter in the base, **open end up**, inside the centring arcs.
3. Lower the body over it. Push down until the shell sits on the base shoulder, compressing the foam, then **twist clockwise** about 35° until the detents click.
4. Power on. *Base locked* should read on and the ring should go green within a minute as the SEN66 warms up.

Changing the filter

Lay the unit on its side (or lift it), twist the base anticlockwise, pull the filter out by its strap, and fit the new one. Then reset the filter life: **hold the Timer pad for 5 s** (three rising beeps), or press *Reset filter life* in Home Assistant.

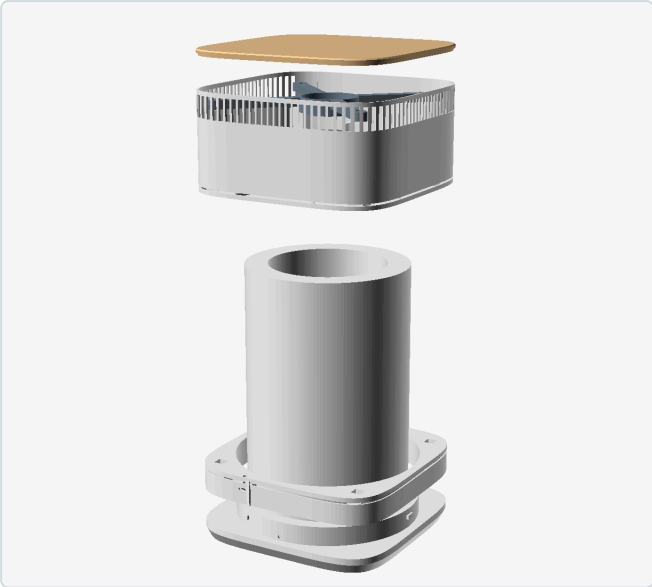
10 Parts gallery

Renders from OpenSCAD 2026.10.02 (manifold backend). The STLs in `cad/stl/<build>/` are exported in print orientation; these views show the parts as installed.

Side Table



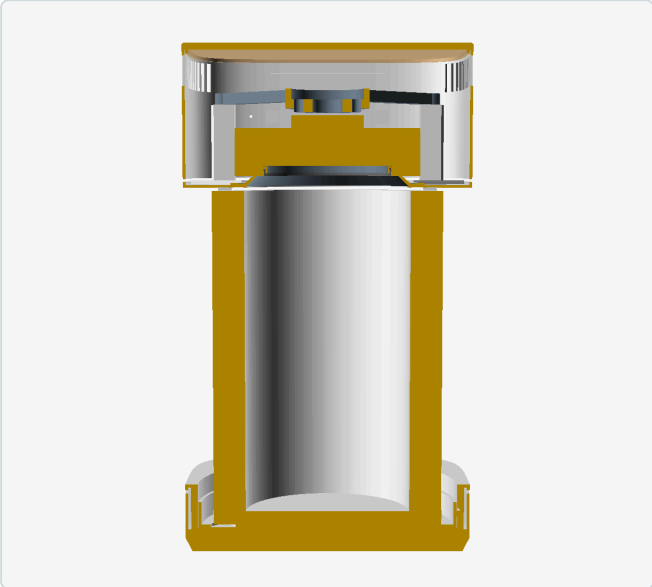
`Side Table`: Squircle 250 x 250 x 443 mm, fabric-sleeve intake, louvres under the tabletop on three sides.



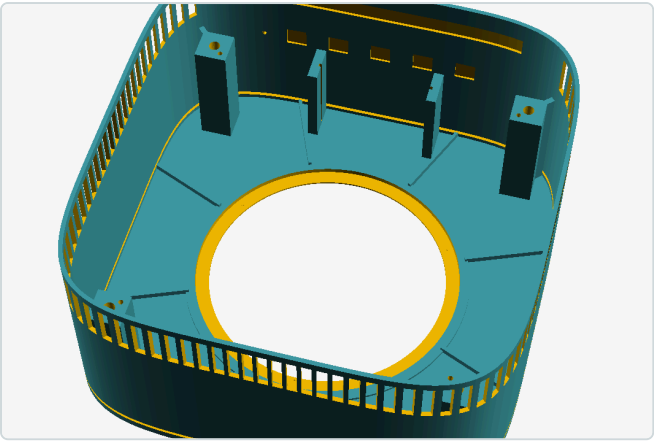
`Side Table, exploded`: Base, ring, corner posts, filter, head with blower and bridge, tabletop.



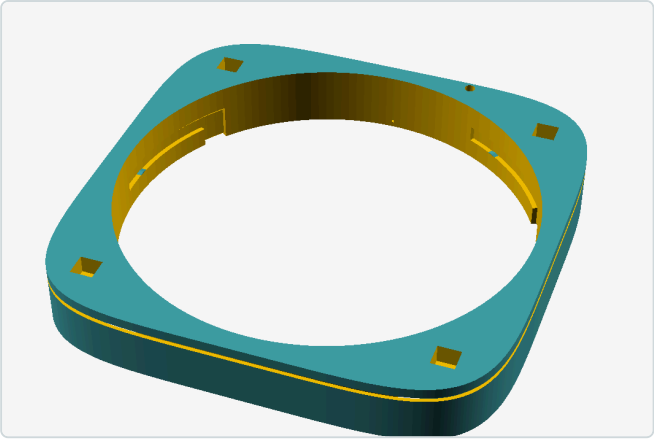
`Side Table frame`: Sleeve removed: four printed posts carry the head; front light line and touch pads.



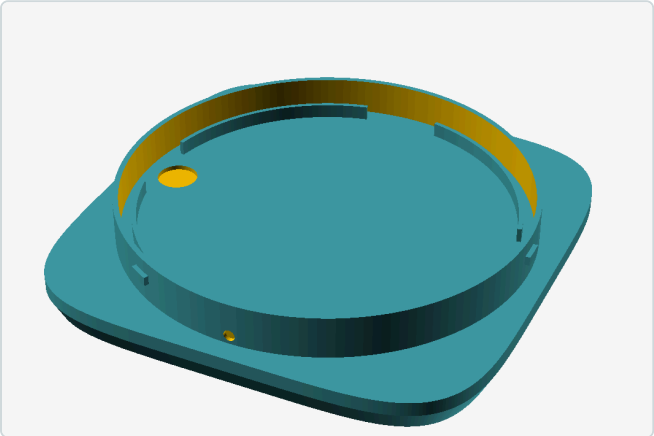
`Side Table cutaway`: Fabric -> filter -> deck nozzle -> blower -> louvres.



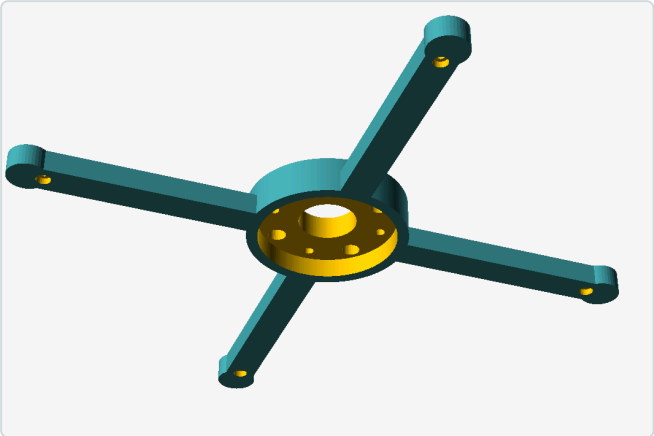
`head (side table)`: Ribbed 2.4 mm deck, columns, card guides, light-line and pad recesses, louvre band.



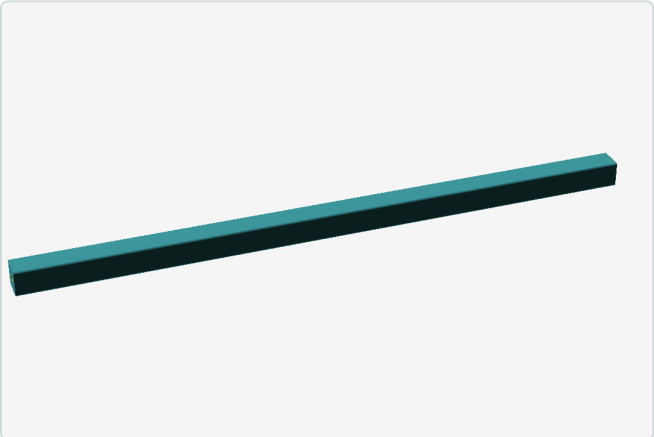
`ring`: Bayonet J-slots, post sockets, elastic groove, USB-C pocket, hall pocket.



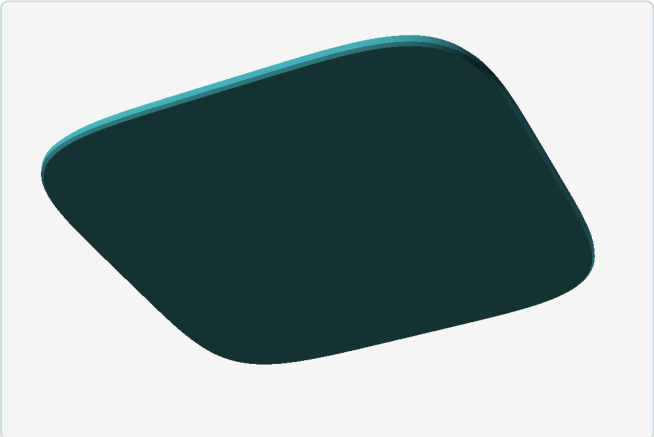
`base (side table)`: Squircle plinth with round bayonet collar; twist to release the filter.



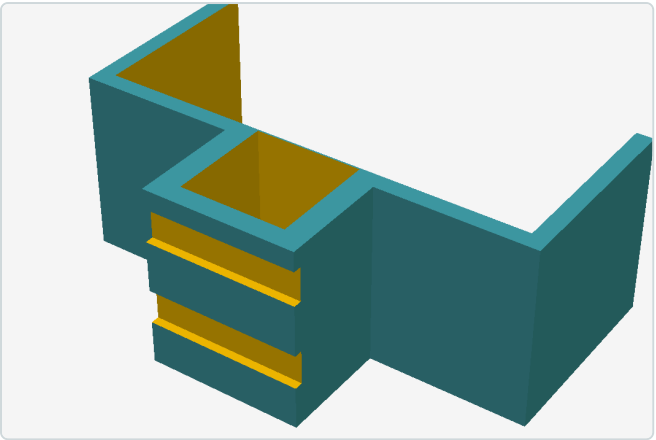
`bridge (side table)`: Spokes to the column tops, hollow hub for the motor flange.



`post x 4`: 12 mm square, printed lying flat diagonally.

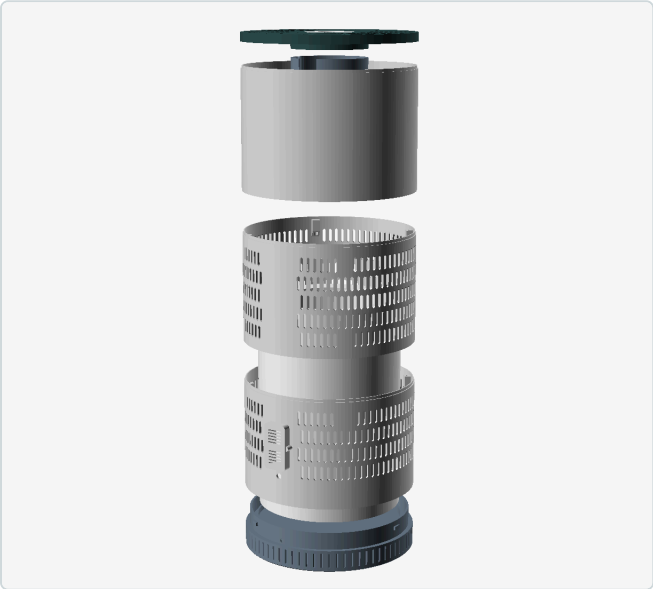


`top`: Printed tabletop, or cut timber from top_template.svg.

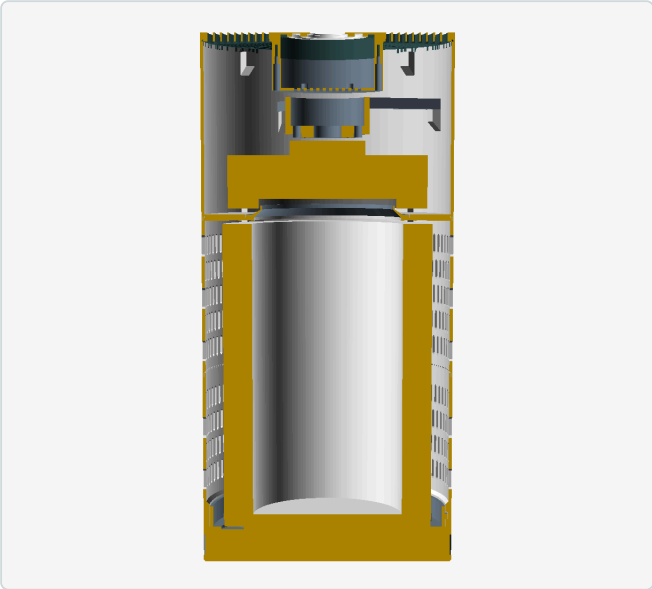


pms_clip: PMS5003 cradle, zip-tied to the back-right post inside the sleeve.

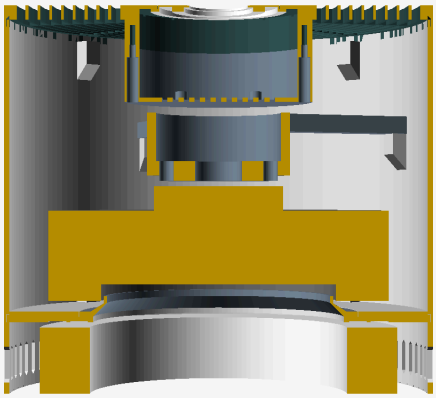
Tower



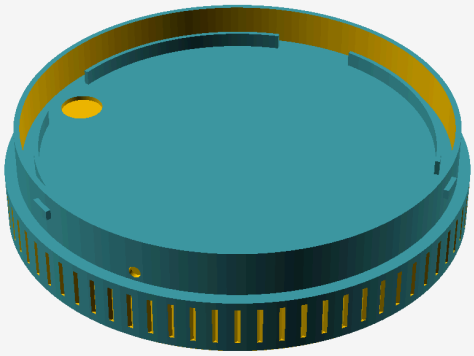
Tower, exploded: Base, two perforated shell rings, filter, head, top cap.



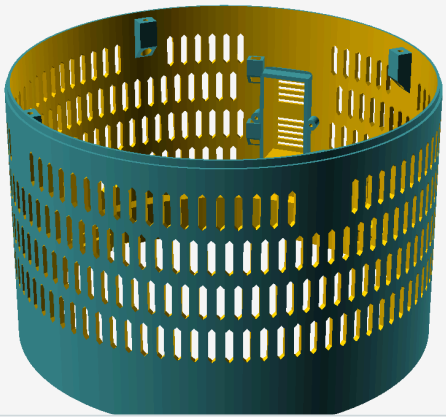
Tower cutaway: Room air enters through the shell slots, passes radially through the filter, rises through the deck nozzle into the blower, and leaves through the top grille.



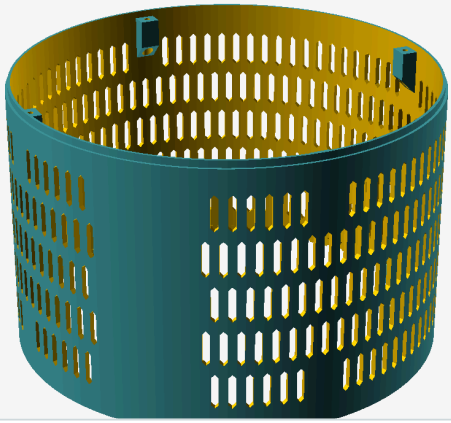
`Tower head section`: Inlet-ring lip inside the blower eye; bridge hub on the motor's stationary back face; tray and control pod above.



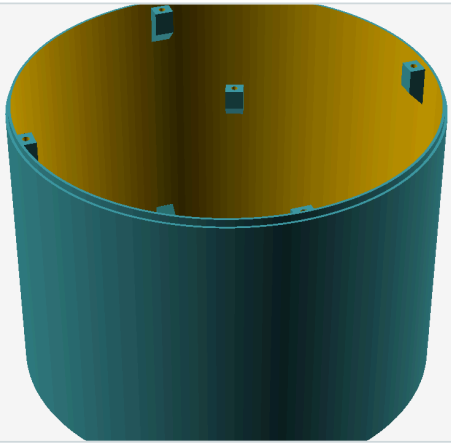
`base`: Twist-lock bayonet (3 lugs), filter platform with centring arcs, magnet pocket.



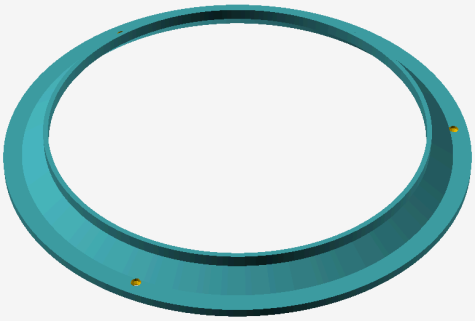
`shell_lower`: Bayonet J-slots, hall-sensor pocket, power input, sensor window and frame, cable guides.



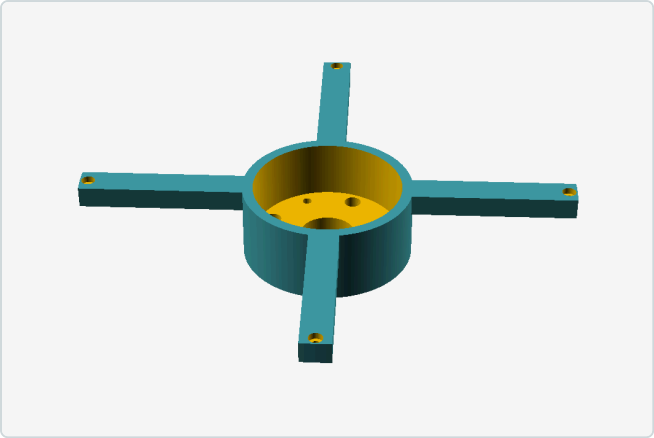
`shell_upper`: Perforated intake ring; joins the head and the lower ring.



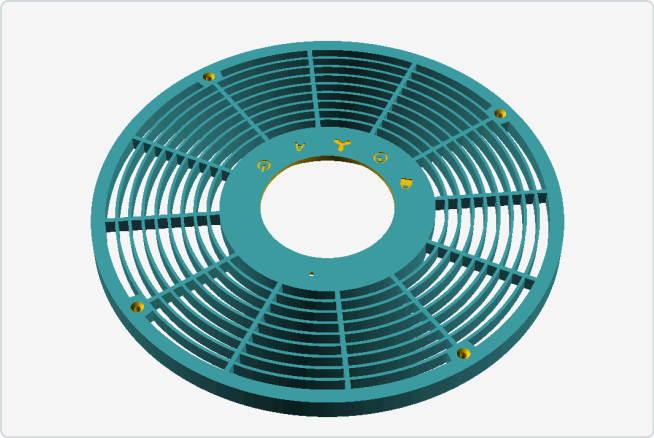
`head_body`: Universal blower housing: deck with gasket grooves, inlet-ring counterbore, bridge seats, wire pass.



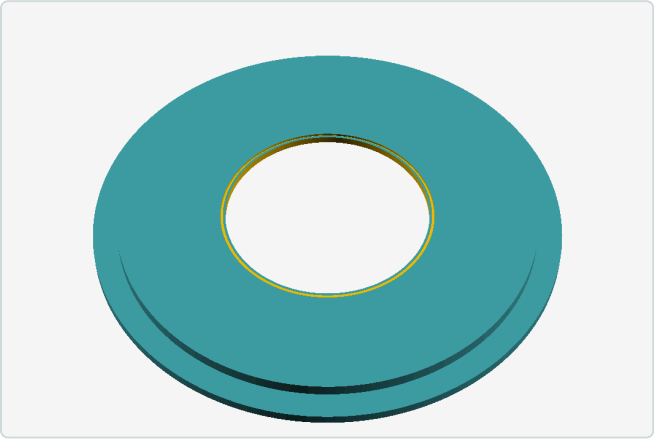
`inlet_ring`: Blower-specific 45-degree converging nozzle; the lip enters the impeller eye.



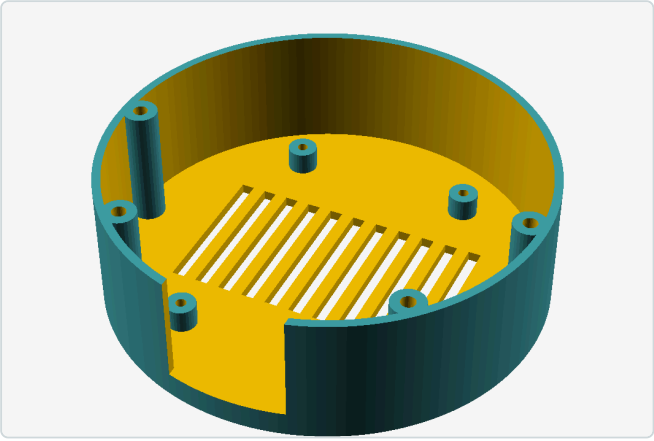
blower_bridge: Four spokes and a hollow drop-hub; the motor bolts to the hub floor.



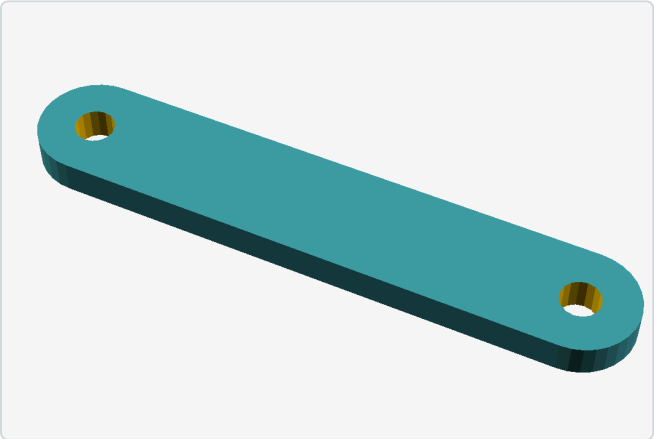
top_cap: Outlet grille, control island with 5 touch-pad recesses and icons, light window, control pod.



control_disc: White PETG: display window (reference) or logo (value), LED-ring pocket under a 0.8 mm light skin.



electronics_tray: Carries the 50 x 70 mm perfboard with the ESP32-S3 DevKitC.



sensor_clip: Holds the air sensor in its window.

11 Bill of materials

Prices in AUD, incl. GST where the retailer shows it, checked 2026-10-04. "listed" = read off a retailer page; "estimate" = typical AliExpress/hardware-store price, not confirmed for a specific listing. AliExpress prices exclude shipping. Generated by `build_bom.py`; edit the tables there.

Totals

Build	Total (AUD)	Biggest items
Reference tower (ebm-papst R1G190, SEN66, TFT)	**816.91**	EC backward-curved blower 400, Sensirion SEN66 99, PETG filament 47
Value tower (160 mm blower, PMS5003, USB-C)	**194.47**	EC backward-curved fan 160 x 45 mm 45, PETG filament 44, Plantower PMS5003 laser PM sensor + cable 22
Side Table (value internals, fabric sleeve)	**194.75**	EC backward-curved fan 160 x 45 mm 45, PETG filament 37, Plantower PMS5003 laser PM sensor + cable 22

The value builds assume you reuse a USB-C PD charger of 45 W or more. Buying one adds ~A$25-40 (or use an RCM 24 V 2 A brick + 2.1 mm jack, ~A$43).

Upgrade path: the value builds take the reference blower later. Swap in an ebm-papst R1G190 (~A$400) or an AliExpress 190 mm EC fan (~A$117), reprint `inlet_ring` and the bridge, and power it from a 24 V 3 A brick.

Reference tower (ebm-papst R1G190, SEN66, TFT)

Group	Item	Qty	Unit	Total	Source	Note
Air path	EC backward-curved blower, 24 V: reference ebm-papst R1G190-AB27-02 (190 mm, 51 W, PWM + tach)	1	400.00	400.00	Sager / RS (US$244-282, about A$370-430; AU stock unverified)	Datasheet-verified; the CAD and firmware defaults match it. R1G175-AB63-02 (34 W) is the smaller sibling.
Air path	HEPA H13 + activated-carbon filter, Xiaomi 2/2S/3/3H/Pro compatible (200 x 293 mm)	1	30.00	30.00	eBay AU / AliExpress generic (2-pack ~A$51-56); genuine A$40-60	Generic with or without RFID; the RFID tag is not used
Air path	EPDM closed-cell foam weatherstrip, 9 x 6 mm, 2 m	1	9.00	9.00	Bunnings (Raven / Gripset)	Two rings on the deck underside seal the filter top
Printed	PETG filament, ~1.94 kg (measured from the STLs, cad/tools_volume.py)	1.94	24.00	46.56	budget PETG, ~A$22-28 per kg (estimate)	
Printed	White or natural PETG for the control disc (~15 g)	1	1.00	1.00	offcut of any spool	
Controller	ESP32-S3-DevKitC-1 N16R8 (44-pin clone) or genuine N8R8	1	14.00	14.00	AliExpress A$10.59-16.05; genuine N8R8 Core ~A$49.70	R8 PSRAM: never use GPIO33-37
Controller	Sensirion SEN66 (PM1/2.5/4/10, RH, T, VOC, NOx, CO2)	1	99.35	99.35	DigiKey AU SEN66-SIN-T	3.3 V only
Controller	JST GHR-06V-S housing + 6 pre-crimped 26 AWG leads (SEN66 cable, <= 50 cm)	1	6.00	6.00	DigiKey AU (GHR-06V-S + ASSHSSH28K305)	or Adafruit 6331 SEN6x breakout A$7.95
Controller	GC9A01 1.28in round TFT 240x240, SPI	1	6.00	6.00	AliExpress A$3.49-6.80; Waveshare 19192 US$14.99	check board diameter <= 38 mm
Controller	WS2812B 24-LED ring, OD 65.5 / ID 52.3 mm	1	6.00	6.00	AliExpress A$3.13-6.79; Adafruit 1586 DigiKey A$24.68	

Group	Item	Qty	Unit	Total	Source	Note
Controller	BH1750 ambient light module (GY-302)	1	3.89	3.89	AliExpress; DigiKey A$7.21	
Controller	Allegro A1120EUA-T hall switch (3-24 V)	1	2.87	2.87	DigiKey AU	replaces A3144 (5 V part)
Controller	Neodymium disc magnet 6 x 2 mm (pack)	1	5.29	5.29	AliExpress	for the base interlock
Controller	TDK PS1240P02BT piezo transducer	1	1.17	1.17	DigiKey AU	
Controller	Copper foil tape, 12 mm (touch pads)	1	5.00	5.00	Jaycar / AliExpress	
Power	24 V 3 A power supply, 2.1 mm plug, RCM/SAA (Altronics M8971A)	1	42.25	42.25	Altronics	needs >= 3 A with the R1G190; a 2.5 A pack is only OK with the R1G175
Power	IEC C13 mains lead (AU plug) for the M8971A	1	10.50	10.50	Jaycar PS4106	
Power	2.1 mm metal chassis DC jack, 5 A (Altronics P0622)	1	5.35	5.35	Altronics	8 mm hole
Power	Buck 24 V -> 5 V 3 A, Pololu D36V28F5	1	36.15	36.15	Core Electronics	budget: LM2596S module A$2.95, set to 5.0 V first
Power	LD1117V33 3.3 V LDO (SEN66 rail)	1	2.00	2.00	Jaycar / DigiKey	
Discretes	IRF9540N P-MOSFET (blower high-side switch)	1	5.95	5.95	Jaycar ZT2467	
Discretes	BC547 NPN	1	0.30	0.30	Jaycar ZT2152	
Discretes	2N7000 / BS170 N-MOSFET (PWM driver)	1	0.75	0.75	Jaycar ZT2400	
Discretes	1N4742 12 V zener (gate clamp)	1	0.65	0.65	Jaycar ZR1412	
Discretes	SB560 Schottky (reverse polarity)	1	0.87	0.87	DigiKey AU	
Discretes	P6KE30A TVS	1	1.37	1.37	DigiKey AU	not 1.5KE27A
Discretes	BAT85 Schottky (FG clamp)	1	0.30	0.30	DigiKey AU	
Discretes	PTC resettable fuse 3.0 A hold, 30 V (RXEF300)	1	1.00	1.00	DigiKey AU	
Discretes	33 k resistor + second 1N4742 12 V zener (blower PWM pull-up, ebm-papst circuit)	1	0.80	0.80	Jaycar	
Discretes	74AHCT125N level shifter (DIP-14)	1	1.50	1.50	DigiKey AU	
Discretes	Capacitors: 470 uF 35 V, 470 uF 10 V, 220 uF 10 V, 2 x 100 uF, 10 uF, 4 x 100 nF, 1 nF	1	6.00	6.00	Jaycar / DigiKey	
Discretes	Resistors: 10 k x5, 4.7 k x3, 100 k x2, 1 k, 330, 100 x2	1	3.00	3.00	Jaycar / stock	
Discretes	Clip-on TO-220 heatsink	1	2.00	2.00	Jaycar	
Hardware	Stripboard 95 x 76 mm (cut to 50 x 70)	1	8.25	8.25	Jaycar HP9540	
Hardware	JST-XH 2.54 mm header + housing + crimp kit	1	8.00	8.00	AliExpress	
Hardware	Female header strips 2 x 22 (DevKitC socket)	1	2.00	2.00	Jaycar / AliExpress	
Hardware	Hook-up wire: 20 AWG red/black 1 m, 26 AWG silicone assortment	1	8.00	8.00	Jaycar	
Hardware	M3 x 5.7 x 4.6 brass heat-set inserts (100 pack)	1	10.79	10.79	AliExpress	~24 used

Group	Item	Qty	Unit	Total	Source	Note
Hardware	M3 x 12 socket-head screws (16) + M3 x 12 countersunk (4) + M3 x 8 (8)	1	9.00	9.00	Jaycar HP0406 etc.	
Hardware	M4 x 16 screws (4) + thin rubber washers for the blower back face	1	4.00	4.00	Jaycar / Bunnings	max 6 mm thread engagement in the ebm-papst motor
Hardware	M2.5 x 8 self-tapping screws (sensor clip)	1	2.00	2.00	Jaycar	
Hardware	12.7 mm self-adhesive rubber feet (4)	1	4.00	4.00	Jaycar HP0815	
Hardware	Heat-shrink, zip ties, hot glue	1	4.00	4.00	stock	

Group	AUD
Air path	439.00
Printed	47.56
Controller	149.57
Power	96.25
Discretes	24.49
Hardware	60.04
Total	**816.91**

Value tower (160 mm blower, PMS5003, USB-C)

Group	Item	Qty	Unit	Total	Source	Note
Air path	EC backward-curved fan 160 x 45 mm, 24 V, PWM in + FG out (runs on 20 V)	1	45.00	45.00	AliExpress, e.g. item 1005013134063403 (listed A$40.79 + shipping)	estimate; ask the seller for drawing, pinout, PWM spec
Air path	HEPA + carbon filter, Xiaomi 2/2S/3/3H/Pro compatible (200 x 293 mm), generic	1	18.00	18.00	AliExpress A$9-38 / eBay 4-pack ~A$25 each	estimate
Air path	Closed-cell foam tape, 9 x 6 mm, 2 m	1	5.00	5.00	AliExpress / Bunnings	estimate
Printed	PETG filament, ~1.83 kg (value tower, measured from the STLs)	1.83	24.00	43.92	budget PETG ~A$22-28/kg	estimate
Controller	ESP32-S3-DevKitC-1 N16R8 clone (44-pin)	1	14.00	14.00	AliExpress A$10.59-16.05	listed
Controller	Plantower PMS5003 laser PM sensor + cable	1	22.00	22.00	AliExpress	estimate
Controller	WS2812B 24-LED ring (OD 65.5 mm)	1	6.00	6.00	AliExpress A$3.13-6.79	listed
Controller	GL5528 LDR + 10 k resistor (ambient light)	1	0.50	0.50	Jaycar / AliExpress	estimate
Controller	Allegro A1120EUA-T hall switch	1	2.87	2.87	DigiKey AU	listed
Controller	Neodymium magnets 6 x 3 mm (pack of 20)	1	5.29	5.29	AliExpress A$5.29-5.89	listed
Controller	TDK PS1240P02BT piezo	1	1.17	1.17	DigiKey AU	listed

Group	Item	Qty	Unit	Total	Source	Note
Controller	Touch pads: aluminium foil tape or kitchen foil + glue	1	0.00	0.00	household	copper tape +A$3
Power	USB-C PD trigger board, fixed 20 V	1	3.00	3.00	AliExpress	estimate
Power	USB-C PD charger >= 45 W + USB-C cable (reuse your laptop/phone charger)	1	0.00	0.00	already owned	buying one adds ~A$25-40; or an RCM 24 V 2 A brick (Altronics M8969 A$39.95) + jack
Power	LM2596S buck module (set to 5.0 V first)	1	2.95	2.95	Core Electronics	listed
Discretes	2N7000, 1N4742 12 V zener, 1N4001, BAT85, 2.5 A PTC, P6KE27A TVS	1	4.27	4.27	Jaycar / DigiKey	listed
Discretes	Resistors (10 k x4, 4.7 k, 33 k, 1 k, 330, 100 x2) + caps (470 uF 35 V, 220 uF, 470 uF 10 V, 100 nF x3, 1 nF)	1	3.50	3.50	Jaycar / stock	estimate
Hardware	Perfboard 50 x 70 mm	1	1.50	1.50	AliExpress	estimate
Hardware	Pin headers, Dupont/JST connectors	1	3.00	3.00	AliExpress	estimate
Hardware	Hook-up wire (22 / 26 AWG)	1	3.00	3.00	Jaycar / stock	estimate
Hardware	M3 self-tapping screw assortment (no heat-set inserts)	1	6.00	6.00	AliExpress / Bunnings	estimate
Hardware	Blower screws (4) + rubber washers	1	1.50	1.50	Bunnings	estimate; size per blower drawing
Hardware	Hot glue, heat-shrink, zip ties	1	2.00	2.00	stock	estimate

Group	AUD
Air path	68.00
Printed	43.92
Controller	51.83
Power	5.95
Discretes	7.77
Hardware	17.00
Total	**194.47**

Side Table (value internals, fabric sleeve)

Group	Item	Qty	Unit	Total	Source	Note
Air path	EC backward-curved fan 160 x 45 mm, 24 V, PWM in + FG out (runs on 20 V)	1	45.00	45.00	AliExpress, e.g. item 1005013134063403 (listed A$40.79 + shipping)	estimate; ask the seller for drawing, pinout, PWM spec
Air path	HEPA + carbon filter, Xiaomi 2/2S/3/3H/Pro compatible (200 x 293 mm), generic	1	18.00	18.00	AliExpress A$9-38 / eBay 4-pack ~A$25 each	estimate
Air path	Closed-cell foam tape, 9 x 6 mm, 2 m	1	5.00	5.00	AliExpress / Bunnings	estimate

Group	Item	Qty	Unit	Total	Source	Note
Printed	PETG filament, ~1.55 kg (side table incl. 4 posts, measured from the STLs)	1.55	24.00	37.20	budget PETG ~A$22-28/kg	estimate
Controller	ESP32-S3-DevKitC-1 N16R8 clone (44-pin)	1	14.00	14.00	AliExpress A$10.59-16.05	listed
Controller	Plantower PMS5003 laser PM sensor + cable	1	22.00	22.00	AliExpress	estimate
Controller	WS2812B strip 60 LED/m, 10 LEDs cut from a 1 m reel	1	4.00	4.00	AliExpress	estimate
Controller	GL5528 LDR + 10 k resistor (ambient light)	1	0.50	0.50	Jaycar / AliExpress	estimate
Controller	Allegro A1120EUA-T hall switch	1	2.87	2.87	DigiKey AU	listed
Controller	Neodymium magnets 6 x 3 mm (pack of 20)	1	5.29	5.29	AliExpress A$5.29-5.89	listed
Controller	TDK PS1240P02BT piezo	1	1.17	1.17	DigiKey AU	listed
Controller	Touch pads: aluminium foil tape or kitchen foil + glue	1	0.00	0.00	household	copper tape +A$3
Power	USB-C PD trigger board, fixed 20 V	1	3.00	3.00	AliExpress	estimate
Power	USB-C PD charger >= 45 W + USB-C cable (reuse your laptop/phone charger)	1	0.00	0.00	already owned	buying one adds ~A$25-40; or an RCM 24 V 2 A brick (Altronics M8969 A$39.95) + jack
Power	LM2596S buck module (set to 5.0 V first)	1	2.95	2.95	Core Electronics	listed
Discretes	2N7000, 1N4742 12 V zener, 1N4001, BAT85, 2.5 A PTC, P6KE27A TVS	1	4.27	4.27	Jaycar / DigiKey	listed
Discretes	Resistors (10 k x4, 4.7 k, 33 k, 1 k, 330, 100 x2) + caps (470 uF 35 V, 220 uF, 470 uF 10 V, 100 nF x3, 1 nF)	1	3.50	3.50	Jaycar / stock	estimate
Hardware	Perfboard 50 x 70 mm	1	1.50	1.50	AliExpress	estimate
Hardware	Pin headers, Dupont/JST connectors	1	3.00	3.00	AliExpress	estimate
Hardware	Hook-up wire (22 / 26 AWG)	1	3.00	3.00	Jaycar / stock	estimate
Hardware	M3 self-tapping screw assortment (no heat-set inserts)	1	6.00	6.00	AliExpress / Bunnings	estimate
Hardware	Blower screws (4) + rubber washers	1	1.50	1.50	Bunnings	estimate; size per blower drawing
Hardware	Hot glue, heat-shrink, zip ties	1	2.00	2.00	stock	estimate
Enclosure	Stretch knit / mesh fabric, ~0.35 m x 0.95 m (sleeve, also the pre-filter)	1	6.00	6.00	Spotlight / any fabric shop	estimate
Enclosure	3 mm round elastic cord, 2 m (sleeve hems)	1	3.00	3.00	Spotlight / Kmart	estimate

Group	AUD
Air path	68.00
Printed	37.20
Controller	49.83
Power	5.95
Discretes	7.77

Group	AUD
Hardware	17.00
Enclosure	9.00
Total	**194.75**

Running cost

- Filter: ~A$18-30 generic (A$40-60 genuine) every 6-12 months.
- Electricity: ~5-10 W average in Auto, roughly A$15-25/year at 28 c/kWh.
- For comparison, Levoit Core 400S filters are A$79 and Philips AC3220 filters A$141-159.

Appendix A: CAD parameters

All dimensions are in millimetres. Edit them in `cad/openair.scad` and re-export with `cad/render.sh`. Derived values (shell height, head height, hub drop) follow automatically. Tower blocks come first; the side-table blocks (`cad/sidetable.scad`) follow. Build profiles switch between reference and value values with `build = "reference" | "value"`.

Build profile

```
// "reference": ebm-papst R1G190 blower, SEN66, round TFT, 24 V brick + DC jack, brass inserts.
// "value":     160 mm 24 V EC blower, PMS5003, LED-only UI, USB-C PD input, self-tapping screws.
build    = "reference";
is_value = build == "value";
```

Filter: measure yours

```
filter_od     = 200;   // Xiaomi M8R-FLH / SCG4021GL family: 200 x 293 mm
filter_h      = 293;
filter_id     = 144;   // open-top hole; listings disagree (117-145), the seal design tolerates it
filter_spacer = 0;     // raise the filter if yours is shorter than 293
```

Body

```
od      = 240;
wall    = 3;
fit     = 0.3;         // radial sliding clearance
$fn     = 160;
```

Base and bayonet

```
base_h      = is_value ? 20 : 30;
collar_h    = 24;
collar_wall = 3;
band_h      = 26;
band_t      = 3;
lug_w       = 12;
lug_h       = 5;
lug_t       = 2.6;
lug_z       = 11.5;
lock_angle  = 35;
detent_h    = 0.4;
```

Seal

```
gasket_thk_compressed = 3;    // two rings of 9x6 mm EPDM tape, ~50 % squeeze
gasket_r              = [78, 90];
deck_hole_d           = 136;
```

Shell perforation

```
slot_w     = 4;
slot_h     = 20;
slot_gap   = 5.2;
slot_cols  = 84;
spine_half = is_value ? 16 : 13;   // degrees of solid "spine" at the back: power, sensor window, wiring
guide_angle = spine_half - 2.5;
```

Fasteners

```
use_inserts  = !is_value;
insert_d     = use_inserts ? 4.0 : 2.8;   // M3 x 5.7 brass heat-set insert, or a pilot for M3 self-tapping screws
insert_depth = 6.5;
screw_d      = 3.4;
screw_head_d = 6.5;
boss_angles  = [45, 135, 225, 315];
```

Hardware

```
power_input = is_value ? "usbc_pd" : "dc_jack";
dc_jack_d   = 8.2;     // Altronics P0622 2.1 mm chassis jack (8 mm hole); Jaycar PS0522 needs 11
usbc_slot   = [9.8, 4.0];   // receptacle opening for a USB-C PD trigger board glued inside
magnet_d    = 6.2;     // 6 x 2 mm N52 disc
magnet_t    = 2.2;
foot_d      = 13;      // 12.7 mm self-adhesive bumpers
```

SEN66 window, rear of the lower ring

```
// SEN66: 55.2 x 25.6 x 21.3, inlets and outlet on one 55.2 x 25.6 face. PMS5003: 50 x 38 x 21.
sen_l     = is_value ? 50 : 55.2;
sen_w     = is_value ? 38 : 25.6;
sen_t     = is_value ? 21 : 21.3;
sen_proud = 9;         // how far the sensor bump stands out of the shell
sen_skin  = 1.6;
sen_z0    = 56;        // bottom of the sensor, in lower-ring coordinates
sen_face  = r_out + sen_proud - sen_skin;
sen_frame_back = sen_face - sen_t + 2;
```

Top cap

```
cap_h        = 12;
grille_t     = 8;
grille_rib   = 1.6;
grille_gap   = 5.0;
grille_spoke = 2.4;
r_isl        = 52;
skin         = 2.0;
pod_h        = 20;
r_disc       = 34;
r_disc_fl    = 38;
pad_r        = 43;
pad_size     = 12;
pad_angles   = [150, 120, 90, 60, 30];   // power, mode, speed, timer, lock — near (front) side
lux_angle    = 270;
lux_r        = 45;
pod_bosses   = [0, 180, 225, 315];
```

Display + LEDs

```
disp_window_d = 33;      // GC9A01 1.28": 32.4 mm visible
disp_glass_d  = 38.5;    // Waveshare 1.28" round board is 37.5 mm
ring_od       = 65.5;    // 24 x WS2812B ring (Adafruit 1586 size)
ring_id       = 52.3;
led_skin      = 0.8;
```

Electronics tray

```
tray_h       = 32;
board_w      = 50;       // 50 x 70 mm perfboard, long side front-to-back, USB toward the rear exit
board_l      = 70;
```

Blower: set from your blower's datasheet

```
// Reference: ebm-papst R1G190-AB27-02 (24 V, 51 W). R1G175-AB63-02 shares the flange and eye.
// Value: a 160 x 45 mm 24 V EC backward-curved fan. These numbers are PLACEHOLDERS: replace them from
// the seller's drawing before exporting inlet_ring and blower_bridge.
blower_imp_d    = is_value ? 160 : 190;   // impeller outer diameter
blower_h        = is_value ? 55 : 67;     // inlet edge to the motor's back (mounting) face
blower_eye_d    = is_value ? 110 : 131;   // impeller eye, inner diameter: the nozzle lip goes inside it
blower_motor_d  = is_value ? 62 : 72;     // rotor can (rotates: the hub must not touch it)
blower_pcd      = is_value ? 50 : 58;     // M4 tapped holes in the motor back face, 4 x 90 deg
blower_holes    = 4;
blower_hole_d   = 4.5;   // M4 clearance
blower_flange_d = is_value ? 58 : 66;     // stationary back-face contact diameter
```

Head layout

```
blower_max_d   = 200;
blower_max_h   = is_value ? 65 : 95;
deck_t         = 6;      // must exceed the 4.3 mm socket so the deck laps into the wall
deck_bore_d    = 140;    // through-hole in the deck
ring_seat_d    = 156;    // counterbore for the inlet ring flange
ring_seat_t    = 3;
imp_gap        = 6;      // deck top to shroud bottom
nozzle_overlap = 2.5;    // nozzle lip inside the eye (ebm-papst's own nozzle overlaps 2 mm)
nozzle_clear   = 1.5;    // radial clearance between nozzle lip and eye
nozzle_wall    = 1.6;
bridge_t       = 10;
island_drop    = (pod_h + skin - cap_h) + tray_h;   // control pod + tray hanging below the head top
head_clear     = 6;
```

Side table

```
st_w        = 250;     // outer width and depth (fits a 256 mm bed)
st_n        = 5;       // superellipse exponent (2 = circle, larger = squarer)
st_wall     = 2.0;
st_ring_h   = 34;      // lower frame ring: bayonet, power input, hall sensor
st_rod      = 95;      // corner posts at x = y = +/-95
st_post     = 12;      // square corner posts, printed lying flat
st_sock     = 10;      // socket depth at each end
st_boss     = 19;      // square boss around each socket
st_louvre_h = 26;
st_rim_h    = 5;
st_top_t    = 4.5;
st_groove   = 2.5;     // elastic-cord groove (3 mm round elastic)
st_led_len  = 168;     // 10 LEDs from a 60/m WS2812B strip
st_pads_x   = [52, 26, 0, -26, -52];   // power, mode, speed, timer, lock: left to right seen from the front
st_usb_a    = 250;     // USB-C PD input, back face left of centre
st_board    = [70, 50];
```

Appendix B: Firmware listing

Complete ESPHome configuration as shipped: three entry files that pick packages around a shared core. All three compile on ESPHome 2026.9.1 (ESP-IDF 5.5.5) for the ESP32-S3.

firmware/openair.yaml (33 lines)

```
# OpenAir Tower, reference build: ebm-papst R1G190 blower, SEN66, round TFT, BH1750, 24-LED ring.
# ESPHome >= 2026.9. Pin map and wiring: ../docs/electronics.md

substitutions:
  name: openair
  friendly_name: OpenAir Tower
  timezone: Australia/Melbourne

  # Blower: defaults are for the ebm-papst R1G190-AB27-02 reference blower (docs/airflow.md).
  blower_pwm_freq: 2000Hz       # ebm-papst: 1-10 kHz, 100 % = max
  blower_pwm_inverted: "true"   # true with the open-drain driver on the board
  blower_min_power: "0.12"      # ebm-papst stops below ~10 % duty
  blower_pulses_per_rev: "2"    # tach pulses per revolution
  blower_max_rpm: "2950"        # tach reading at 100 % with a clean filter fitted
  airflow_at_max: "460"         # m3/h through a clean filter at 100 % (estimate; calibrate)
  filter_efficiency: "0.97"     # single-pass PM efficiency used for the CADR estimate

  # Filter life: whichever runs out first
  filter_rated_m3: "750000"
  filter_rated_days: "365"

  # Touch: on the S3 a pad fires when (reading - baseline) > threshold.
  # Calibrate with esp32_touch setup_mode: true, see docs/electronics.md.
  touch_threshold: "1500"

  # UI
  led_count: "24"

packages:
  core: !include packages/core.yaml
  air_sensor: !include sensors/sen66.yaml
  ui: !include packages/ui_tft.yaml
  light_sensor: !include packages/light_bh1750.yaml
```

firmware/openair-value.yaml (28 lines)

```
# OpenAir value build (tower): 160 mm 24 V EC blower on USB-C PD 20 V, PMS5003, LED-only UI, LDR.
# ESPHome >= 2026.9. Wiring: ../docs/value-build.md

substitutions:
  name: openair
  friendly_name: OpenAir
  timezone: Australia/Melbourne

  # Blower: placeholders for a 160 x 45 mm 24 V EC backward-curved fan. Set from the seller's datasheet.
  blower_pwm_freq: 1000Hz
  blower_pwm_inverted: "true"
  blower_min_power: "0.12"
  blower_pulses_per_rev: "3"
  blower_max_rpm: "3000"
  airflow_at_max: "250"
  filter_efficiency: "0.97"

  filter_rated_m3: "500000"
  filter_rated_days: "365"
  touch_threshold: "1500"

  led_count: "24"

packages:
  core: !include packages/core.yaml
  air_sensor: !include sensors/pms5003.yaml
  ui: !include packages/ui_leds.yaml
  light_sensor: !include packages/light_ldr.yaml
```

firmware/openair-sidetable.yaml (9 lines)

```
# OpenAir Side Table: the value build with a 10-LED light line on the front instead of the 24-LED ring.

substitutions:
  name: openair-table
```

```
  friendly_name: OpenAir Side Table
  led_count: "10"

packages:
  value: !include openair-value.yaml
```

firmware/packages/core.yaml (908 lines)

```
# OpenAir core logic, shared by every build. Entry files (openair*.yaml) provide substitutions
# and pick the air-sensor, UI and light-sensor packages.

# OpenAir Tower — ESPHome firmware (ESP32-S3-DevKitC-1 N16R8)
# Tested with ESPHome 2026.9.1. Pin map and wiring: ../docs/electronics.md

esphome:
  name: ${name}
  friendly_name: ${friendly_name}
  min_version: 2026.9.0
  project:
    name: diy.openair-tower
    version: "1.0.0"
  on_boot:
    - priority: 600
      then:
        - output.turn_off: blower_power
    - priority: -100
      then:
        - script.execute: refresh_ui
        - rtttl.play: "boot:d=16,o=6,b=180:c,e,g"

esp32:
  board: esp32-s3-devkitc-1
  flash_size: 16MB
  framework:
    type: esp-idf

psram:
  mode: octal
  speed: 80MHz

preferences:
  flash_write_interval: 5min

logger:
  level: INFO

api:
  encryption:
    key: !secret api_encryption_key
  actions:
    - action: reset_filter
      then:
        - script.execute: reset_filter
    - action: set_timer
      variables:
        hours: float
      then:
        - number.set:
            id: off_timer
            value: !lambda "return hours;"

ota:
  - platform: esphome
    encryption: {}

wifi:
  ssid: !secret wifi_ssid
  password: !secret wifi_password
  ap:
    ssid: "OpenAir Setup"
    password: !secret fallback_ap_password
  on_connect:
    - script.execute: refresh_ui
  on_disconnect:
    - script.execute: refresh_ui

captive_portal:

web_server:
  port: 80
  version: 3
  ota: false

time:
  - platform: sntp
    id: rtc
```

```
    timezone: ${timezone}
    on_time:
      - seconds: 0
        minutes: 0
        then:
          - lambda: |-
              if (!id(night_schedule).state) return;
              auto now = id(rtc).now();
              int h = now.hour;
              if (h == (int) id(night_start).state && id(purifier).state)
                id(set_mode).execute("Sleep");
              else if (h == (int) id(night_end).state && id(purifier).state &&
                       id(purifier).get_preset_mode() == "Sleep")
                id(set_mode).execute("Auto");

# ------------------------------------------------------------------ state

globals:
  - id: filter_m3
    type: float
    restore_value: true
    initial_value: "0"
  - id: filter_installed
    type: uint32_t
    restore_value: true
    initial_value: "0"
  - id: timer_end_ms
    type: uint32_t
    initial_value: "0"
  - id: auto_target
    type: float
    initial_value: "20"
  - id: pm_smoothed
    type: float
    initial_value: "NAN"
  - id: last_step_down_ms
    type: uint32_t
    initial_value: "0"
  - id: stall_count
    type: int
    initial_value: "0"
  - id: turbo_since_ms
    type: uint32_t
    initial_value: "0"
  - id: page
    type: int
    initial_value: "0"
  - id: lock_long_fired
    type: bool
    initial_value: "false"
  - id: timer_long_fired
    type: bool
    initial_value: "false"
  - id: auto_paused
    type: bool
    initial_value: "false"
  - id: speed_show_until
    type: uint32_t
    initial_value: "0"
  - id: lights_dark
    type: bool
    initial_value: "false"
  - id: light_level
    type: float
    initial_value: "0.5"

# ------------------------------------------------------------------ buses

esp32_touch:
  setup_mode: false
  measurement_duration: 0.25ms

# ------------------------------------------------------------------ outputs

output:
  - platform: ledc
    id: blower_pwm
    pin: GPIO15
    frequency: ${blower_pwm_freq}
    inverted: ${blower_pwm_inverted}
    min_power: ${blower_min_power}
    max_power: 1.0
    zero_means_zero: true
  - platform: gpio
    id: blower_power
    pin: GPIO17
  - platform: ledc
    id: buzzer_out
```

```
    pin: GPIO7

rtttl:
  id: buzzer
  output: buzzer_out
  gain: 60%

# ------------------------------------------------------------------ fan

fan:
  - platform: speed
    id: purifier
    name: Purifier
    icon: mdi:air-purifier
    output: blower_pwm
    speed_count: 100
    restore_mode: RESTORE_DEFAULT_OFF
    preset_modes:
      - Auto
      - Sleep
      - Turbo
    on_turn_on:
      - lambda: |-
          id(stall_count) = 0;
          id(auto_paused) = false;
      - if:
          condition:
            binary_sensor.is_off: base_locked
          then:
            - script.execute: interlock_trip
          else:
            - output.turn_on: blower_power
    on_turn_off:
      - output.turn_off: blower_power
      - lambda: id(auto_paused) = false;
      - number.set:
          id: off_timer
          value: 0
      - script.execute: refresh_ui
    on_preset_set:
      - script.execute:
          id: apply_preset
          preset: !lambda "return x.str();"
    on_speed_set:
      - script.execute: refresh_ui

script:
  # Fan triggers fire inside the fan's own state publish, so follow-up fan calls are deferred.
  - id: apply_preset
    mode: restart
    parameters:
      preset: string
    then:
      - delay: 20ms
      - lambda: |-
          if (preset == "Turbo") id(turbo_since_ms) = millis();
          if (preset == "Turbo" || preset == "Sleep") {
            int s = preset == "Turbo" ? 100 : (int) id(sleep_speed).state;
            if (id(purifier).speed != s) {
              auto call = id(purifier).turn_on();
              call.set_speed(s);
              call.set_preset_mode(preset.c_str());
              call.perform();
            }
          }
      - script.execute: refresh_ui

  - id: interlock_trip
    mode: restart
    then:
      - delay: 20ms
      - fan.turn_off: purifier
      - output.turn_off: blower_power
      - script.execute:
          id: beep
          tune: "err:d=8,o=5,b=200:c,p,c"
      - script.execute: refresh_ui

  - id: locked_feedback
    mode: restart
    then:
      - script.execute:
          id: beep
          tune: "no:d=16,o=5,b=200:c,p,c"
      - script.execute: ui_locked
      - delay: 1500ms
      - script.execute: refresh_ui
```

```
  - id: show_speed
    mode: restart
    then:
      - lambda: id(speed_show_until) = millis() + 3000;
      - delay: 50ms
      - script.execute: refresh_ui
      - delay: 3100ms
      - script.execute: refresh_ui

  - id: set_mode
    parameters:
      mode: string
    then:
      - lambda: |-
          auto call = id(purifier).turn_on();
          if (mode == "Manual") {
            int s = id(purifier).speed;
            call.set_speed(s < 1 ? 40 : s);
          } else {
            call.set_preset_mode(mode.c_str());
            if (mode == "Auto") call.set_speed((int) id(auto_target));
          }
          call.perform();

  - id: beep
    parameters:
      tune: string
    then:
      - if:
          condition:
            lambda: |-
              return id(sounds).state && id(purifier).get_preset_mode() != "Sleep";
          then:
            - rtttl.play: !lambda "return tune;"

  - id: reset_filter
    then:
      - lambda: |-
          id(filter_m3) = 0;
          auto now = id(rtc).now();
          id(filter_installed) = now.is_valid() ? now.timestamp : 0;
      - component.update: filter_life
      - script.execute:
          id: beep
          tune: "rst:d=8,o=6,b=180:c,e,g,c7"

  - id: refresh_ui
    mode: restart
    then:
      - lambda: |-
          bool on = id(purifier).state;
          float pm = id(pm25).has_state() ? id(pm25).state : NAN;
          bool sleep = on && id(purifier).get_preset_mode() == "Sleep";
          bool dark = id(ambient).has_state() && id(ambient).state < id(dark_lux).state;

          // ring colour by PM2.5 (µg/m3): WHO-2021-inspired indoor bands
          float r = 0.2, g = 0.2, b = 0.2;
          if (!std::isnan(pm)) {
            if (pm <= 5)        { r = 0.0; g = 0.6; b = 1.0; }
            else if (pm <= 12)  { r = 0.0; g = 1.0; b = 0.2; }
            else if (pm <= 25)  { r = 1.0; g = 0.85; b = 0.0; }
            else if (pm <= 50)  { r = 1.0; g = 0.45; b = 0.0; }
            else if (pm <= 100) { r = 1.0; g = 0.0; b = 0.0; }
            else                { r = 0.6; g = 0.0; b = 1.0; }
          }
          float lux = id(ambient).has_state() ? id(ambient).state : 200;
          float level = std::min(1.0f, std::max(0.08f, lux / 300.0f)) * id(led_brightness).state / 100.0f;
          bool lights_off = !on || sleep || (dark && id(dark_lights_off).state);

          auto ring = id(aq_ring).make_call();
          if (id(blower_fault).state || id(base_locked).state == false) {
            ring.set_state(true); ring.set_rgb(1, 0, 0); ring.set_brightness(0.6); ring.set_effect("Alert");
          } else if (lights_off) {
            ring.set_state(false);
          } else {
            ring.set_state(true); ring.set_rgb(r, g, b); ring.set_brightness(level);
            ring.set_effect(millis() < id(speed_show_until) ? "Speed" : "None");
          }
          ring.perform();
          id(lights_dark) = lights_off;
          id(light_level) = level;

      - script.execute: ui_update

# ------------------------------------------------------------------ sensors

sensor:
  - platform: pulse_counter
```

```
    id: blower_rpm
    name: Blower speed
    pin:
      number: GPIO16
      mode:
        input: true
        pullup: true
    unit_of_measurement: rpm
    icon: mdi:fan
    accuracy_decimals: 0
    update_interval: 2s
    internal_filter: 13us
    filters:
      - lambda: return x / ${blower_pulses_per_rev}.0f;
    entity_category: diagnostic

  - platform: template
    id: airflow
    name: Estimated airflow
    unit_of_measurement: m³/h
    icon: mdi:weather-windy
    accuracy_decimals: 0
    update_interval: 10s
    lambda: |-
      if (!id(purifier).state || !id(blower_rpm).has_state()) return 0.0f;
      return ${airflow_at_max} * std::min(1.2f, id(blower_rpm).state / ${blower_max_rpm}.0f);

  - platform: template
    id: cadr
    name: Estimated CADR
    unit_of_measurement: m³/h
    icon: mdi:air-filter
    accuracy_decimals: 0
    update_interval: 10s
    lambda: return id(airflow).state * ${filter_efficiency};

  - platform: template
    id: ach
    name: Air changes per hour
    unit_of_measurement: ACH
    icon: mdi:home-circle
    accuracy_decimals: 1
    update_interval: 10s
    lambda: return id(cadr).state / std::max(1.0f, (float) id(room_volume).state);

  - platform: template
    id: filter_life
    name: Filter life
    unit_of_measurement: "%"
    icon: mdi:air-filter
    accuracy_decimals: 0
    update_interval: 60s
    lambda: |-
      float by_volume = 1.0f - id(filter_m3) / ${filter_rated_m3}.0f;
      float by_age = 1.0f;
      auto now = id(rtc).now();
      if (now.is_valid() && id(filter_installed) > 0)
        by_age = 1.0f - (now.timestamp - id(filter_installed)) / (86400.0f * ${filter_rated_days});
      return std::max(0.0f, std::min(by_volume, by_age)) * 100.0f;

  - platform: template
    id: filter_volume
    name: Filter air processed
    unit_of_measurement: m³
    icon: mdi:counter
    accuracy_decimals: 0
    update_interval: 300s
    entity_category: diagnostic
    lambda: return id(filter_m3);

  - platform: template
    id: timer_left
    name: Off timer remaining
    unit_of_measurement: min
    icon: mdi:timer-outline
    accuracy_decimals: 0
    update_interval: 30s
    lambda: |-
      if (id(timer_end_ms) == 0) return 0.0f;
      return std::max(0.0f, (int32_t)(id(timer_end_ms) - millis()) / 60000.0f);

  - platform: wifi_signal
    name: Wi-Fi signal
    update_interval: 60s
    entity_category: diagnostic

  - platform: uptime
    name: Uptime
```

```
    entity_category: diagnostic

text_sensor:
  - platform: template
    name: Air quality
    icon: mdi:air-filter
    update_interval: 15s
    lambda: |-
      if (!id(pm25).has_state()) return {"unknown"};
      float pm = id(pm25).state;
      if (pm <= 5) return {"excellent"};
      if (pm <= 12) return {"good"};
      if (pm <= 25) return {"fair"};
      if (pm <= 50) return {"poor"};
      if (pm <= 100) return {"very poor"};
      return {"hazardous"};

binary_sensor:
  - platform: gpio
    id: base_locked
    name: Base locked
    device_class: lock
    pin:
      number: GPIO39
      mode:
        input: true
        pullup: true
      inverted: true
    filters:
      - delayed_on_off: 300ms
    on_release:
      - fan.turn_off: purifier
      - script.execute:
          id: beep
          tune: "open:d=8,o=5,b=200:g,c"
      - script.execute: refresh_ui
    on_press:
      - script.execute: refresh_ui

  - platform: template
    id: blower_fault
    name: Blower fault
    device_class: problem
    lambda: return id(stall_count) >= 4;
    on_press:
      - script.execute:
          id: beep
          tune: "fault:d=4,o=5,b=120:c,p,c,p,c"
      - script.execute: refresh_ui
    on_release:
      - script.execute: refresh_ui

  - platform: template
    id: filter_due
    name: Filter replacement due
    device_class: problem
    lambda: return id(filter_life).has_state() && id(filter_life).state <= 10;

  - platform: status
    name: Status

  # Touch pads, left to right from the front: power, mode, speed, timer, lock
  - platform: esp32_touch
    id: pad_power
    pin: GPIO1
    threshold: ${touch_threshold}
    on_click:
      min_length: 30ms
      max_length: 800ms
      then:
        - if:
            condition:
              switch.is_on: child_lock
            then:
              - script.execute: locked_feedback
            else:
              - lambda: |-
                  if (id(purifier).state) {
                    id(purifier).turn_off().perform();
                  } else if (id(purifier).has_preset_mode() || id(purifier).speed > 0) {
                    id(purifier).turn_on().perform();
                  } else {
                    id(set_mode).execute("Auto");
                  }
              - script.execute: { id: beep, tune: "k:d=32,o=6,b=200:c" }
              - script.execute: show_speed

  - platform: esp32_touch
```

```
    id: pad_mode
    pin: GPIO2
    threshold: ${touch_threshold}
    on_click:
      min_length: 30ms
      max_length: 800ms
      then:
        - if:
            condition:
              switch.is_on: child_lock
            then:
              - script.execute: locked_feedback
            else:
              - lambda: |-
                  auto p = id(purifier).get_preset_mode();
                  if (!id(purifier).state || p.empty()) id(set_mode).execute("Auto");
                  else if (p == "Auto") id(set_mode).execute("Sleep");
                  else if (p == "Sleep") id(set_mode).execute("Turbo");
                  else id(set_mode).execute("Manual");
              - script.execute: { id: beep, tune: "k:d=32,o=6,b=200:e" }
              - script.execute: show_speed

  - platform: esp32_touch
    id: pad_speed
    pin: GPIO4
    threshold: ${touch_threshold}
    on_click:
      min_length: 30ms
      max_length: 800ms
      then:
        - if:
            condition:
              switch.is_on: child_lock
            then:
              - script.execute: locked_feedback
            else:
              - lambda: |-
                  static const int steps[] = {20, 40, 60, 80, 100};
                  int cur = id(purifier).state ? id(purifier).speed : 0;
                  int next = steps[0];
                  if (id(purifier).state && id(purifier).get_preset_mode().empty()) {
                    for (int s : steps) if (s > cur) { next = s; break; }
                    if (cur >= 100) next = steps[0];
                  } else {
                    for (int s : steps) if (s >= cur) { next = s; break; }
                  }
                  auto call = id(purifier).turn_on();
                  call.set_speed(next);
                  call.perform();
              - script.execute: { id: beep, tune: "k:d=32,o=6,b=200:g" }
              - script.execute: show_speed

  - platform: esp32_touch
    id: pad_timer
    pin: GPIO5
    threshold: ${touch_threshold}
    on_press:
      - lambda: id(timer_long_fired) = false;
      - delay: 5s
      - if:
          condition:
            and:
              - binary_sensor.is_on: pad_timer
              - switch.is_off: child_lock
          then:
            - lambda: id(timer_long_fired) = true;
            - script.execute: reset_filter
    on_release:
      - if:
          condition:
            lambda: return !id(timer_long_fired);
          then:
            - if:
                condition:
                  switch.is_on: child_lock
                then:
                  - script.execute: locked_feedback
                else:
                  - lambda: |-
                      float h = id(off_timer).state;
                      float next = h < 1 ? 1 : h < 2 ? 2 : h < 4 ? 4 : h < 8 ? 8 : 0;
                      auto call = id(off_timer).make_call();
                      call.set_value(next);
                      call.perform();
                  - script.execute: { id: beep, tune: "k:d=32,o=7,b=200:c" }

  - platform: esp32_touch
    id: pad_lock
```

```
    pin: GPIO6
    threshold: ${touch_threshold}
    on_press:
      - lambda: id(lock_long_fired) = false;
      - delay: 3s
      - if:
          condition:
            binary_sensor.is_on: pad_lock
          then:
            - lambda: id(lock_long_fired) = true;
            - switch.toggle: child_lock
    on_release:
      - if:
          condition:
            lambda: return !id(lock_long_fired);
          then:
            - lambda: id(page) = (id(page) + 1) % 5;
            - script.execute: refresh_ui
            - script.execute: show_speed
            - script.execute: { id: beep, tune: "k:d=32,o=7,b=200:e" }

# ------------------------------------------------------------------- HA controls

switch:
  - platform: template
    id: child_lock
    name: Child lock
    icon: mdi:lock
    optimistic: true
    restore_mode: RESTORE_DEFAULT_OFF
    on_turn_on:
      - script.execute: { id: beep, tune: "lk:d=16,o=6,b=200:g,c" }
      - script.execute: refresh_ui
    on_turn_off:
      - script.execute: { id: beep, tune: "ul:d=16,o=6,b=200:c,g" }
      - script.execute: refresh_ui

  - platform: template
    id: sounds
    name: Sounds
    icon: mdi:volume-high
    optimistic: true
    restore_mode: RESTORE_DEFAULT_ON
    entity_category: config

  - platform: template
    id: dark_lights_off
    name: Lights off in the dark
    icon: mdi:theme-light-dark
    optimistic: true
    restore_mode: RESTORE_DEFAULT_OFF
    entity_category: config
    on_turn_on:
      - script.execute: refresh_ui
    on_turn_off:
      - script.execute: refresh_ui

  - platform: template
    id: night_schedule
    name: Night schedule
    icon: mdi:weather-night
    optimistic: true
    restore_mode: RESTORE_DEFAULT_OFF
    entity_category: config

  - platform: template
    id: auto_can_stop
    name: Auto stops when air is clean
    icon: mdi:fan-off
    optimistic: true
    restore_mode: RESTORE_DEFAULT_OFF
    entity_category: config

number:
  - platform: template
    id: off_timer
    name: Off timer
    icon: mdi:timer-outline
    unit_of_measurement: h
    min_value: 0
    max_value: 24
    step: 0.5
    optimistic: true
    initial_value: 0
    on_value:
      - lambda: |-
          id(timer_end_ms) = x > 0 ? millis() + (uint32_t)(x * 3600000.0f) : 0;
      - component.update: timer_left
```

```
    - script.execute: refresh_ui

- platform: template
  id: auto_max_speed
  name: Auto max speed
  icon: mdi:speedometer
  unit_of_measurement: "%"
  min_value: 20
  max_value: 100
  step: 5
  optimistic: true
  restore_value: true
  initial_value: 100
  entity_category: config

- platform: template
  id: sleep_speed
  name: Sleep speed
  icon: mdi:sleep
  unit_of_measurement: "%"
  min_value: 10
  max_value: 50
  step: 5
  optimistic: true
  restore_value: true
  initial_value: 15
  entity_category: config

- platform: template
  id: led_brightness
  name: Light brightness
  icon: mdi:brightness-6
  unit_of_measurement: "%"
  min_value: 5
  max_value: 100
  step: 5
  optimistic: true
  restore_value: true
  initial_value: 60
  entity_category: config
  on_value:
    - script.execute: refresh_ui

- platform: template
  id: dark_lux
  name: Dark threshold
  icon: mdi:brightness-4
  unit_of_measurement: lx
  min_value: 1
  max_value: 50
  step: 1
  optimistic: true
  restore_value: true
  initial_value: 5
  entity_category: config

- platform: template
  id: room_volume
  name: Room volume
  icon: mdi:cube-outline
  unit_of_measurement: m³
  min_value: 10
  max_value: 400
  step: 1
  optimistic: true
  restore_value: true
  initial_value: 96
  entity_category: config

- platform: template
  id: night_start
  name: Night starts (hour)
  icon: mdi:weather-night
  min_value: 0
  max_value: 23
  step: 1
  optimistic: true
  restore_value: true
  initial_value: 22
  entity_category: config

- platform: template
  id: night_end
  name: Night ends (hour)
  icon: mdi:weather-sunny
  min_value: 0
  max_value: 23
  step: 1
```

```
    optimistic: true
    restore_value: true
    initial_value: 7
    entity_category: config

button:
  - platform: template
    name: Reset filter life
    icon: mdi:air-filter
    entity_category: config
    on_press:
      - script.execute: reset_filter
  - platform: template
    name: Identify
    icon: mdi:bell-ring
    entity_category: diagnostic
    on_press:
      - rtttl.play: "id:d=8,o=6,b=180:c,e,g,e,c"
  - platform: restart
    name: Restart
    entity_category: diagnostic

# ------------------------------------------------------------------ control loops

interval:
  # Auto mode: smoothed PM2.5 and VOC/NOx -> target speed, stepping up fast and down slowly.
  - interval: 5s
    then:
      - lambda: |-
          if (!id(purifier).state || id(purifier).get_preset_mode() != "Auto") return;
          if (!id(pm25).has_state()) return;
          float pm = id(pm25).state;
          id(pm_smoothed) = std::isnan(id(pm_smoothed)) ? pm : id(pm_smoothed) * 0.8f + pm * 0.2f;
          float p = id(pm_smoothed);

          float want;
          if (p <= 3)       want = id(auto_can_stop).state ? 0 : 15;
          else if (p <= 6)  want = 25;
          else if (p <= 12) want = 35;
          else if (p <= 25) want = 55;
          else if (p <= 50) want = 75;
          else              want = 100;
          if (id(voc).has_state() && id(voc).state > 250) want = std::max(want, 55.0f);
          if (id(nox).has_state() && id(nox).state > 100) want = std::max(want, 55.0f);
          want = std::min(want, (float) id(auto_max_speed).state);

          uint32_t now = millis();
          float cur = id(auto_target);
          if (want > cur) {
            cur = std::min(want, cur + 10);
          } else if (want < cur && now - id(last_step_down_ms) > 60000) {
            cur = std::max(want, cur - 5);
            id(last_step_down_ms) = now;
          }
          if (want > id(auto_target)) id(last_step_down_ms) = now;
          id(auto_target) = cur;

          int s = (int) roundf(cur);
          if (s < 1) {
            if (!id(auto_paused)) id(blower_pwm).set_level(0);
            id(auto_paused) = true;
            return;
          }
          if (id(auto_paused) || s != id(purifier).speed) {
            id(auto_paused) = false;
            auto call = id(purifier).turn_on();
            call.set_speed(s);
            call.set_preset_mode("Auto");
            call.perform();
          }

  # Turbo reverts to Auto after 30 minutes; off timer; stall detection; filter accounting.
  - interval: 10s
    then:
      - lambda: |-
          uint32_t now = millis();
          if (id(purifier).state && id(purifier).get_preset_mode() == "Turbo" &&
              now - id(turbo_since_ms) > 30UL * 60UL * 1000UL)
            id(set_mode).execute("Auto");

          if (id(timer_end_ms) != 0 && (int32_t)(now - id(timer_end_ms)) >= 0) {
            id(timer_end_ms) = 0;
            auto call = id(purifier).turn_off();
            call.perform();
          }

          bool running = id(purifier).state && id(purifier).speed >= 5 && !id(auto_paused);
          if (running && id(blower_rpm).has_state() && id(blower_rpm).state < 60) id(stall_count)++;
```

```
          else id(stall_count) = 0;

          if (id(purifier).state && id(airflow).has_state())
            id(filter_m3) += id(airflow).state * 10.0f / 3600.0f;

          auto t = id(rtc).now();
          if (id(filter_installed) == 0 && t.is_valid() && t.timestamp > 1700000000)
            id(filter_installed) = t.timestamp;

  - interval: 30s
    then:
      - script.execute: refresh_ui

# ------------------------------------------------------------------ lights + display

light:
  - platform: esp32_rmt_led_strip
    id: aq_ring
    name: Air quality ring
    pin: GPIO18
    num_leds: ${led_count}
    chipset: WS2812
    channel_colors: GRB
    default_transition_length: 800ms
    restore_mode: ALWAYS_OFF
    effects:
      - addressable_lambda:
          name: Speed
          update_interval: 100ms
          lambda: |-
            int n = id(purifier).state ? (int) ceilf(id(purifier).speed * it.size() / 100.0f) : 0;
            for (int i = 0; i < it.size(); i++) it[i] = i < n ? current_color : Color::BLACK;
      - pulse:
          name: Alert
          transition_length: 400ms
          update_interval: 400ms
          min_brightness: 10%
          max_brightness: 80%
```

firmware/packages/ui_tft.yaml (167 lines)

```
# Round 1.28" GC9A01 TFT (LVGL) + PWM backlight. Reference build.

spi:
  clk_pin: GPIO12
  mosi_pin: GPIO11

output:
  - platform: ledc
    id: backlight_out
    pin: GPIO21
    frequency: 20000Hz

light:
  - platform: monochromatic
    id: backlight
    name: Display backlight
    output: backlight_out
    restore_mode: ALWAYS_ON
    default_transition_length: 400ms
    entity_category: config

script:
  - id: ui_locked
    then:
      - lvgl.label.update:
          id: lbl_status
          text: "LOCKED"

  - id: ui_update
    mode: restart
    then:
      - lambda: |-
          auto bl = id(backlight).make_call();
          if (id(lights_dark) && !id(blower_fault).state) bl.set_state(false);
          else { bl.set_state(true); bl.set_brightness(std::max(0.12f, id(light_level))); }
          bl.perform();
      - lvgl.label.update:
          id: lbl_value
          text: !lambda |-
            sensor::Sensor *src[] = {id(pm25), id(co2), id(voc), id(temperature), id(filter_life)};
            auto *sens = src[id(page) % 5];
            if (!sens->has_state() || std::isnan(sens->state)) return std::string("--");
            char buf[16];
            snprintf(buf, sizeof buf, id(page) == 3 ? "%.1f" : "%.0f", sens->state);
            return std::string(buf);
```

```
      - lvgl.label.update:
          id: lbl_unit
          text: !lambda |-
            static const char *units[] = {"PM2.5 µg/m³", "CO₂ ppm", "VOC index", "°C", "filter %"};
            return std::string(units[id(page) % 5]);
      - lvgl.label.update:
          id: lbl_mode
          text: !lambda |-
            if (id(base_locked).state == false) return std::string("BASE OPEN");
            if (id(blower_fault).state) return std::string("FAN FAULT");
            if (!id(purifier).state) return std::string("OFF");
            auto p = id(purifier).get_preset_mode();
            char buf[24];
            if (p.empty()) snprintf(buf, sizeof buf, "MANUAL %d%%", id(purifier).speed);
            else snprintf(buf, sizeof buf, "%s %d%%", p.c_str(), id(purifier).speed);
            std::string s(buf);
            for (auto &c : s) c = toupper(c);
            return s;
      - lvgl.label.update:
          id: lbl_status
          text: !lambda |-
            std::string s;
            if (id(child_lock).state) s += "LOCK ";
            if (id(timer_end_ms) != 0) {
              char buf[12];
              snprintf(buf, sizeof buf, "%.0fm ", id(timer_left).has_state() ? id(timer_left).state : 0.0f);
              s += buf;
            }
            if (id(filter_life).has_state() && id(filter_life).state <= 10) s += "FILTER ";
            if (!wifi::global_wifi_component->is_connected()) s += "NO WIFI";
            return s;
      - lvgl.arc.update:
          id: arc_speed
          value: !lambda "return id(purifier).state ? id(purifier).speed : 0;"

display:
  - platform: mipi_spi
    id: tft
    model: GC9A01A
    cs_pin: GPIO10
    dc_pin: GPIO13
    reset_pin: GPIO14
    invert_colors: true
    transform:
      swap_xy: false
      mirror_x: false
      mirror_y: true
    auto_clear_enabled: false
    update_interval: never

font:
  - file: "gfonts://Inter@700"
    id: font_value
    size: 72
    bpp: 4
    glyphs: "0123456789.-"
  - file: "gfonts://Inter@500"
    id: font_small
    size: 18
    bpp: 4
    glyphs: " %.-0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyzµ³₂°/"
  - file: "gfonts://Inter@500"
    id: font_status
    size: 14
    bpp: 4
    glyphs: " %.-0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZm"

lvgl:
  displays:
    - tft
  buffer_size: 25%
  color_depth: 16
  bg_color: 0x000000
  default_font: font_small
  pages:
    - id: main_page
      bg_color: 0x000000
      widgets:
        - arc:
            id: arc_speed
            align: CENTER
            width: 232
            height: 232
            start_angle: 135
            end_angle: 45
            min_value: 0
            max_value: 100
            value: 0
```

```
            adjustable: false
            arc_width: 8
            arc_color: 0x1A1A1A
            indicator:
              arc_color: 0x3FA7FF
              arc_width: 8
            knob:
              opa: TRANSP
        - label:
            id: lbl_value
            align: CENTER
            y: -12
            text_font: font_value
            text_color: 0xFFFFFF
            text: "--"
        - label:
            id: lbl_unit
            align: CENTER
            y: 38
            text_color: 0x9A9A9A
            text: "PM2.5"
        - label:
            id: lbl_mode
            align: CENTER
            y: -72
            text_color: 0xD0D0D0
            text: "OFF"
        - label:
            id: lbl_status
            align: CENTER
            y: 72
            text_font: font_status
            text_color: 0xFFB000
            text: ""
```

firmware/packages/ui_leds.yaml (15 lines)

```
# LED-only UI (no display): the ring/light line shows air quality, speed, faults. Value builds.

script:
  - id: ui_locked
    then:
      - light.turn_on:
          id: aq_ring
          red: 100%
          green: 50%
          blue: 0%
          effect: Alert

  - id: ui_update
    then:
      - lambda: ""
```

firmware/packages/light_bh1750.yaml (17 lines)

```
# BH1750 ambient light sensor on I2C. Reference build.

i2c:
  sda: GPIO8
  scl: GPIO9
  frequency: 100kHz
  scan: true

sensor:
  - platform: bh1750
    id: ambient
    name: Ambient light
    update_interval: 5s
    filters:
      - sliding_window_moving_average:
          window_size: 6
          send_every: 2
```

firmware/packages/light_ldr.yaml (19 lines)

```
# Light-dependent resistor (GL5528 to 3.3 V, 10 k to GND) on GPIO9: rough lux for auto-dimming. Value builds.

sensor:
  - platform: adc
    id: ambient
    name: Ambient light
```

```
    pin: GPIO9
    attenuation: 12db
    unit_of_measurement: lx
    device_class: illuminance
    accuracy_decimals: 0
    update_interval: 2s
    filters:
      - lambda: |-
          float r = 10000.0f * (3.3f - x) / std::max(x, 0.005f);
          return std::min(2000.0f, 10.0f * powf(15000.0f / std::max(r, 1.0f), 1.0f / 0.7f));
      - sliding_window_moving_average:
          window_size: 5
          send_every: 2
```

firmware/sensors/sen66.yaml (39 lines)

```
# Sensirion SEN66 on I2C (0x6B): PM1/2.5/4/10, RH, T, VOC index, NOx index, CO2. 3.3 V supply only.
i2c:
  sda: GPIO8
  scl: GPIO9
  frequency: 100kHz
  scan: true

sensor:
  - platform: sen6x
    type: SEN66
    update_interval: 10s
    pm_1_0:
      name: PM1.0
    pm_2_5:
      name: PM2.5
      id: pm25
      on_value:
        - script.execute: refresh_ui
    pm_4_0:
      name: PM4.0
    pm_10_0:
      name: PM10
      id: pm10
    temperature:
      name: Temperature
      id: temperature
    humidity:
      name: Humidity
      id: humidity
    voc_index:
      name: VOC index
      id: voc
    nox_index:
      name: NOx index
      id: nox
    co2:
      name: CO2
      id: co2
```

firmware/sensors/pms5003.yaml (47 lines)

```
# Budget option: Plantower PMS5003 on UART (PM1/2.5/10 only). 5 V supply, 3.3 V logic.
# Wire PMS TX -> GPIO40, PMS RX -> GPIO41. The gas/climate ids stay defined (never published)
# so the shared display and auto-mode logic compile unchanged.
uart:
  rx_pin: GPIO40
  tx_pin: GPIO41
  baud_rate: 9600

sensor:
  - platform: pmsx003
    type: PMSX003
    pm_1_0:
      name: PM1.0
    pm_2_5:
      name: PM2.5
      id: pm25
      on_value:
        - script.execute: refresh_ui
    pm_10_0:
      name: PM10
      id: pm10

  - platform: template
    id: temperature
    internal: true
    update_interval: never
```

```
    lambda: return NAN;
  - platform: template
    id: humidity
    internal: true
    update_interval: never
    lambda: return NAN;
  - platform: template
    id: voc
    internal: true
    update_interval: never
    lambda: return NAN;
  - platform: template
    id: nox
    internal: true
    update_interval: never
    lambda: return NAN;
  - platform: template
    id: co2
    internal: true
    update_interval: never
    lambda: return NAN;
```

firmware/secrets.example.yaml (6 lines)

```
# Copy to secrets.yaml (or add these keys to your Home Assistant ESPHome secrets).
wifi_ssid: "your-ssid"
wifi_password: "your-wifi-password"
fallback_ap_password: "choose-8+-chars"
# 32 random bytes, base64. Generate with: openssl rand -base64 32
api_encryption_key: "REPLACE_WITH_BASE64_KEY"
```

Appendix C: OpenSCAD source

Complete parametric source. Every part module is here; the per-part entry files in `cad/parts/` just call these modules.

cad/openair.scad (686 lines)

```
// OpenAir Tower — parametric 3D-printed air purifier
// Library: shared parameters and every part module. Per-part entry files live in parts/.
// Units: mm. World origin: floor centre, +Y = front, −Y = back (power jack side).

/* [Build profile] */
// "reference": ebm-papst R1G190 blower, SEN66, round TFT, 24 V brick + DC jack, brass inserts.
// "value":     160 mm 24 V EC blower, PMS5003, LED-only UI, USB-C PD input, self-tapping screws.
build    = "reference";
is_value = build == "value";

/* [Filter: measure yours] */
filter_od     = 200;   // Xiaomi M8R-FLH / SCG4021GL family: 200 x 293 mm
filter_h      = 293;
filter_id     = 144;   // open-top hole; listings disagree (117–145), the seal design tolerates it
filter_spacer = 0;     // raise the filter if yours is shorter than 293

/* [Body] */
od      = 240;
wall    = 3;
fit     = 0.3;         // radial sliding clearance
$fn     = 160;

/* [Base and bayonet] */
base_h      = is_value ? 20 : 30;
collar_h    = 24;
collar_wall = 3;
band_h      = 26;
band_t      = 3;
lug_w       = 12;
lug_h       = 5;
lug_t       = 2.6;
lug_z       = 11.5;
lock_angle  = 35;
detent_h    = 0.4;

/* [Seal] */
gasket_thk_compressed = 3;    // two rings of 9x6 mm EPDM tape, ~50 % squeeze
gasket_r              = [78, 90];
deck_hole_d           = 136;

/* [Shell perforation] */
slot_w     = 4;
slot_h     = 20;
slot_gap   = 5.2;
slot_cols  = 84;
spine_half = is_value ? 16 : 13;   // degrees of solid "spine" at the back: power, sensor window, wiring
guide_angle = spine_half - 2.5;

/* [Fasteners] */
use_inserts  = !is_value;
insert_d     = use_inserts ? 4.0 : 2.8;   // M3 x 5.7 brass heat-set insert, or a pilot for M3 self-tapping screws
insert_depth = 6.5;
screw_d      = 3.4;
screw_head_d = 6.5;
boss_angles  = [45, 135, 225, 315];

/* [Hardware] */
power_input = is_value ? "usbc_pd" : "dc_jack";
dc_jack_d   = 8.2;     // Altronics P0622 2.1 mm chassis jack (8 mm hole); Jaycar PS0522 needs 11
usbc_slot   = [9.8, 4.0];   // receptacle opening for a USB-C PD trigger board glued inside
magnet_d    = 6.2;     // 6 x 2 mm N52 disc
magnet_t    = 2.2;
foot_d      = 13;      // 12.7 mm self-adhesive bumpers

r_out      = od / 2;
r_in       = r_out - wall;
r_band     = r_in - band_t;
r_collar_o = r_band - fit;
r_collar_i = r_collar_o - collar_wall;
r_boss_in  = r_in - 9;
r_boss_mid = (r_in + r_boss_in) / 2;

platform_z   = base_h + 4 + filter_spacer;
filter_top_z = platform_z + filter_h;
deck_z       = filter_top_z + gasket_thk_compressed;
shell_h      = deck_z - base_h;
ring_h       = shell_h / 2;
```

```
lower_z      = base_h;
upper_z      = base_h + ring_h;

eps = 0.01;

// ------------------------------------------------------------- helpers

module tube(ro, ri, h) {
    difference() {
        cylinder(r = ro, h = h);
        translate([0, 0, -eps]) cylinder(r = ri, h = h + 2 * eps);
    }
}

module arc_block(r1, r2, a0, a1, z0, z1) {
    translate([0, 0, z0])
        rotate([0, 0, a0])
            rotate_extrude(angle = a1 - a0, $fn = 360)
                translate([r1, 0]) square([r2 - r1, z1 - z0]);
}

function deg_for(len, r) = len / r * 180 / PI;

module radial_slot(w, h, depth) {
    rotate([90, 0, 0])
        linear_extrude(height = depth, center = true)
            polygon([[0, -h / 2], [w / 2, -h / 2 + w / 2], [w / 2, h / 2 - w / 2],
                     [0, h / 2], [-w / 2, h / 2 - w / 2], [-w / 2, -h / 2 + w / 2]]);
}

function in_spine(a) = let(d = abs(((a - 270) % 360 + 540) % 360 - 180)) d < spine_half;

function near_boss(a, z0, z1, ranges) =
    len([for (b = boss_angles, r = ranges)
         if (abs(((a - b) % 360 + 540) % 360 - 180) < deg_for(slot_w / 2 + 5.5 + 1.5, r_in)
             && z1 > r[0] - 1 && z0 < r[1] + 1) 1]) > 0;

module perforation(z0, z1, boss_ranges = []) {
    n = floor((z1 - z0 + slot_gap) / (slot_h + slot_gap));
    used = n * slot_h + (n - 1) * slot_gap;
    zs = z0 + (z1 - z0 - used) / 2 + slot_h / 2;
    for (row = [0 : n - 1], col = [0 : slot_cols - 1]) {
        a = (col + (row % 2) * 0.5) * 360 / slot_cols;
        zc = zs + row * (slot_h + slot_gap);
        if (!in_spine(a) && !near_boss(a, zc - slot_h / 2, zc + slot_h / 2, boss_ranges))
            rotate([0, 0, a - 90])
                translate([0, r_in + wall / 2, zc])
                    radial_slot(slot_w, slot_h, wall + 2);
    }
}

module cable_guide(z) {
    rotate([0, 0, 270 - 90])
    translate([0, 0, z])
    difference() {
        hull() {
            translate([-5, r_in - 8, 0]) cube([10, 8 + 0.2, 10]);
            translate([-5, r_in - 0.2, -8]) cube([10, 0.2, 8]);
        }
        translate([-3, r_in - 6.5, -9]) cube([6, 5, 20]);
    }
}

// M3 boss with a through-hole and a deep counterbore, gusseted underneath so it prints upright.
module through_boss(a, ztop, h = 16) {
    w = 11;
    rotate([0, 0, a - 90])
    difference() {
        union() {
            translate([-w / 2, r_boss_in, ztop - h]) cube([w, r_in - r_boss_in + 0.2, h]);
            hull() {
                translate([-w / 2, r_boss_in, ztop - h]) cube([w, r_in - r_boss_in + 0.2, eps]);
                translate([-w / 2, r_in - 0.2, ztop - h - (r_in - r_boss_in)]) cube([w, 0.4, eps]);
            }
        }
        translate([0, r_boss_mid, ztop - h - 20]) cylinder(d = screw_d, h = h + 21, $fn = 32);
        translate([0, r_boss_mid, ztop - h - 20]) cylinder(d = screw_head_d, h = h + 20 - 5, $fn = 32);
    }
}

module insert_boss(a, zbot, h = 12) {
    w = 11;
    rotate([0, 0, a - 90])
    difference() {
        hull() {
            translate([-w / 2, r_boss_in, zbot]) cube([w, r_in - r_boss_in + 0.2, h]);
            translate([-w / 2, r_in - 0.2, zbot]) cube([w, 0.4, h + (r_in - r_boss_in)]);
        }
```

```
        translate([0, r_boss_mid, zbot - eps]) cylinder(d = insert_d, h = insert_depth, $fn = 32);
    }
}

module spigot(ztop) {
    translate([0, 0, ztop - eps]) tube(r_in + 1.5, r_in, 4);
}

module socket_cut(zbot, chamfer = true) {
    translate([0, 0, zbot])
        rotate_extrude($fn = 360)
            polygon(chamfer
                ? [[r_in - eps, -eps], [r_in + 1.7, -eps], [r_in + 1.7, 4.3], [r_in - eps, 6.0]]
                : [[r_in - eps, -eps], [r_in + 1.7, -eps], [r_in + 1.7, 4.3], [r_in - eps, 4.3]]);
}

// ------------------------------------------------------------------ filter (reference only)

module filter_dummy() {
    color("WhiteSmoke")
    translate([0, 0, platform_z])
    difference() {
        cylinder(d = filter_od, h = filter_h);
        translate([0, 0, 12]) cylinder(d = filter_id, h = filter_h);
    }
}

// ------------------------------------------------------------------ base

module lug(a) {
    rotate([0, 0, a - deg_for(lug_w, r_collar_o) / 2])
        rotate_extrude(angle = deg_for(lug_w, r_collar_o), $fn = 360)
            translate([r_collar_o - 0.5, base_h + lug_z]) square([lug_t + 0.5, lug_h]);
}

module base() {
    difference() {
        union() {
            cylinder(r = r_out, h = base_h);
            translate([0, 0, base_h - eps]) tube(r_collar_o, r_collar_i, collar_h);
            translate([0, 0, base_h - eps]) cylinder(r = r_collar_i + eps, h = platform_z - base_h);
            for (i = [0 : 3])
                arc_block(filter_od / 2 + 2, filter_od / 2 + 4.4, i * 90 + 12, i * 90 + 78,
                          platform_z - 1, platform_z + 8);
            for (i = [0 : 2]) lug(i * 120);
        }
        // bottom edge chamfer
        translate([0, 0, -eps]) difference() {
            cylinder(r = r_out + 1, h = 1.2);
            cylinder(r1 = r_out - 1.2, r2 = r_out + 0.01, h = 1.2);
        }
        // twist grip
        for (i = [0 : 71])
            rotate([0, 0, i * 5])
                translate([r_out, 0, 6]) rotate([0, 0, 45]) translate([-1, -1, 0]) cube([2, 2, base_h - 10]);
        // feet
        for (i = [0 : 3])
            rotate([0, 0, 45 + i * 90]) translate([0, r_out - 22, -eps]) cylinder(d = foot_d, h = 1.2);
        // interlock magnet, radial, on the collar's outer face at the back
        rotate([0, 0, 270 - 90])
            translate([0, r_collar_o - magnet_t + eps, base_h + 5])
                rotate([-90, 0, 0]) cylinder(d = magnet_d, h = magnet_t + 0.1, $fn = 32);
        // lock / unlock arrows on the skirt
        for (s = [-1, 1])
            rotate([0, 0, 90 + s * 8])
                translate([0, r_out - 0.6, base_h - 8]) rotate([-90, 0, 0])
                    linear_extrude(1) polygon(s > 0 ? [[-3, 0], [3, 2.5], [3, -2.5]] : [[3, 0], [-3, 2.5], [-3,
-2.5]]);
        // thumb notch to lift the filter out
        rotate([0, 0, 90]) translate([0, filter_od / 2 - 8, platform_z - 2.5]) cylinder(d = 22, h = 10);
    }
}

// ------------------------------------------------------------------ shell rings

/* [SEN66 window, rear of the lower ring] */
// SEN66: 55.2 x 25.6 x 21.3, inlets and outlet on one 55.2 x 25.6 face. PMS5003: 50 x 38 x 21.
sen_l     = is_value ? 50 : 55.2;
sen_w     = is_value ? 38 : 25.6;
sen_t     = is_value ? 21 : 21.3;
sen_proud = 9;         // how far the sensor bump stands out of the shell
sen_skin  = 1.6;
sen_z0    = 56;        // bottom of the sensor, in lower-ring coordinates
sen_face  = r_out + sen_proud - sen_skin;
sen_frame_back = sen_face - sen_t + 2;

module sensor_bump() {
    w = sen_w + 9; h = sen_l + 9;
```

```
    rotate([0, 0, 270 - 90])
        hull() for (x = [-w / 2 + 5, w / 2 - 5], z = [sen_z0 - 4.5 + 5, sen_z0 - 4.5 + h - 5])
            translate([x, r_in - 1, z]) rotate([-90, 0, 0]) cylinder(r = 5, h = r_out + sen_proud - r_in + 1, $fn =
40);
}

module sensor_frame() {
    rotate([0, 0, 270 - 90]) {
        difference() {
            translate([-(sen_w / 2 + 2.8), sen_frame_back, sen_z0 - 2.8]) cube([sen_w + 5.6, sen_face -
sen_frame_back, sen_l + 5.6]);
            translate([-(sen_w / 2 + 0.3), sen_frame_back - 1, sen_z0 - 0.3]) cube([sen_w + 0.6, sen_face -
sen_frame_back + 1 + eps, sen_l + 0.6]);
        }
        for (x = [-1, 1]) translate([x * (sen_w / 2 + 5), sen_frame_back, sen_z0 + sen_l / 2]) rotate([-90, 0, 0])
            difference() { cylinder(d = 7, h = sen_face - sen_frame_back, $fn = 32); translate([0, 0, -1]) cylinder(d
= 2.2, h = 9, $fn = 16); }
    }
}

module sensor_cuts() {
    rotate([0, 0, 270 - 90]) {
        translate([-(sen_w / 2 + 0.3), sen_frame_back - 1, sen_z0 - 0.3]) cube([sen_w + 0.6, sen_face - sen_frame_back
+ 1, sen_l + 0.6]);
        // face vents: two groups (ends) split by a solid band in the middle
        for (z = [sen_z0 + 3 : 4.2 : sen_z0 + sen_l - 4])
            if (abs(z + 1 - (sen_z0 + sen_l / 2)) > 5)
                translate([-(sen_w / 2 - 2.5), sen_face - 1, z]) cube([sen_w - 5, sen_skin + 12, 2.0]);
    }
}

// Clip that holds the SEN66 in its window (two M2.5 x 8 self-tappers).
module sensor_clip() {
    difference() {
        hull() for (x = [-1, 1]) translate([x * (sen_w / 2 + 5), 0, 0]) cylinder(d = 8, h = 2.4, $fn = 32);
        for (x = [-1, 1]) translate([x * (sen_w / 2 + 5), 0, -1]) cylinder(d = 2.8, h = 5, $fn = 16);
    }
}

module bayonet_band_cuts() {
    a_lug = deg_for(lug_w, r_band);
    a_clr = deg_for(1.2, r_band);
    r_cut = r_collar_o + lug_t + 0.4;
    for (i = [0 : 2]) {
        ae = i * 120 - lock_angle;
        // vertical entry with a lead-in
        arc_block(r_band - 1, r_cut, ae - a_lug / 2 - a_clr, ae + a_lug / 2 + a_clr, -eps, lug_z + lug_h + 1);
        rotate([0, 0, ae - a_lug / 2 - a_clr])
            rotate_extrude(angle = a_lug + 2 * a_clr, $fn = 360)
                polygon([[r_band - 1, -eps], [r_cut + 1.0, -eps], [r_cut, 1.0], [r_band - 1, 1.0]]);
        // locking run: flat floor, 45-degree roof so it prints without support
        rotate([0, 0, ae - a_lug / 2 - a_clr])
            rotate_extrude(angle = lock_angle + a_lug + 2 * a_clr, $fn = 360)
                polygon([[r_band - 1, lug_z - 0.2], [r_cut, lug_z - 0.2],
                         [r_cut, lug_z + lug_h + 1.2], [r_band - 1, lug_z + lug_h + 1.2 + (r_cut - r_band + 1)]]);
    }
}

module bayonet_detents() {
    a_lug = deg_for(lug_w, r_band);
    r_cut = r_collar_o + lug_t + 0.4;
    for (i = [0 : 2]) {
        ae = i * 120 - lock_angle;
        a_d = ae + lock_angle - a_lug / 2 - 2.5;
        hull() {
            arc_block(r_band, r_cut + eps, a_d - 1.2, a_d - 1.1, lug_z - 0.3, lug_z - 0.2);
            arc_block(r_band, r_cut + eps, a_d, a_d + 0.4, lug_z - 0.3, lug_z - 0.2 + detent_h);
            arc_block(r_band, r_cut + eps, a_d + 1.6, a_d + 1.7, lug_z - 0.3, lug_z - 0.2);
        }
    }
}

module hall_pocket() {
    rotate([0, 0, 270 - 90]) {
        translate([-2.4, r_band - eps, 3]) cube([4.8, 1.8, 4.2]);
        translate([-1.75, r_band - eps, 7]) cube([3.5, 2.0, band_h]);
    }
}

module shell_lower() {
    h = ring_h;
    difference() {
        union() {
            tube(r_out, r_in, h);
            tube(r_in + eps, r_band, band_h);
            translate([0, 0, band_h - eps]) difference() {
                cylinder(r = r_in + eps, h = band_t);
                translate([0, 0, -eps]) cylinder(r1 = r_band, r2 = r_in, h = band_t + 2 * eps);
```

```
            }
            spigot(h);
            for (a = boss_angles) through_boss(a, h);
            for (z = [70, 112]) rotate([0, 0, guide_angle]) cable_guide(z);
            sensor_bump();
            sensor_frame();
        }
        bayonet_band_cuts();
        hall_pocket();
        sensor_cuts();
        translate([0, 0, -1]) cylinder(r = filter_od / 2 + 3, h = h + 10);
        perforation(band_h + band_t + 4, h - 6, [[h - 25, h]]);
        if (power_input == "dc_jack")
            rotate([0, 0, 270 - 90]) translate([0, r_out + 1, 40]) rotate([90, 0, 0]) cylinder(d = dc_jack_d, h = 2 *
wall + 2, $fn = 48);
        else
            rotate([0, 0, 270 - 90]) translate([0, r_out + 1, 40]) rotate([90, 0, 0])
                linear_extrude(2 * wall + 2) offset(r = usbc_slot[1] / 2 - 0.01) square([usbc_slot[0] - usbc_slot[1],
0.02], center = true);
    }
    bayonet_detents();
}

module shell_upper() {
    h = ring_h;
    difference() {
        union() {
            tube(r_out, r_in, h);
            spigot(h);
            for (a = boss_angles) {
                through_boss(a, h);
                insert_boss(a, 0);
            }
            for (z = [40, 92]) rotate([0, 0, guide_angle]) cable_guide(z);
        }
        socket_cut(0);
        perforation(7.5, h - 6, [[0, 21], [h - 25, h]]);
    }
}

// ------------------------------------------------------------------ top cap, control disc, electronics tray
// Top cap frame: z = 0 is where it seats on the head, top surface at z = cap_h. Print it upside down.

/* [Top cap] */
cap_h        = 12;
grille_t     = 8;
grille_rib   = 1.6;
grille_gap   = 5.0;
grille_spoke = 2.4;
r_isl        = 52;
skin         = 2.0;
pod_h        = 20;
r_disc       = 34;
r_disc_fl    = 38;
pad_r        = 43;
pad_size     = 12;
pad_angles   = [150, 120, 90, 60, 30];   // power, mode, speed, timer, lock — near (front) side
lux_angle    = 270;
lux_r        = 45;
pod_bosses   = [0, 180, 225, 315];

/* [Display + LEDs] */
disp_window_d = 33;      // GC9A01 1.28": 32.4 mm visible
disp_glass_d  = 38.5;    // Waveshare 1.28" round board is 37.5 mm
ring_od       = 65.5;    // 24 x WS2812B ring (Adafruit 1586 size)
ring_id       = 52.3;
led_skin      = 0.8;

/* [Electronics tray] */
tray_h       = 32;
board_w      = 50;       // 50 x 70 mm perfboard, long side front-to-back, USB toward the rear exit
board_l      = 70;

module pad_icon(i) {
    // drawn upright in +Y, rotated so it reads from the front of the unit
    if (i == 0) { difference() { circle(d = 7); circle(d = 5); translate([-1.6, 0]) square([3.2, 5]); }
translate([-0.6, 0]) square([1.2, 4.5]); }
    if (i == 1) text("A", size = 6, font = "Liberation Sans:style=Bold", halign = "center", valign = "center");
    if (i == 2) for (k = [0 : 2]) rotate(k * 120) hull() { circle(d = 1.8); translate([0, 3.6]) circle(d = 2.8); }
    if (i == 3) { difference() { circle(d = 7.4); circle(d = 5.6); } translate([-0.5, 0]) square([1, 2.6]);
rotate(-60) translate([-0.5, 0]) square([1, 2.0]); }
    if (i == 4) { translate([-3, -3.5]) square([6, 4.5]); difference() { translate([0, 1]) circle(d = 5);
translate([0, 1]) circle(d = 3.2); translate([-3, -2]) square([6, 3]); } }
}

module grille_2d() {
    difference() {
        circle(r = r_in + 0.8);
```

```
        circle(r = r_isl);
        for (r = [r_isl + 3 : grille_gap + grille_rib : r_in - 4])
            difference() { circle(r = r + grille_gap); circle(r = r); }
    }
    for (k = [0 : 11]) rotate(k * 30 + 15) translate([r_isl - 1, -grille_spoke / 2]) square([r_in - r_isl,
grille_spoke]);
}

module top_cap() {
    difference() {
        union() {
            tube(r_out, r_in, cap_h);
            translate([0, 0, cap_h - grille_t]) linear_extrude(grille_t) grille_2d();
            translate([0, 0, cap_h - skin]) cylinder(r = r_isl + 0.5, h = skin);
            translate([0, 0, cap_h - pod_h - skin]) tube(r_isl + 0.5, r_isl - 1.5, pod_h + skin);
            for (a = boss_angles) rotate([0, 0, a - 90])
                hull() {
                    translate([-6, r_boss_in - 1, cap_h - grille_t]) cube([12, r_in - r_boss_in + 1.2, grille_t]);
                    translate([-8, r_in - 0.2, cap_h - grille_t]) cube([16, 0.4, grille_t]);
                }
            for (a = pod_bosses) rotate([0, 0, a - 90])
                translate([0, r_isl - 4, cap_h - pod_h - skin]) cylinder(d = 8, h = pod_h, $fn = 32);
        }
        socket_cut(0);
        // countersunk M3 into the head's top inserts
        for (a = boss_angles) rotate([0, 0, a - 90]) translate([0, r_boss_mid, 0]) {
            translate([0, 0, -1]) cylinder(d = screw_d, h = cap_h + 2, $fn = 32);
            translate([0, 0, cap_h - 1.8]) cylinder(d1 = screw_d, d2 = 6.4, h = 1.8 + eps, $fn = 32);
        }
        // control disc opening
        translate([0, 0, cap_h - skin - 1]) cylinder(r = r_disc + 0.2, h = skin + 2);
        // touch-pad recesses underneath and icons on top
        for (i = [0 : len(pad_angles) - 1]) rotate([0, 0, pad_angles[i]]) translate([pad_r, 0, 0]) {
            rotate([0, 0, -pad_angles[i] - 90])
                translate([-pad_size / 2, -pad_size / 2, cap_h - skin - 1]) cube([pad_size, pad_size, 1 + 0.6]);
            translate([0, 0, cap_h - 0.6]) linear_extrude(0.6 + eps) rotate(-pad_angles[i] + 180 + 90 - 90)
pad_icon(i);
        }
        // light-sensor window
        rotate([0, 0, lux_angle]) translate([lux_r, 0, cap_h - skin - 1]) cylinder(d = 2.6, h = skin + 2, $fn = 24);
        // pod screw holes for the tray (M3 self-tap 2.8 mm)
        for (a = pod_bosses) rotate([0, 0, a - 90])
            translate([0, r_isl - 4, cap_h - pod_h - skin - 1]) cylinder(d = 2.8, h = 12, $fn = 24);
    }
}

// Control disc: print in white or natural PETG so the LED ring glows through the thin skin.
module control_disc() {
    t = skin + 1.6;
    difference() {
        union() {
            cylinder(r = r_disc, h = t);
            cylinder(r = r_disc_fl, h = 1.6);
        }
        if (!is_value) {
            translate([0, 0, -1]) cylinder(d = disp_window_d, h = t + 2);
            translate([0, 0, -eps]) cylinder(d = disp_glass_d, h = t - led_skin - 0.4);
            translate([0, 0, t - 0.4]) difference() { cylinder(d = disp_window_d + 1.6, h = 0.4 + eps); cylinder(d1 =
disp_window_d, d2 = disp_window_d + 0.8, h = 0.4 + eps); }
        } else {
            translate([0, 0, t - 0.5]) linear_extrude(0.5 + eps)
                text("OpenAir", size = 6, font = "Liberation Sans:style=Bold", halign = "center", valign = "center");
        }
        translate([0, 0, -eps]) tube(ring_od / 2 + 0.3, ring_id / 2 - 0.3, t - led_skin);
    }
}

// Electronics tray: hangs under the control pod, carries a 50 x 70 perfboard with the ESP32-S3 DevKitC-1.
module electronics_tray() {
    difference() {
        union() {
            difference() {
                cylinder(r = r_isl + 0.5, h = tray_h);
                translate([0, 0, 2.4]) cylinder(r = r_isl - 1.5, h = tray_h);
            }
            for (a = pod_bosses) rotate([0, 0, a - 90])
                translate([0, r_isl - 4, 0]) cylinder(d = 8, h = tray_h, $fn = 32);
            for (sx = [-1, 1], sy = [-1, 1])
                translate([sx * (board_w / 2 - 3.5), sy * (board_l / 2 - 3.5), 0]) cylinder(d = 7, h = 8, $fn = 32);
        }
        for (a = pod_bosses) rotate([0, 0, a - 90]) translate([0, r_isl - 4, -1]) {
            cylinder(d = screw_d, h = tray_h + 2, $fn = 32);
            cylinder(d = screw_head_d, h = tray_h - 6, $fn = 32);
        }
        for (sx = [-1, 1], sy = [-1, 1])
            translate([sx * (board_w / 2 - 3.5), sy * (board_l / 2 - 3.5), 1]) cylinder(d = 2.5, h = 10, $fn = 24);
        // cable exit toward the back: blower, sensor, power, hall and the USB port
        rotate([0, 0, 270]) translate([r_isl - 6, -14, 6]) cube([10, 28, tray_h]);
```

```
        // vent slots in the floor
        for (x = [-30 : 6 : 30]) translate([x - 1.5, -18, -1]) cube([3, 36, 4]);
    }
}

// ------------------------------------------------------------------ head: blower housing
// Head frame: z = 0 is the deck underside (sits on the upper shell ring and on the filter gasket).
// The blower is an EC backward-curved "plug fan": inlet eye facing down into the filter core,
// external-rotor motor on top, hung from the bridge by its stator flange.
// head_body is universal (any plug fan up to blower_max_d x blower_max_h). Only inlet_ring and
// blower_bridge depend on the blower you buy: set the [Blower] block and re-export those two.

/* [Blower: set from your blower's datasheet] */
// Reference: ebm-papst R1G190-AB27-02 (24 V, 51 W). R1G175-AB63-02 shares the flange and eye.
// Value: a 160 x 45 mm 24 V EC backward-curved fan. These numbers are PLACEHOLDERS: replace them from
// the seller's drawing before exporting inlet_ring and blower_bridge.
blower_imp_d    = is_value ? 160 : 190;   // impeller outer diameter
blower_h        = is_value ? 55 : 67;     // inlet edge to the motor's back (mounting) face
blower_eye_d    = is_value ? 110 : 131;   // impeller eye, inner diameter: the nozzle lip goes inside it
blower_motor_d  = is_value ? 62 : 72;     // rotor can (rotates: the hub must not touch it)
blower_pcd      = is_value ? 50 : 58;     // M4 tapped holes in the motor back face, 4 x 90 deg
blower_holes    = 4;
blower_hole_d   = 4.5;   // M4 clearance
blower_flange_d = is_value ? 58 : 66;     // stationary back-face contact diameter

/* [Head layout] */
blower_max_d   = 200;
blower_max_h   = is_value ? 65 : 95;
deck_t         = 6;      // must exceed the 4.3 mm socket so the deck laps into the wall
deck_bore_d    = 140;    // through-hole in the deck
ring_seat_d    = 156;    // counterbore for the inlet ring flange
ring_seat_t    = 3;
imp_gap        = 6;      // deck top to shroud bottom
nozzle_overlap = 2.5;    // nozzle lip inside the eye (ebm-papst's own nozzle overlaps 2 mm)
nozzle_clear   = 1.5;    // radial clearance between nozzle lip and eye
nozzle_wall    = 1.6;
bridge_t       = 10;
island_drop    = (pod_h + skin - cap_h) + tray_h;   // control pod + tray hanging below the head top
head_clear     = 6;

imp_z0    = deck_t + imp_gap;
seat_z    = imp_z0 + blower_max_h;                    // bridge seats (fixed)
hub_drop  = seat_z - (imp_z0 + blower_h);             // how far the bridge hub reaches down to the flange
head_h    = seat_z + bridge_t + head_clear + island_drop;
r_nozzle  = blower_eye_d / 2 - nozzle_clear - nozzle_wall;   // throat (inner) radius; lip OD = eye - 2 x clear

assert(blower_imp_d <= blower_max_d, "blower too wide for the head");
assert(blower_h <= blower_max_h, "blower too tall: raise blower_max_h and re-export head_body");

module deck() {
    rotate_extrude($fn = 240)
        polygon([
            [deck_bore_d / 2, 0], [r_in - 0.05, 0], [r_in - 0.05, 4.6], [r_in + 1.5, 4.6], [r_in + 1.5, deck_t],
            [ring_seat_d / 2, deck_t], [ring_seat_d / 2, deck_t - ring_seat_t], [deck_bore_d / 2, deck_t -
ring_seat_t],
        ]);
}

module top_insert_boss(a, ztop, h = 14) {
    w = 11;
    rotate([0, 0, a - 90])
    difference() {
        union() {
            translate([-w / 2, r_boss_in, ztop - h]) cube([w, r_in - r_boss_in + 0.2, h]);
            hull() {
                translate([-w / 2, r_boss_in, ztop - h]) cube([w, r_in - r_boss_in + 0.2, eps]);
                translate([-w / 2, r_in - 0.2, ztop - h - (r_in - r_boss_in)]) cube([w, 0.4, eps]);
            }
        }
        translate([0, r_boss_mid, ztop - insert_depth]) cylinder(d = insert_d, h = insert_depth + eps, $fn = 32);
    }
}

bridge_seat_angles = [22.5, 112.5, 202.5, 292.5];
ring_screw_angles  = [30, 150, 270];

module head_body() {
    difference() {
        union() {
            tube(r_out, r_in, head_h);
            deck();
            for (a = boss_angles) {
                insert_boss(a, 0);
                top_insert_boss(a, head_h);
            }
            for (a = bridge_seat_angles) top_insert_boss(a, seat_z, 12);
            spigot(head_h);
        }
```

```
        socket_cut(0, chamfer = false);
        for (a = boss_angles) rotate([0, 0, a - 90])
            translate([0, r_boss_mid, -eps]) cylinder(d = insert_d, h = insert_depth, $fn = 32);
        // gasket guide grooves on the deck underside (two rings of 9 mm foam tape)
        for (r = gasket_r) translate([0, 0, -eps]) tube(r + 5, r - 5, 0.4);
        // wire pass from the shell annulus into the head (seal with hot glue)
        rotate([0, 0, 270 - 90]) translate([0, r_in - 6, -1]) cylinder(d = 7, h = deck_t + 2, $fn = 32);
        // pilot holes for the inlet ring screws (M3 self-tapping)
        for (a = ring_screw_angles) rotate([0, 0, a]) translate([ring_seat_d / 2 - 5, 0, 0.8]) cylinder(d = 2.6, h =
deck_t, $fn = 16);
    }
}

// Inlet ring: flange in the deck counterbore, converging 45-degree mouth, vertical lip into the blower eye.
module inlet_ring() {
    t = ring_seat_t - 0.2;
    lip_top = imp_z0 + nozzle_overlap - (deck_t - ring_seat_t);
    r_mouth = deck_bore_d / 2;
    rise = abs(r_mouth - r_nozzle);
    difference() {
        rotate_extrude($fn = 240)
            polygon([
                [r_mouth, 0], [ring_seat_d / 2 - 0.3, 0], [ring_seat_d / 2 - 0.3, t],
                [max(r_nozzle, r_mouth) + nozzle_wall, t], [r_nozzle + nozzle_wall, max(t, rise)],
                [r_nozzle + nozzle_wall, lip_top], [r_nozzle, lip_top], [r_nozzle, rise],
            ]);
        for (a = ring_screw_angles) rotate([0, 0, a]) translate([ring_seat_d / 2 - 5, 0, -1]) {
            cylinder(d = 3.2, h = t + 2, $fn = 16);
            translate([0, 0, t - 1.6]) cylinder(d1 = 3.2, d2 = 6.2, h = 1.6 + eps, $fn = 16);
        }
    }
}

// Blower bridge: four spokes from the seats to a hollow hub. The motor's back face bolts to the
// hub's floor; the screws are driven from above, down through the hub's bore.
hub_floor_t = 8;
hub_boss_t  = 3;     // contact boss proud of the floor, so the floor clears the spinning rotor can

module blower_bridge() {
    hub_id = blower_pcd + 12;
    hub_od = max(hub_id + 10, blower_motor_d + 8);
    spoke_w = 14;
    r_tip = r_boss_mid + 7;
    z_floor = -hub_drop;                        // underside of the contact boss = motor back face
    difference() {
        union() {
            intersection() {
                for (a = bridge_seat_angles) rotate([0, 0, a - 90])
                    translate([-spoke_w / 2, 0, 0]) cube([spoke_w, r_tip, bridge_t]);
                cylinder(r = r_in - 0.6, h = bridge_t);
            }
            translate([0, 0, z_floor + hub_boss_t]) cylinder(d = hub_od, h = hub_drop - hub_boss_t + bridge_t);
            translate([0, 0, z_floor]) cylinder(d = blower_flange_d, h = hub_boss_t + eps);
        }
        translate([0, 0, z_floor + hub_boss_t + hub_floor_t]) cylinder(d = hub_id, h = hub_drop + bridge_t);
        translate([0, 0, z_floor - 1]) cylinder(d = 26, h = hub_drop + bridge_t + 2);
        for (k = [0 : blower_holes - 1]) rotate([0, 0, 45 + k * 360 / blower_holes])
            translate([blower_pcd / 2, 0, z_floor - 1]) cylinder(d = blower_hole_d, h = hub_boss_t + hub_floor_t + 2,
$fn = 32);
        // cable notches between the bolts (the ebm-papst lead exits 29 mm off-axis)
        for (k = [0 : blower_holes - 1]) rotate([0, 0, k * 360 / blower_holes])
            translate([blower_pcd / 2, 0, z_floor - 1]) cylinder(d = 8, h = hub_boss_t + hub_floor_t + 2, $fn = 32);
        for (a = bridge_seat_angles) rotate([0, 0, a - 90]) translate([0, r_boss_mid, -1]) {
            cylinder(d = screw_d, h = bridge_t + 2, $fn = 32);
            translate([0, 0, bridge_t + 1 - 3]) cylinder(d = screw_head_d, h = 4, $fn = 32);
        }
    }
}

// ------------------------------------------------------------------ assembly preview

module blower_dummy() {
    color("DimGray") {
        translate([0, 0, imp_z0]) difference() {
            cylinder(d = blower_imp_d, h = blower_h * 0.75);
            translate([0, 0, -1]) cylinder(d = blower_eye_d, h = 10);
        }
        translate([0, 0, imp_z0 + blower_h * 0.75]) cylinder(d = blower_motor_d, h = blower_h * 0.25 - hub_boss_t);
    }
}

module assembly(explode = 0) {
    color("SlateGray") base();
    translate([0, 0, lower_z + explode]) color("Gainsboro") shell_lower();
    translate([0, 0, upper_z + 2 * explode]) color("Gainsboro") shell_upper();
    translate([0, 0, explode * 0.5]) filter_dummy();
    translate([0, 0, deck_z + 3 * explode]) {
        color("Gainsboro") head_body();
```

```
        translate([0, 0, deck_t - ring_seat_t + explode * 0.1]) color("SlateGray") inlet_ring();
        translate([0, 0, explode * 0.2]) blower_dummy();
        translate([0, 0, seat_z + explode * 0.3]) color("SlateGray") blower_bridge();
        translate([0, 0, head_h + explode * 0.6]) {
            color("DarkSlateGray") top_cap();
            translate([0, 0, cap_h - skin - 1.6 + 0.01]) color("White") control_disc();
            translate([0, 0, cap_h - pod_h - skin - tray_h - explode * 0.2]) color("SlateGray") electronics_tray();
        }
    }
}
```

cad/sidetable.scad (283 lines)

```
// OpenAir Side Table: squircle "side table" enclosure around the value-build internals.
// Same filter, base bayonet, deck seal, inlet ring and blower bridge geometry as the tower;
// fabric sleeve intake, louvred outlet under a flat tabletop, front light line + touch pads.
// Units: mm. World origin: floor centre, +Y = front (controls), -Y = back.

include <openair.scad>
build  = "value";
deck_t = 4;            // no socket joint here, so the tower's 6 mm deck isn't needed

/* [Side table] */
st_w        = 250;     // outer width and depth (fits a 256 mm bed)
st_n        = 5;       // superellipse exponent (2 = circle, larger = squarer)
st_wall     = 2.0;
st_ring_h   = 34;      // lower frame ring: bayonet, power input, hall sensor
st_rod      = 95;      // corner posts at x = y = +/-95
st_post     = 12;      // square corner posts, printed lying flat
st_sock     = 10;      // socket depth at each end
st_boss     = 19;      // square boss around each socket
st_louvre_h = 26;
st_rim_h    = 5;
st_top_t    = 4.5;
st_groove   = 2.5;     // elastic-cord groove (3 mm round elastic)
st_led_len  = 168;     // 10 LEDs from a 60/m WS2812B strip
st_pads_x   = [52, 26, 0, -26, -52];   // power, mode, speed, timer, lock: left to right seen from the front
st_usb_a    = 250;     // USB-C PD input, back face left of centre
st_board    = [70, 50];

st_ring_z   = base_h;
st_seat_z   = deck_t + imp_gap + blower_max_h;            // bridge seats, head frame
st_head_h   = st_seat_z + bridge_t + 2 + st_louvre_h + st_rim_h;
st_louvre_z = st_head_h - st_rim_h - st_louvre_h;
st_ctrl_z   = st_louvre_z - 22;                           // touch-pad row
st_led_z    = st_louvre_z + st_louvre_h / 2;              // light line, level with the louvres
st_rod_r    = st_rod * sqrt(2);
st_post_len = deck_z - (st_ring_z + st_ring_h) + 2 * st_sock - 0.6;
st_seat_off = 7;       // bridge screw sits 7 mm inboard of the post axis (the post screw uses the centre)

function se_pt(a, n, t) = let(c = cos(t), s = sin(t))
    [a * sign(c) * pow(abs(c), 2 / n), a * sign(s) * pow(abs(s), 2 / n)];
function se_r(a, n, t) = 1 / pow(pow(abs(cos(t)) / a, n) + pow(abs(sin(t)) / a, n), 1 / n);   // boundary radius at
polar angle t
function st_front_y(x) = st_w / 2 * pow(1 - pow(abs(x) / (st_w / 2), st_n), 1 / st_n);

module squircle(w, n = st_n, N = 240) {
    polygon([for (i = [0 : N - 1]) se_pt(w / 2, n, 360 * i / N)]);
}

module rod_xy() { for (sx = [-1, 1], sy = [-1, 1]) translate([sx * st_rod, sy * st_rod]) rotate([0, 0, atan2(sy, sx)])
children(); }
module post_sq(d) { rotate([0, 0, 45]) square(d, center = true); }

// ------------------------------------------------------------------ base (twist-lock plinth)

module st_base() {
    difference() {
        union() {
            hull() {
                linear_extrude(6) offset(delta = -6) squircle(st_w);
                translate([0, 0, 12]) linear_extrude(eps) squircle(st_w);
            }
            translate([0, 0, 12 - eps]) linear_extrude(base_h - 12 + eps) squircle(st_w);
            translate([0, 0, base_h - eps]) tube(r_collar_o, r_collar_i, collar_h);
            translate([0, 0, base_h - eps]) cylinder(r = r_collar_i + eps, h = platform_z - base_h);
            for (i = [0 : 3])
                arc_block(filter_od / 2 + 2, filter_od / 2 + 4.4, i * 90 + 12, i * 90 + 78, platform_z - 1, platform_z
+ 8);
            for (i = [0 : 2]) lug(i * 120);
        }
        for (i = [0 : 3])
            rotate([0, 0, 45 + i * 90]) translate([0, r_out - 26, -eps]) cylinder(d = foot_d, h = 1.2);
        rotate([0, 0, 270 - 90])
            translate([0, r_collar_o - magnet_t + eps, base_h + 5])
                rotate([-90, 0, 0]) cylinder(d = magnet_d, h = magnet_t + 0.1, $fn = 32);
```

```
        for (s = [-1, 1])
            translate([s * 10, st_w / 2 - 0.6, base_h - 8]) rotate([-90, 0, 0])
                linear_extrude(1) polygon(s > 0 ? [[-3, 0], [3, 2.5], [3, -2.5]] : [[3, 0], [-3, 2.5], [-3, -2.5]]);
        rotate([0, 0, 90]) translate([0, filter_od / 2 - 8, platform_z - 2.5]) cylinder(d = 22, h = 10);
    }
}

// ------------------------------------------------------------------ lower ring

module st_groove_cut(z) {
    translate([0, 0, z]) linear_extrude(st_groove)
        difference() { squircle(st_w + 2); offset(delta = -st_groove) squircle(st_w); }
}

module st_ring() {
    h = st_ring_h;
    difference() {
        union() {
            difference() {
                linear_extrude(h) squircle(st_w);
                translate([0, 0, -1]) cylinder(r = r_band, h = h + 2);
                // hollow from below, leaving the outer wall, the bayonet band and a 3 mm top skin
                translate([0, 0, -eps]) linear_extrude(h - 3) difference() {
                    offset(delta = -st_wall) squircle(st_w);
                    circle(r = r_out);
                }
            }
            linear_extrude(h) rod_xy() post_sq(st_boss);
            // housing for the USB-C PD trigger board
            rotate([0, 0, st_usb_a]) translate([r_out - 1, -11, 0]) cube([se_r(st_w / 2, st_n, st_usb_a) - r_out - 1,
22, h - 3]);
        }
        rod_xy() {
            translate([0, 0, h - st_sock]) linear_extrude(st_sock + 1) post_sq(st_post + 0.4);
            translate([0, 0, -1]) cylinder(d = screw_d, h = h + 2, $fn = 24);
            translate([0, 0, -1]) cylinder(d = screw_head_d, h = h - st_sock - 4 + 1, $fn = 24);
        }
        bayonet_band_cuts();
        hall_pocket();
        st_groove_cut(h - 8);
        // USB-C receptacle slot + board pocket (board slid in from below and glued)
        rotate([0, 0, st_usb_a]) {
            translate([r_out + 2, -8, -eps]) cube([se_r(st_w / 2, st_n, st_usb_a) - st_wall - r_out - 3.5, 16, h -
6]);
            translate([r_out + 2, 0, 18]) rotate([0, 90, 0])
                linear_extrude(se_r(st_w / 2, st_n, st_usb_a)) offset(r = usbc_slot[1] / 2 - 0.01) square([0.02,
usbc_slot[0] - usbc_slot[1]], center = true);
            translate([r_out + 8, 0, h - 10]) cylinder(d = 6, h = 12, $fn = 24);
        }
    }
    bayonet_detents();
}

// ------------------------------------------------------------------ head

module st_slot_cut(z0, h) {
    pitch = 3.5;
    for (a = [135 + pitch / 2 : pitch : 405 - pitch / 2])
        rotate([0, 0, a]) translate([se_r(st_w / 2, st_n, a) - 10, -2.75, z0]) cube([14, 5.5, h]);
}

module st_head() {
    h = st_head_h;
    difference() {
        union() {
            // deck: 2.4 mm plate, 4 mm boss where the inlet ring seats, ribs on the plenum side
            linear_extrude(2.4) squircle(st_w);
            cylinder(r = ring_seat_d / 2 + 6, h = deck_t);
            intersection() {
                for (k = [0 : 7]) rotate([0, 0, k * 45 + 22.5]) translate([ring_seat_d / 2 + 4, -1, 0]) cube([st_w, 2,
6]);
                linear_extrude(6) offset(delta = -st_wall / 2) squircle(st_w);
            }
            linear_extrude(h) difference() { squircle(st_w); offset(delta = -st_wall) squircle(st_w); }
            // corner columns around the rods, webbed to the wall, carrying the bridge
            linear_extrude(st_seat_z) rod_xy() post_sq(st_boss);
            for (a = [45, 135, 225, 315]) rotate([0, 0, a])
                translate([st_rod_r, -1.6, 0]) cube([se_r(st_w / 2, st_n, a) - st_rod_r - 1, 3.2, st_seat_z]);
            // card guides for the 70 x 50 board, standing up behind the front controls
            for (sx = [-1, 1]) translate([sx * (st_board[0] / 2 + 1.5) - 3, st_w / 2 - st_wall - 22, 0])
                difference() {
                    cube([6, 23, deck_t + st_board[1] + 4]);
                    translate([3 - sx * 2.5 - 1, 9, deck_t]) cube([2.0 + 0.2, 2.0, st_board[1] + 10]);
                }
        }
        // deck: bore, inlet-ring counterbore, ring screws, gasket grooves
        translate([0, 0, -1]) cylinder(d = deck_bore_d, h = deck_t + 2);
        translate([0, 0, deck_t - ring_seat_t]) cylinder(d = ring_seat_d, h = ring_seat_t + 1);
```

```
        for (a = ring_screw_angles) rotate([0, 0, a]) translate([ring_seat_d / 2 - 5, 0, 0.8]) cylinder(d = 2.6, h =
deck_t, $fn = 16);
        for (r = gasket_r) translate([0, 0, -eps]) tube(r + 5, r - 5, 0.4);
        // rods: through the columns, nut traps at the top of each column
        rod_xy() {
            translate([0, 0, -1]) linear_extrude(st_sock + 1) post_sq(st_post + 0.4);
            translate([0, 0, st_sock - 1]) cylinder(d = screw_d, h = 6, $fn = 24);
            translate([0, 0, st_sock + 3]) cylinder(d = screw_head_d, h = st_seat_z, $fn = 24);
            translate([-st_seat_off, 0, st_seat_z - 10]) cylinder(d = 2.8, h = 11, $fn = 16);
        }
        // wire pass from the sleeve zone (back-left corner)
        rotate([0, 0, 225]) translate([st_rod_r - 14, 0, -1]) cylinder(d = 7, h = deck_t + 2, $fn = 24);
        // fabric groove just above the deck
        st_groove_cut(5);
        // outlet louvres on the back and sides
        st_slot_cut(st_louvre_z, st_louvre_h);
        // front: light line (0.8 mm skin), touch-pad recesses and icons, light-sensor hole
        translate([0, 0, st_led_z - 5.5]) linear_extrude(11) intersection() {
            difference() { offset(delta = -0.8) squircle(st_w); offset(delta = -st_wall - 0.5) squircle(st_w); }
            translate([-st_led_len / 2, 0]) square([st_led_len, st_w]);
        }
        for (i = [0 : len(st_pads_x) - 1]) let(x = st_pads_x[i], yf = st_front_y(x)) translate([x, 0, st_ctrl_z]) {
            translate([-6, yf - st_wall - 0.5, -6]) cube([12, st_wall - 1.2 + 0.5, 12]);
            translate([0, yf + eps, 0]) rotate([90, 0, 0]) linear_extrude(0.5 + eps) rotate(180) mirror([0, 1, 0])
scale(0.9) pad_icon(i);
        }
        translate([-72, st_front_y(-72) + 1, st_ctrl_z]) rotate([90, 0, 0]) cylinder(d = 2.6, h = st_wall + 3, $fn =
20);
    }
}

// ------------------------------------------------------------------ blower bridge on the rods

module st_bridge() {
    hub_id = blower_pcd + 12;
    hub_od = max(hub_id + 10, blower_motor_d + 8);
    z_floor = -hub_drop;
    difference() {
        union() {
            for (a = [45, 135, 225, 315]) rotate([0, 0, a]) {
                translate([0, -7, 0]) cube([st_rod_r, 14, bridge_t]);
                translate([st_rod_r, 0, 0]) cylinder(d = 18, h = bridge_t, $fn = 40);
            }
            translate([0, 0, z_floor + hub_boss_t]) cylinder(d = hub_od, h = hub_drop - hub_boss_t + bridge_t);
            translate([0, 0, z_floor]) cylinder(d = blower_flange_d, h = hub_boss_t + eps);
        }
        translate([0, 0, z_floor + hub_boss_t + hub_floor_t]) cylinder(d = hub_id, h = hub_drop + bridge_t);
        translate([0, 0, z_floor - 1]) cylinder(d = 26, h = hub_drop + bridge_t + 2);
        for (k = [0 : blower_holes - 1]) rotate([0, 0, 45 + k * 360 / blower_holes])
            translate([blower_pcd / 2, 0, z_floor - 1]) cylinder(d = blower_hole_d, h = hub_boss_t + hub_floor_t + 2,
$fn = 32);
        for (k = [0 : blower_holes - 1]) rotate([0, 0, k * 360 / blower_holes])
            translate([blower_pcd / 2, 0, z_floor - 1]) cylinder(d = 8, h = hub_boss_t + hub_floor_t + 2, $fn = 32);
        rod_xy() translate([-st_seat_off, 0, -1]) {
            cylinder(d = screw_d, h = bridge_t + 2, $fn = 24);
            translate([0, 0, bridge_t + 1 - 3]) cylinder(d = screw_head_d, h = 4, $fn = 24);
        }
    }
}

// ------------------------------------------------------------------ corner post (print 4, lying flat)

module st_post() {
    difference() {
        translate([0, 0, st_post / 2]) rotate([0, 90, 0])
            linear_extrude(st_post_len, center = true) offset(r = 1) offset(delta = -1) square(st_post, center =
true);
        for (sx = [-1, 1]) translate([sx * (st_post_len / 2 - 8), 0, st_post / 2]) rotate([0, sx * 90, 0])
            cylinder(d = 2.8, h = 9, $fn = 16);
    }
}

// ------------------------------------------------------------------ tabletop

module st_top() {
    difference() {
        union() {
            hull() {
                linear_extrude(eps) offset(delta = -1.5) squircle(st_w);
                translate([0, 0, 1.5]) linear_extrude(st_top_t - 1.5) squircle(st_w);
            }
            // locating lip that drops inside the head's rim
            translate([0, 0, st_top_t - eps]) linear_extrude(3)
                difference() { offset(delta = -st_wall - 0.4) squircle(st_w); offset(delta = -st_wall - 3.4)
squircle(st_w); }
        }
    }
}
```

```
// Wooden-top cutting template (export as SVG/DXF at 1:1).
module st_top_template() { squircle(st_w); }

// ------------------------------------------------------------------ PMS5003 cradle (zip-tied to a rod)

module st_pms_clip() {
    s = [50.6, 38.6, 21.4];
    difference() {
        union() {
            translate([-s[0] / 2 - 2, -2, 0]) cube([s[0] + 4, s[2] + 2, s[1] / 2 + 2]);
            translate([-st_post / 2 - 3, -st_post - 3, 0]) cube([st_post + 6, st_post + 3, s[1] / 2 + 2]);
        }
        translate([-s[0] / 2, 0, 2]) cube([s[0], s[2] + 1, s[1]]);
        translate([-st_post / 2 - 0.2, -st_post - 0.6, -1]) cube([st_post + 0.4, st_post + 0.4, s[1]]);
        for (z = [5, 15]) translate([-st_post / 2 - 4, -st_post - 4, z]) cube([st_post + 8, 2, 4]);
        translate([-s[0] / 2 + 6, -3, 4]) cube([12, 4, 6]);
    }
}

// ------------------------------------------------------------------ assembly preview

module st_rods() {
    color("WhiteSmoke") rod_xy() translate([0, 0, st_ring_z + st_ring_h - st_sock + 0.3])
        linear_extrude(st_post_len) post_sq(st_post);
}

module st_sleeve() {
    color("Gainsboro", 0.92) translate([0, 0, st_ring_z + st_ring_h - 8])
        linear_extrude(deck_z + 7.5 - (st_ring_z + st_ring_h - 8))
            difference() { offset(delta = -0.2) squircle(st_w); offset(delta = -1.4) squircle(st_w); }
}

module st_assembly(explode = 0, sleeve = true) {
    color("WhiteSmoke") st_base();
    translate([0, 0, st_ring_z + explode * 0.6]) color("WhiteSmoke") st_ring();
    translate([0, 0, explode * 0.3]) filter_dummy();
    if (sleeve && explode == 0) st_sleeve();
    translate([0, 0, explode * 1.1]) st_rods();
    translate([0, 0, deck_z + explode * 1.6]) {
        color("WhiteSmoke") st_head();
        translate([0, 0, deck_t - ring_seat_t]) color("SlateGray") inlet_ring();
        blower_dummy();
        translate([0, 0, st_seat_z + explode * 0.3]) color("SlateGray") st_bridge();
        translate([0, 0, st_head_h + st_top_t + explode * 0.6]) mirror([0, 0, 1]) color("BurlyWood") st_top();
    }
}
```